Beyond the Linear Barrier: Secret Sharing for Evolving (Weighted) Threshold Access Structures with Poly-logarithmic Share Size

Danilo Francati[1], Sara Giammusso[1,2], and Daniele Venturi[1]

[1] Sapienza University of Rome, Rome, Italy
[2] Banca d'Italia, Rome, Italy

Abstract. Evolving secret sharing allows a dealer to share a secret message between a growing number of n parties. Crucially, the dealer does not know an upper bound on n, neither it knows the access structure before party n arrives; furthermore, the dealer is not allowed to update the shares of old parties.

We construct new secret sharing schemes for so-called evolving (weighted) threshold access, in which the arrival of party n determines the number of parties $t_n \leq n$ that are required in order to reconstruct the secret. We also consider the more general case in which party n has associated a weight w_n with logarithmic size, and the authorized subsets of parties are those for which the sum of the corresponding weights exceeds the current threshold t_n. In particular, we obtain:

- A secret sharing scheme for evolving threshold access structures with *adaptive* privacy in the *plain model*, and with share size $\mathsf{poly}(\lambda, \log n)$. This construction requires one-way functions (OWFs) and indistinguishability obfuscation (iO) for Turing machines.
- A secret sharing scheme for evolving weighted threshold access structures with *adaptive* privacy in the *plain model*, and with share size $\mathsf{poly}(\lambda, \log W_n)$ where W_n is the sum of the weights up to party n. This construction requires OWFs and iO for Turing machines, and additionally assumes that the weights are fixed (i.e., cannot change over time).
- A secret sharing scheme for evolving weighted threshold access structures with *static* privacy in the *plain model*, and with share size $\mathsf{poly}(\lambda, \log W_n)$. This construction allows the weight of old parties to change over time, but it requires somewhere statistically binding hash functions and achieves only static privacy.

Previous constructions of secret sharing schemes for evolving (weighted) threshold access structures achieved (much worse) share sizes linear in W_n (and in the security parameter) and, when considering adaptive privacy, they require the random oracle model.

Keywords: evolving secret sharing; adaptive security; computational security; threshold access structures.

1 Introduction

A secret sharing scheme for an access structure $\mathcal{A}_n$ allows a trusted dealer to share a secret message μ among (a fixed set of) n parties, in such a way that the following two properties are met: (i) any *authorized* subset of parties $\mathcal{I} \in \mathcal{A}_n$ can recover the secret message by pulling together their shares; (ii) any *unauthorized* subset of parties $\mathcal{U} \notin \mathcal{A}_n$ obtains zero information about the secret message. Property (i) is usually referred to as *correctness*, whereas property (ii) is usually known as *privacy*, which can be either computational or statistical depending on the power of the adversary. Secret sharing has been introduced independently by Blakley [5] and Shamir [24], and is an essential ingredient in many cryptographic applications, including threshold cryptography and multi-party computation.

The most prominent example of access structure $\mathcal{A}_n$ is the so-called *threshold* access structure, in which the authorized parties consist of all subsets of at least $t \leq n$ parties. The latter can be generalized by associating to party i a *weight* w_i, so that a subset $\mathcal{I}$ of parties is authorized if and only if $\sum_{i \in \mathcal{I}} w_i \geq t$; in this case, we call $\mathcal{A}_n$ a *weighted* threshold access structure.

Evolving secret sharing. The above definition of secret sharing assumes that the number of parties $n \in \mathbb{N}$ is fixed. The setting of *evolving* access structures, where n grows and the access structure can change over time, has been considered for the first time by Komargodski, Naor and Yogev [19]. In the information-theoretic setting, we can think of an evolving access structure as an infinite sequence of collections of subsets $\mathcal{A} = \{\mathcal{A}_n\}_{n \geq 1}$, where $\mathcal{A}_n$ captures the authorized subsets when there are n parties. Clearly, an evolving access structure must be *monotone*: if a subset $\mathcal{I} \in \mathcal{A}_n$ is authorized when there are n parties, then $\mathcal{I}$ will also be authorized when there are $n' > n$ parties.[3] Furthermore, an evolving access structure must be *rigid*: if a subset $\mathcal{U} \notin \mathcal{A}_n$ is unauthorized when there are n parties, then $\mathcal{I}$ will also be unauthorized when there are $n' > n$ parties. Rigidity is essential in case the dealer is not allowed to update the shares of old parties (which is desired).

Francati and Venturi [13] were the first to consider evolving secret sharing in the *computational* setting. Here, the privacy property holds only computationally. Moreover, the number of parties n is an *unbounded* polynomial in the underlying security parameter $\lambda \in \mathbb{N}$. Note, in particular, that the dealer does not know an upper bound on n, neither it knows the current access structure $\mathcal{A}_n$ before the n-th party arrives. In [13], the authors provide constructions of secret sharing schemes for several evolving access structures, under standard hardness assumptions. In particular, assuming one-way functions (OWFs), they construct a secret sharing scheme for the so-called *evolving threshold* access structure, which can be represented as a sequence of thresholds $\mathcal{A}_n = \{t_n\}_{n \geq 1}$, in such a way that the authorized subsets at time n consist of all the subsets of at least t_n

[3] Additionally, if $\mathcal{I} \in \mathcal{A}_n$ is authorized, the same holds true for any superset $\mathcal{I}' \supset \mathcal{I}$; this is the usual monotonicity requirement for non-evolving secret sharing.

parties always including[4] the n-th party. The share size of party n in this construction is $\lambda \cdot (n+1)$; in contrast, the best information-theoretic construction achieves a much worse share size of $O(n^4)$ [25].

In a follow-up work, Francati, Giammusso and Venturi [12], assuming OWFs, gave a construction for the more general case of *evolving weighted threshold* access structures $\mathcal{A}_n = \{(w_n, t_n)\}_{n \geq 1}$, where a subset $\mathcal{I}$ of parties is authorized if and only if $\sum_{i \in \mathcal{I}} w_i \geq t_n$.[5] The share size of party n in this construction is $\lambda \cdot (1+W_n)$, where W_n is the sum of the weights up to the n-th party.

All of the above mentioned constructions only achieve *non-adaptive* privacy, in which the attacker must decide the corrupted parties all at the same time. Francati, Giammusso and Venturi [12] were the first to consider *adaptive* computational privacy in the setting of evolving secret sharing, meaning that the attacker can corrupt parties incrementally, in an adaptive fashion. Assuming trapdoor permutations, they also gave a construction of a secret sharing scheme for the evolving weighted threshold access structure with adaptive privacy in the random oracle model (ROM). The share size of party n in this construction is $\lambda \cdot (n + W_n)$.

We note that in the non-evolving setting, the share size for (weighted) threshold access structures is much lower. Indeed, Shamir's original scheme for threshold access structures achieves share size $O(\log n)$. Moreover, in a recent work, Devadas, Jain, Waters and Wu [10] gave a construction of secret sharing for weighted threshold access structures with share size $\mathsf{poly}(\lambda, \log W)$, assuming OWFs and indistinguishability obfuscation (iO) for circuits. Moreover, their construction achieves only static security. In light of the above discussion, the following question remains open:

> *Can we construct secret sharing schemes for the* evolving *(weighted) threshold access structure with* adaptive *(or* static*) privacy in the* plain model*, and with* poly-logarithmic *share size?*

We remark that, prior to our work, in the evolving setting, secret sharing schemes with *succinct* shares (i.e., with share size much smaller than the size of some natural computational representation of the access structures) are known only for very limited access structures such as graphs access structures and access structures represented by formulas in conjunctive normal form [13].

1.1 Our Contributions

In this paper, we provide a positive answer to the above question. Our first contribution is a construction of a secret sharing scheme for evolving threshold access structures with share size $\mathsf{poly}(\lambda, \log n)$, assuming OWFs and iO for Turing machines.

[4] If we do not include the n-th party, the sequence of thresholds must be increasing, i.e. $t_1 \leq t_2 \leq \cdots \leq t_n$

[5] In fact, they consider the more general case where the weights of old parties change over time.

Theorem 1 (Informal). *Assuming the existence of OWFs and iO for Turing machines (both with polynomial security), there is a construction of secret sharing scheme for evolving threshold access structures with adaptive privacy in the plain model, and with share size* $\mathsf{poly}(\lambda, \log n)$.

Our second contribution is for the more general case of weighted threshold access structures, under the same assumptions. We first consider the case in which weights are of size $O(\log \lambda)$ and the weight of party n is fixed when the n-th party arrives (and does not change in the future).

Theorem 2 (Informal). *Assuming the existence of OWFs and iO for Turing machines (both with polynomial security), there is a construction of a secret sharing scheme for evolving weighted threshold access structures (with fixed weights of size at most* $O(\log \lambda)$*) with adaptive privacy in the plain model, and with share size* $\mathsf{poly}(\lambda, \log W_n)$.

As our third, and last, contribution, we consider the more general case in which the $O(\log \lambda)$-bits weights are not fixed. This construction additionally requires somewhere statistically binding (SSB) hash functions, but we are able to prove its security only in the static setting.

Theorem 3 (Informal). *Assuming the existence of SSB hash functions and iO for Turing machines (both with polynomial security), there is a construction of a secret sharing scheme for evolving weighted threshold access structures (with adaptive weights of size at most* $O(\log \lambda)$*) with static privacy in the plain model, and with share size* $\mathsf{poly}(\lambda, \log W_n)$.

Once again, we highlight that both our constructions for weighted threshold access structures support weights of $O(\log \lambda)$ bits. We leave it as an open problem to remove this assumption.

1.2 Technical Overview

In this section we provide a high level overview of the techniques underlying our constructions. We begin by briefly recalling the notion of evolving secret sharing (ESS), a variant of secret sharing in which parties arrive one by one and the dealer does not know the final number of participants in advance. At each step, the dealer distributes a new share, without updating the already distributed shares. We then focus on evolving threshold access structures, in which authorization is determined by a *threshold that may change over time*, always associated with the party of maximum index in the reconstructing set. We consider three variants of this notion: *evolving thresholds without weights*, where authorization depends only on the size of the reconstructing set; *evolving thresholds with fixed weights*, where each party is assigned a static weight and reconstruction depends on the sum of weights; and *evolving thresholds with dynamic weights*, where both thresholds and party weights may evolve over time, subject to the monotonicity and rigidity conditions. Our three constructions correspond exactly to these three settings and progressively generalize each other. For the

first two settings, we prove adaptive computational privacy, while for the third setting we only prove static computational privacy. Moreover, we obtain succinct share sizes: polynomial in $(\lambda, \log n)$ for evolving thresholds without weights, and polynomial in $(\lambda, \log W_n)$ for both fixed weights and dynamic weights evolving thresholds, where W_n denotes the sum of all parties' weights at time n.

Previous approach: "secret sharing to the past." We start by recalling the main ideas behind the previous constructions of ESS for (weighted) threshold access structures [13,12]. Here, when party n arrives, the dealer computes a fresh (non-evolving) secret sharing of the message μ, yielding shares $\mathsf{sh}_1, \ldots, \mathsf{sh}_n$. Hence, the share of party n consists of sh_n along with $n-1$ encryptions $\mathsf{ct}_1, \ldots, \mathsf{ct}_{n-1}$ of the shares $\mathsf{sh}_1, \ldots, \mathsf{sh}_{n-1}$, where ct_i is encrypted using the secret key of the i-th party.

The reconstruction algorithm decrypts the ciphertexts contained in the share of party n, where n is now the maximal index contained in the authorized set $\mathcal{I}$, decrypts the ciphertexts corresponding to the parties in $\mathcal{I}$, and runs the reconstruction algorithm of the underlying (non-evolving) secret sharing scheme. Clearly, the above approach always yields shares of size linear in n, as the n-th share contains the shares of all the previous parties (in encrypted form). We refer to this approach as *"secret sharing to the past."*

We also emphasize that, in the non-evolving setting, there already exist secret sharing schemes for weighted threshold access structures with succinct shares [10]. However, as we discuss in section 2, these schemes cannot be adapted to the evolving setting, as they suffer from the same limitation as the "secret sharing to the past" approach. We refer the reader to section 2 for more details.

Our new approach: "secret sharing to the future." We introduce a new technique that fundamentally departs from the *"secret sharing to the past"* approach. Our approach satisfies the invariant that assigning a share to a newly joining party does not require embedding any additional information to ensure compatibility with the past. On the contrary, the newly generated shares are already compatible with parties arriving in the future. We dub our approach as *"secret sharing to the future."*

We start by describing our first setting: evolving threshold without weights. At a high level, when a new party n arrives, the dealer is going to encrypt the message μ using a secret key K_n, resulting in a ciphertext $\mathsf{ct}_n = K_n \cdot \mu$ which will be included in the share σ_n of the n-th party. While this requires sampling a fresh encryption key K_n, at the same time, we would like that K_n is recoverable by combining σ_n with sufficiently many shares of older parties. This suggests viewing K_n not as an independently sampled value, but as the evaluation of a function that can be reconstructed from a sufficient number of points. The central idea is to define K_n as the evaluation at zero of a polynomial f_n over an exponentially large field $\mathbb{F} = \mathrm{GF}(2^\lambda)$, namely $K_n = f_n(0)$; hence, we let each party hold an evaluation of f_n at a distinct point. However, since the shares of older parties cannot be modified, the polynomial f_n must be defined in a way that allows those parties to obtain their evaluations without any update.

We achieve this by defining f_n implicitly, by interpolating pseudorandom values of the form $\mathsf{Eval}(K_f, n||t_n||(2^\lambda - j))$ over the (binary representation of the) set of points $(2^\lambda - j) \in \mathbb{F}$ indexed by $j \in [0, t_n)$, where t_n is the threshold at time n and where Eval is the evaluation algorithm of a (puncturable) pseudorandom function (PPRF) using an independent secret key K_f. The inclusion of the pair (n, t_n) in the PPRF input guarantees that the encryption keys $\{K_i\}_{i\in[n]}$ used to encrypt the message at different times are all distinct and pseudorandom, even though they are derived from a single master key K_f. The choice of the interpolation points is not arbitrary: by placing these points at the upper end of the field $\mathbb{F}$, we intentionally leave the low positions, namely the points $i \in [n]$, unconstrained. This allows us to later assign to each party i the evaluation $f_n(i)$, computed via Lagrange interpolation. Moreover, since the number of parties satisfies $n = \mathsf{poly}(\lambda)$, the points in $[n]$ and those of the form $(2^\lambda - j)$ never overlap, ensuring that the interpolation constraints and the party evaluation points remain disjoint.

Each party i is then given an obfuscation of a Turing machine $M_p^{\mathsf{share}}[K_f, i, \mathsf{pos}, t^*, v](\cdot,\cdot)$, where K_f is the PPRF key, i is the index of the party, and pos, t^*, v are auxiliary values used only in the security proof (and set to 0 in the real construction). The share of the i-th party consists of $\sigma_i = (\mathsf{ct}_i, \widetilde{M}_i)$ where $\widetilde{M}_i$ is an obfuscation of $M_i = M_p^{\mathsf{share}}[K_f, i, 0, 0, 0](\cdot,\cdot)$. Upon input the public identifiers (n, t_n) of the polynomial, the machine $\widetilde{M}_i$ internally recomputes the defining values of f_n, namely the set $\{\mathsf{Eval}(K_f, n||t_n||(2^\lambda - j))\}_{j\in[0,t_n)}$, and outputs the evaluation of the polynomial at the party's index using Lagrange interpolation, yielding $\widetilde{M}_i(n, t_n) = f_n(i)$. To enforce the semantics of evolving threshold access structures—where the reconstruction threshold is determined by the party with maximum index in the reconstructing set—the machine additionally checks that the input parameter n is greater than or equal to the hardcoded index i; otherwise it aborts. Reconstruction is then performed by a qualified reconstructing set $\mathcal{I}$ by collecting the evaluations $\{f_n(i) = \widetilde{M}_i(n, t_n)\}_{i\in\mathcal{I}}$ and by applying Lagrange interpolation at the point 0, yielding $K'_n = f_n(0)$, which is used to decrypt the ciphertext and recover the message.

What makes this approach powerful is that the polynomial f_n is entirely determined by public identifiers along with a single PPRF key K_f. In particular, since the obfuscated programs depend only on a party's own identity and on public parameters, they remain valid for all future polynomials. Concretely, at time $n+1$, when a new party joins and the latest party becomes $n+1$, the *same* obfuscated program $\widetilde{M}_i$ can be invoked on the new input $(n+1, t_{n+1})$. Since $n+1 \geq i$, the check again passes, and the machine now computes a *different* set of defining values $\{\mathsf{Eval}(K_f, (n+1)||t_{n+1}||(2^\lambda - j))\}_{j\in[0,t_{n+1})}$ thereby outputting $\widetilde{M}_i(n+1, t_{n+1}) = f_{n+1}(i)$ via Lagrange interpolation over these points. As before, the corresponding key K'_{n+1} can be reconstructed and used to decrypt ct_{n+1}, without requiring any update to the share of older parties.

Proving security while keeping the circuit size small. We prove privacy via a hybrid argument whose end goal is to remove the message from *every share*

that the adversary may ever obtain. Since the adversary may adaptively corrupt parties, this means that for every index $i^* \in [n]$ we will show that the ciphertext component ct_{i^*} can be replaced by a random value without the attacker noticing, which in turn requires the key K_{i^*} to be indistinguishable from uniform given the adversary's entire view.

To achieve the above, we must eliminate all information about K_{i^*} that may be inferred from *other* shares. Recall that in the computational setting, the number of parties satisfies $n \in \mathsf{poly}(\lambda)$, and therefore $\omega(\log \lambda)$ bits are sufficient to encode in binary any party index n and any threshold $t_n \leq n$. With this in mind, observe that $K_{i^*} = f_{i^*}(0)$ is correlated with the evaluations $f_{i^*}(z)$ that parties $z \leq i^*$ (recall that parties $z > i^*$ would abort when reconstructing with $\max(\mathcal{I}) = i^*$) could obtain by running their obfuscated machines $M_p^{\mathsf{share}}[K_f, z, \mathsf{pos}, t^*, v]$ with input (i^*, t_{i^*}). These machines hardcode a PPRF key K_f and a party index z, and on input (i^*, t_{i^*}) recompute t_{i^*} PPRF evaluations and perform Lagrange interpolation to output $f_{i^*}(z)$. As a preparatory step, we therefore modify the shares of corrupted parties in such a way that, on input (n_g, z), if party $z \leq i^*$, we change the obfuscated machine contained in σ_z so that on the specific input $(k, t) = (i^*, t_{i^*})$ it bypasses the PPRF and interpolation code and instead returns a hardcoded value v. This freezes the observable outputs corresponding to shares of f_{i^*}, allowing us in subsequent hybrids to progressively remove all correlation between these values and the underlying PPRF key K_f, until K_{i^*} becomes uniform and the ciphertext ct_{i^*} can be randomized. Importantly, at this stage we hardcode only a *constant size* amount of information per machine, namely a single input pair (pos, t^*) of length $\omega(\log \lambda)$ and one field element $v \in \mathbb{F}$ of length $\log(\lambda + 1)$. This careful control of what is hardcoded is essential. If we were to accumulate hardcoded behaviour for many inputs (e.g., for all previously treated indices, or for an adaptively growing set), then the description of each machine—and hence the share—would reach size linear in n, destroying succinctness.

Once the behaviour on input (i^*, t_{i^*}) is frozen, we progressively remove correlation with K_{i^*}. A crucial technical ingredient that enables these transformations is the fact that an adversary with access to *at most* $t_{i^*} - 1$ evaluations of a degree-$(t_{i^*} - 1)$ polynomial, cannot distinguish whether the evaluation of the polynomial at a fresh point is sampled uniformly at random, or instead computed as a Lagrange interpolation from the existing $t_{i^*} - 1$ evaluations together with one additional randomly sampled evaluation. We formalize this property in Lemma 1, which follows by perfect privacy of Shamir secret sharing.

We then replace the t_{i^*} PPRF evaluations used to define the polynomial f_{i^*} as follows. *One* of these PPRF evaluations is swapped with the key $K_{i^*} = f_{i^*}(0)$: after puncturing the PPRF key K_f at the corresponding input $(i^* || t_{i^*} || 2^\lambda)$ (and hardcoding the punctured key to all the machines included in the shares requested by the adversary), we first leverage PPRF security to switch the value $\mathsf{Eval}(K_f, i^* || t_{i^*} || 2^\lambda)$ with a uniformly random value and then we use Lemma 1 to stop computing K_{i^*} as a Lagrange interpolation of PPRF outputs and instead treat it as an independent random field element used to interpolate the polyno-

mial f_{i^*}. For the *remaining* $t_{i^*} - 1$ *PPRF evaluations* we again apply Lemma 1 iteratively for every corrupted party $z \leq i^*$: we first puncture K_f at the corresponding input $(i^*||t_{i^*}||(2^\lambda - j^*))$ with $j^* \in [1, t_{i^*})$ and hardcode the punctured key to all machines included in the shares requested by the adversary, then when we reach the j^*-th query where a party $z \leq i^*$ is corrupted, we replace the PPRF evaluation by a uniformly random value, and then reinterpret this random value as the actual share hardcoded into the corresponding obfuscated machine. Only after all $t_{i^*} - 1$ such evaluations have been swapped with the shares that the adversary may obtain we effectively eliminate the residual correlation between K_{i^*} and the adversary's view. At that point, the ciphertext ct_{i^*} can be safely replaced by a uniformly random field element. We note that before replacing any PPRF evaluation by a random value, we must puncture the PPRF key K_f at the corresponding input because K_f is hardcoded into the obfuscated machines, and puncturing ensures that the adversary cannot evaluate the PPRF at the challenged point via the obfuscated code, thereby justifying the application of PPRF security in the hybrids.

Finally, *after the message has been removed from* ct_{i^*}*, we must revert all auxiliary changes*: we restore the original definition of the polynomial f_{i^*}, the associated shares, the key K_{i^*}, and the original behaviour of the obfuscated machines. This revert step is essential because it allows us to repeat the argument for the *next* reconstruction input $(i^* + 1, t_{i^*+1})$. In particular, it ensures that at any point the obfuscated machines only hardcode behaviour for a *single* input (i^*, t_{i^*}), rather than accumulating hardcoded behaviour for all possible reconstructing inputs. As a result, we can remove the message from the encryption one index i^* at a time, without requiring the obfuscation to explicitly encode the full list of corrupted parties or all previously fixed shares. This is exactly what allows the share size to remain succinct: each share $\sigma_i = (\mathsf{ct}_i, \widetilde{M}_i)$ contains a field element ct_i of size $\mathsf{poly}(\lambda)$, and an obfuscation $\widetilde{M}_i$ of a machine whose description hardcodes only (K_f, i) and at most one auxiliary triple (pos, t^*, v) with $|\mathsf{pos}|, |t^*| = \omega(\log \lambda)$, and $|v| = \mathsf{poly}(\lambda)$. By the efficiency guarantees on the obfuscator, namely that it produces an obfuscation of size polynomial in the description size and in the input size and logarithmic in the worst-case running time of the original machine, and the fact that the machine's running time and input length are bounded by $O(2^{\omega(\log \lambda)} \cdot \mathsf{poly}(\lambda, \omega(\log \lambda))$, and $2 \cdot \omega(\log \lambda)$ respectively, we obtain $|\widetilde{M}_i| = \mathsf{poly}(\lambda, \log i)$, and hence overall share size $|\sigma_i| = \mathsf{poly}(\lambda, \log i)$.[6]

Achieving adaptive security. A key feature of our proof strategy is that it is entirely agnostic to the adversary's corruption pattern. Rather than tailoring the hybrids to a specific unauthorized set $\mathcal{U}$, we conceptually align the adversary's sequence of corruption queries with the defining evaluations $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j))$ of the polynomial f_{i^*}. Concretely, whenever the adversary corrupts a party $z \leq i^*$, we associate this event with the next unused index j in the interpolation

[6] Note that the running time of the original machine is super-polynomial only in the worst-case scenario, while, assuming $t \in \mathsf{poly}(\lambda)$ (as in our case), the maximum running time is always polynomial.

set, and treat the corresponding PPRF evaluation as the one that is replaced by a uniformly random value and then interpreted as the share $f_{i^*}(z)$. This implicit coupling allows us to reason about the adversary's view purely in terms of the *number* of revealed polynomial evaluations, rather than their specific identities. Hence, the proof relies only on the fact that the adversary sees fewer than t_{i^*} evaluations, independently of which parties are corrupted or in what order.

This invariant is what enables adaptive security in our construction. We remark that adaptive security in evolving secret sharing is notoriously difficult: Francati, Giammusso, and Venturi [12] were the first to consider *adaptive* computational privacy in this setting, and their techniques rely on trapdoor permutations in the random oracle model. Moreover, indistinguishability obfuscation is typically associated with static security guarantees. In contrast, the careful control of hardcoded information in the obfuscated machines allows us to reconcile indistinguishability obfuscation with adaptive security. We view this as a central technical novelty of our work.

Supporting fixed weights. Extending our construction to evolving threshold access structures with fixed weights requires a substantial refinement of both the sharing algorithm and the security proof. The core technical change is reflected in the structure of the sharing machines themselves. Instead of the machine $M_p^{\mathsf{share}}[K_f, i, \mathsf{pos}, t^*, v](k, t)$, we now work with obfuscated machines of the form

$$M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,i}, W, W^*, w_i, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, q),$$

whose richer interface (justified below) is essential to support weights while preserving succinctness and adaptive security.

The weighted setting introduces a fundamental new challenge that is entirely absent in the unweighted construction. In the unweighted scheme, each party contributes exactly one share, and therefore each obfuscated machine is ever used to generate at most one evaluation of the secret polynomial per reconstruction attempt. As a result, in the security proof we can treat each corrupted party independently, hardcode its single share, and never worry about interactions between multiple evaluations produced by the same machine. In contrast, in the weighted setting a single party i of weight w_i conceptually holds w_i distinct shares—equivalently, this can be viewed as a standard Shamir secret sharing scheme in which each party of weight w_i is *virtualized* into w_i distinct "virtual parties", each holding a separate share of the underlying polynomial. This creates a delicate consistency problem in the simulation.

To illustrate the issue, consider simulating the adversary's corruption queries for a fixed reconstruction index i^*. The first weighted share revealed by a party $z \leq i^*$ can be hardcoded in the obfuscated machine in a straightforward way. However, when simulating the *second* weighted share of the same party, we must remove or bypass the previously hardcoded value while ensuring that the newly revealed share remains consistent with a single degree-$(t_{i^*}-1)$ polynomial. Repeating this process naively would require the simulation to remember all previously revealed evaluation points and values, which would force the machine

description size to grow linearly in the total weight of all parties, thus destroying succinctness.

To overcome this obstacle, we introduce two complementary changes. First, the obfuscated machine takes an additional runtime selector input $q \in [w_i]$, and each invocation produces exactly one weighted share. This design allows the security proof to hardcode only a *single* share at a time, by fixing one selector value $q = q_1^*$, while all other shares—whether belonging to the same party or to different parties—continue to be computed honestly. As a result, the machine description remains succinct. Second, we introduce a compression mechanism based on a PPRF with a counter. Intuitively, instead of explicitly storing all previously revealed evaluations, we "compress" them into a succinct representation generated by a PPRF indexed by the polynomial public parameters n, t_n and a counter. However, compression alone is not sufficient: after compressing previously revealed evaluations, we must still ensure that *all* evaluations—both compressed and freshly generated—lie on the same polynomial. Ensuring this global consistency is essential for indistinguishability and is enforced directly within the obfuscated machine.

This leads us to our third change: the obfuscated machines contain an explicit *while*-loop parametrized by the index l^*. On the special input $(k, t, q) = (i^*, t_{i^*}, \cdot)$, the machines construct its interpolation set by: (i) generating the first l^* evaluation as $\{\mathsf{Eval}(K_f, i^*||t_{i^*}||j))\}$ (where j is a counter) and the related points (of the evaluations $\{\mathsf{Eval}(K_f, i^*||t_{i^*}||j))\}$) as $\{\mathsf{Eval}(K'_p, i^*||t_{i^*}||j)\}$ (where K'_p is an auxiliary PPRF key), (ii) inserting a single hardcoded point corresponding to the currently simulated weighted share by means of (x_2, v_2), and then (iii) completing the remaining points and related evaluations using the original PPRF keys K_p and K_f: $\{\mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j))\}$ and $\{\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j))\}$. The role of K'_p is precisely to provide fresh pseudorandom x-coordinates for the compressed points, allowing us to slide the boundary l^* forward in the hybrids without ever querying punctured inputs. Increasing l^* by one corresponds to compressing one additional weighted share while preserving consistency with a single polynomial. In this way, compression and polynomial consistency are enforced *directly within the circuit*.

This also explains our fourth change: the parties' evaluation points themselves must be pseudorandom and generated via a PPRF (in order to be compressible as described above). Indeed, while polynomial values can be compressed using a PPRF, the adversary's corruption pattern is not known in advance. If evaluation points were fixed or derived from party indices, the simulation would be forced to hardcode them explicitly. By instead choosing evaluation points pseudorandomly via a PPRF, we make the points themselves compressible and simulatable, which in turn requires the circuit to output them directly. This is why the machines must return the x-coordinates alongside the polynomial evaluations (this is in contrast to the first construction that outputs $f_n(i)$ directly since i is publicly known), a design choice that is essential for both succinctness and adaptive security. At high level, such a party's x-coordinates (output by the machine) is generated using an additional hardcoded PPRF key $K_{p,i}$ that

is different for every i-th party. More in details, on input (n, t_n, q) for $q \in [w_i]$, the output of the obfuscated machine is set to $(x_{i,q}, f_n(x_{i,q}))$ where $f_n(X)$ is the polynomial determined by the PPRF outputs (as described in the unweighted construction) and $x_{i,q} = \mathsf{Eval}(K_{p,i}, n||t_n||W + q)$ where W is the sum of the party's weights older than i-th party. Using a different PPRF key $K_{p,i}$ per party is fundamental in order to make parties' x-coordinates computation independent, making possible the simulation during our adaptive security experiment.[7]

Related to the above, we need our fifth change: the reconstruction key (of a generic n-th party) is no longer defined as $f_n(0)$ (since evaluation points are now pseudorandom), but instead as $K_n = f_n(x_{n,0})$ where $x_{n,0} = \mathsf{Eval}(K_p, n||t_n||0)$, and this x-coordinate must therefore be included in the share of the party n.[8] Intuitively, this point is the first polynomial evaluation that is swapped with an independent random field element. Once this swap is performed, all parties must consistently use $x_{n,0}$ firstly to interpolate their share, and finally as reconstruction point to reconstruct the key K_n.

The presence of weights also forces a conceptual change in the hybrid argument. Hybrids no longer progress by the number of corrupted parties, but by the cumulative revealed weight. Note that, since the number of hybrids in the security proof is proportional to the sum of all weights, we require each party's weight to be $O(\log \lambda)$ in order to ensure that the overall number of hybrids remains polynomial. Formally, for each i^* we extract the subsequence $\mathcal{Q} = (q_1, \ldots, q_m)$ of oracle queries that request parties $z_j \leq i^*$, and we track the sum of weights requested so far as $W^*_{z_j} = \sum_{k=1}^{j} w_{z_k}$. Again, we remain agnostic to the corruption pattern: rather than tying hybrids to a specific set of parties, we couple the *order* in which weight accumulates (via $W^*_{z_j}$) with an index $j^* \in [0, t_{i^*})$ that identifies "the next polynomial point to be switched" in the interpolation set. Concretely, the role previously played by "the j^*-th corrupted party" is now played by the *unique share* singled out by j^*: we locate the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$, define the within-party offset $w^* := j^* - W^*_{z_{j-1}} \in [w_{z_j}]$, and interpret (z_j, w^*) as "the share that corresponds to index j^*".

After all weighted shares corresponding to the corrupted parties $z \leq i^*$ have been processed, all such machines no longer computes their shares as Lagrange interpolation, since for every selector $q \leq q_2^*$ (with q_2^* being hardcoded to w_z at this point), the machines directly output $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||(W^* + q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||(W^* + q)))$. In particular, no machine evaluates the PPRF at the input $(i^*||t_{i^*}||0)$, which corresponds to the reconstruction point of the secret key K_{i^*}. Hence, we can puncture K_f at the input $(i^*||t_{i^*}||0)$, hardcode

[7] Looking ahead, using a different key $K_{p,i}$ for each i-th party allows our security reduction to freely puncture $K_{p,i}$ only when the corresponding i-th party is corrupted via the adaptive oracle. This is a fundamental aspect when dealing with adaptive adversaries.

[8] Note that, we use K_p (instead of $K_{p,i}$) when computing $x_{n,0}$. This does not create any problem when proving adaptive privacy. This is because the PPRF input required for computing $x_{n,0}$ does not depend on the parties indexes (and weights) and, in turn, is independent to any adversarial *adaptive* query strategy.

the punctured key into all machines returned to the adversary, and then replace the value $K_{i^*} = \mathsf{Eval}(K_f, i^*||t_{i^*}||0)$ with a uniformly random field element. At this point, the ciphertext component ct_{i^*} can be safely randomized, completing the removal of the message for index i^*. The hybrids are then reversed to restore the original distribution before proceeding to the index $i^* + 1$.

Finally, we note that, unlike the unweighted evolving threshold scheme, the weighted construction does not achieve perfect correctness. Correctness now relies on the assumption that the various PPRF evaluations used to define interpolation points do not collide. Such collisions occur only with negligible probability under standard PPRF security, and therefore the scheme satisfies computational (but not perfect) correctness.

Together, these modifications allow us to support fixed weights while preserving both succinct share size and adaptive security, with share size polynomial in $(\lambda, \log W_n)$, where W_n is the total weight of the parties present at time n.

Supporting dynamic weights. We finally extend our construction to evolving threshold access structures with *dynamic* weights, where the weight vector itself may change over time. This access structure is defined over a sequence of weights function and threshold pairs $(\mathsf{weight}_1, t_1), \ldots, (\mathsf{weight}_n, t_n)$, where $\mathsf{weight}_n : [n] \to \mathbb{N}$ assigns to each party $i \leq n$ its (time-n) weight, and a set $\mathcal{I} \subseteq [n]$ is qualified if $\max(\mathcal{I}) = n$ and $\sum_{i \in \mathcal{I}} \mathsf{weight}_n(i) \geq t_n$.

The transition from fixed to dynamic weights is reflected directly in the obfuscated machines. In the fixed weight construction we could rely on the hardcoded parameter W, representing the cumulative weight up to a given index, to assign each party's w_i "virtual" Shamir shares to a contiguous block of evaluation points. In the dynamic weight scheme we work with machines of the form

$$M_p^{\mathsf{dw-share}}[K_f, K_p, K_p', \mathsf{hk}, W^*, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, h, \pi, w, q),$$

where the new hardcoded component is the hashing key hk, while the component that disappears is the hardcoded cumulative weight value W (the "sum of weights up to party i"), since under dynamic weights there is no single, time-independent notion of prefix sums that the dealer can safely bake into every share. Moreover, we have h, π, and w as additional input parameters (justified below).

These changes address a fundamental security failure of the fixed weight construction when weights are allowed to vary. In the fixed weight scheme, the polynomial f_n (and hence the key K_n used in ct_n) is determined only by the public parameters (n, t_n) together with the fixed PPRF keys. As a consequence, if the weights were allowed to change over time while keeping the same polynomial definition, then the *same polynomial* could be reused across different weight vectors. This would immediately break security: shares corresponding to different qualified sets would become correlated.

A naïve way to address this issue would be to let the obfuscated machines take the entire weight vector $(\mathsf{weight}_n(i))_{i \in [n]}$ as an input, and to define the polynomial as a function of this vector. While this would indeed ensure that different weight configurations induce different polynomials, it would also destroy

succinctness: the description of the Turing machine would necessarily grow linearly with the size of the weight vector.

We resolve this tension by *randomizing the polynomial definition* using a succinct commitment to the weight vector, computed via a somewhere statistically binding (SSB) hash. Concretely, at time n the dealer computes $h_n = \mathsf{Hash}(\mathsf{hk}, (\mathsf{weight}_n(i))_{i\in[n]})$, and uses this hash value as an additional input to the PPRF when defining the polynomial f_n. As a result, all defining evaluations now take the form $\mathsf{Eval}(K_f, n||t_n||h_n||\cdot)$, and—crucially, since we no longer hardcode any prefix sum W—the q-th share of party i is evaluated at the "i-th block": $x_{i,q} = \mathsf{Eval}(K_p, n||t_n||h_n||(2^{i\lambda} + q))$, rather than at a point of the form $\mathsf{Eval}(K_p, n||t_n||h_n||(W + q))$ used in the fixed weight construction.

Correctness is preserved because the access structure at time n—namely (weight_n, t_n)—is public at reconstruction time. Every party i in a qualified set can recompute the same hash value h_n and a valid opening π_i certifying its own weight via the SSB scheme. The obfuscated machines verify these openings locally, ensuring that only shares consistent with the claimed weight vector are used in reconstruction. As in the fixed weight case, correctness is computational rather than perfect, due to the reliance on pseudorandom function evaluations.

As for security, the proof technique of this construction closely mirrors that used for the fixed weight case one. The main difference stems from the need to handle the SSB hashing key, which in turn forces us to restrict the model to the static setting. In particular, before manipulating the polynomial associated with a given reconstruction index i^* and interpolation index j^*, we introduce hybrids in which the SSB hashing key hk is sampled so as to be *statistically binding to a single index*. Intuitively, this index corresponds to the party (and, implicitly, to the specific weighted share) whose polynomial evaluation is about to be swapped in the hybrid sequence. This is the main reason why the construction does not achieve adaptive security. Since hk is hardcoded in all obfuscated machines (which are included in the shares of parties) and the index to be made statistically binding is a party's index chosen by the adversary, we must know this index in advance in order to correctly include the corresponding hk in all machines.

Once hk is fixed in this way, the remainder of the hybrid sequence proceeds similarly as before: we compute h_{i^*} inside the hybrids, we simulate the adversary's view by progressively *compressing* the evaluations and x-coordinates corresponding to revealed weighted shares using the PPRF with counter mechanism.[9] As the compression parameter l^* increases, more weighted shares are generated directly from the compressed domain, until all machines corresponding to parties $z \leq i^*$ no longer compute their outputs via Lagrange interpolation. At this point, no machine ever evaluates the PPRF at $(i^*||t_{i^*}||h_{i^*}||0)$, *which de-*

[9] Unlike the the fixed weights construction, our dynamic weights construction computes the parties' x-coordinates using the same PPRF key K_p (instead of a different $K_{p,i}$ per party). This is acceptable since we focus solely on static security, whereas in the fixed weights case a different key $K_{p,i}$ per party is required in order to achieve adaptive security.

fines the reconstruction point of the secret key. We can therefore puncture K_f at this input, replace the resulting key value with an independent uniform field element, and safely randomize the ciphertext component ct_{i^*}. This completes the removal of the message for index i^*, after which the hybrids are reversed to restore the original distribution and the proof proceeds to index $i^* + 1$.

Importantly, introducing the SSB hashing mechanism does not increase the size of the obfuscated machines (and consequently of the shares): the scheme retains the same succinct share size as in fixed weight construction, namely polynomial in $(\lambda, \log W_n)$. As in the fixed-weight setting, we require each party's weight to be bounded by $O(\log \lambda)$ to ensure that the total number of hybrids, proportional to the cumulative revealed weight, remains polynomial.

2 Related Work

Below, we briefly discuss why alternative approaches used to obtain non-evolving secret sharing for weighted threshold access structures cannot be easily adapted to the evolving setting. Finally, we review additional work on evolving secret sharing in the information-theoretic setting.

Succinct witness encryption. A natural starting point is the work on compact threshold signature-based witness encryption (cSWE) of Avitabile *et al.* [2], which yields sublinear size ciphertexts for threshold relations using indistinguishability obfuscation and strongly puncturable signatures. Since cSWE natively supports threshold predicates, it appears well suited for constructing evolving secret sharing schemes with succinct shares, by encrypting the secret under a threshold relation over the parties' verification keys and using signatures as witnesses for decryption. However, this approach fundamentally fails to achieve succinctness in the evolving setting. Correct decryption in cSWE requires access to the full statement, namely the entire set of verification keys associated with the ciphertext, to validate openings of the underlying SSB commitment to the full set of verification keys. Because the set of participating parties grows over time, any opening generated earlier becomes invalid as new parties join. Consequently, at reconstruction time, each party must be able to reconstruct the entire statement, forcing every share to encode all verification keys and yielding linear size shares. This matches the complexity of the *"secret sharing to the past"* approach [13,12], where the share of the last arriving party necessarily grows linearly with the number of participants.

A related line of work is the succinct witness encryption for batch languages of Waters *et al.* [10], which introduces statement-witness compression techniques, taking as input a full statement together with a witness, and produce auxiliary information (hints) that allow the decryption algorithm to operate without explicit access to the statement, thus enabling succinct secret sharing for non-evolving weighted threshold access structures. While powerful in the static setting, these techniques inherently rely on the statement being fixed at the time the auxiliary information is generated. As a result, they do not extend to

evolving secret sharing, where the statement grows dynamically, and therefore cannot yield sublinear share size in our model. Additionally, the scheme of [10] only achieves static, rather than adaptive, computational privacy.

Evolving secret sharing. Evolving secret sharing was introduced by Komargodski *et al.* [19], who showed that every evolving access structure admits an information-theoretic construction, at the cost of exponential share size. Subsequent work has aimed at reducing this complexity by restricting the class of access structures or by making additional assumptions. We first summarize results in the information-theoretic setting. For *general* access structures, Paskin-Cherniavsky [23] obtained smaller (yet still exponential) shares when the dealer is given the access structure in advance.[10] Desmedt *et al.* [9] studied a variant based on evolving perfect hash families, allowing constructions over non-abelian message spaces. If the access structure is assumed to be *multi-level*, Dutta *et al.* [11] achieved an exponential improvement over [19], again relying on prior knowledge of the structure.

For *fixed threshold* evolving access structures, both generic and specialized evolving schemes were already given in [19], with subsequent refinements by D'Arco *et al.* [7]. A separate line of work considers *dynamic threshold* evolving access structures, where the reconstruction threshold changes with the number of parties. Constructions for this setting were proposed by Komargodski *et al.* [20] and improved by Xing *et al.* [25], resulting in schemes with polynomial share size $O(n^4)$. Additional optimizations for the case of large and small thresholds were given by [25] and Alon *et al.* [1] respectively. Closely related are *ramp* evolving access structures, in which privacy is guaranteed only up to a lower threshold and reconstruction is possible above a higher threshold; evolving ramp schemes for various gap parameters were given by Beimel *et al.* [3,4].

Other evolving access structures have also been studied. In the *k-slice* access structures, no requirement is imposed on sets of size exactly k; constructions for $k = 2, 3$ with sub-exponential complexity were given in [1]. The same work considers access structures described by branching programs and shows that bounded-width layered infinite branching programs admit non-trivial evolving secret-sharing schemes. Lower bounds for evolving access structures based on directed *st-connectivity* and *k-hypergraphs* are also proved in [1], while tight message-complexity bounds for evolving *conditional disclosure of secrets* (CDS) protocols are established by Ben David *et al. [8].*

3 Preliminaries

3.1 Notation

We denote by $[n]$ the set of numbers $\{1, 2, ...n\}$. Capital bold-face letters (such as $\boldsymbol{X}$) are used to denote random variables, small letters (such as x) to denote

[10] These improvements apply only to specific families and do not contradict Mazor's lower bound [22].

concrete values, calligraphic letters (such as $\mathcal{X}$) to denote sets, sans-serif letters (such as A) to denote algorithms. For a string $x \in \{0,1\}^*$, we let $|x|$ be its length; if $\mathcal{X}$ is a set, $|\mathcal{X}|$ represents the cardinality of $\mathcal{X}$. When x is chosen uniformly from a set $\mathcal{X}$, we write $x \xleftarrow{\$} \mathcal{X}$.

If A is a deterministic algorithm (modeled as a Turing machine), we write $y = \mathsf{A}(x)$ to denote a run of A on input x and output y; if A is randomized, we write $y \xleftarrow{\$} \mathsf{A}(x)$ (or $y = \mathsf{A}(x; r)$) to denote a run of A on input x and (uniform) randomness r, and output y. An algorithm A is *probabilistic polynomial-time* (PPT) if A is randomized and for any input $x, r \in \{0,1\}^*$ the computation of $\mathsf{A}(x; r)$ terminates in a polynomial number of steps (in the input size).

Negligible functions and computational indistinguishability. We write $\mathsf{negl}(\lambda)$ to denote an arbitrary (unspecified) negligible function of the security parameter $\lambda \in \mathbb{N}$. Similarly, we write $\mathsf{poly}(\lambda)$ to denote an arbitrary (unspecified) polynomial function of the security parameter. We assume all algorithms take the security parameter (in unary) as input. Finally, we write $\mathsf{suplog}(x)$ for a generic super-logarithmc function $\log^c(x)$ for a constant $c > 1$.

Let $\boldsymbol{X} = \{\boldsymbol{X}(\lambda)\}_{\lambda \in \mathbb{N}}$ and $\boldsymbol{Y} = \{\boldsymbol{Y}(\lambda)\}_{\lambda \in \mathbb{N}}$ be ensembles of random variables. We say $\boldsymbol{X}$ and $\boldsymbol{Y}$ are computationally indistinguishable (denoted $\boldsymbol{X} \approx_c \boldsymbol{Y}$) if for all PPT adversaries A we have $|\mathbb{P}[\mathsf{A}(\boldsymbol{X}(\lambda)) = 1] - \mathbb{P}[\mathsf{A}(\boldsymbol{Y}(\lambda)) = 1]| \leq \mathsf{negl}(\lambda)$.

3.2 Puncturable Pseudorandom Functions

A puncturable pseudorandom function (PPRF) is a triple of polynomial-time algorithms $\mathit{\Pi} = (\mathsf{KeyGen}, \mathsf{Punc}, \mathsf{Eval})$ that behaves as a standard pseudorandom function (PRF) on punctured points. In particular, given $K \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, it is possible to generate a punctured key $K_x = \mathsf{Punc}(K, x)$ that permits evaluation of the PRF (using the evaluation algorithm Eval) on all points except the punctured point x.

In addition to the above, a PPRF must also guarantee that $y = \mathsf{Eval}(K, x)$ is indistinguishable from random even given the punctured key $K_x = \mathsf{Punc}(K, x)$.

Below, we provide the formal definitions.

Definition 1 (Correctness of PPRF). *We say a PPRF* $\mathit{\Pi} = (\mathsf{KeyGen}, \mathsf{Punc}, \mathsf{Eval})$ *is correct if* $\forall \lambda \in \mathbb{N}$, $\forall x \in \mathcal{X}$, *and* $\forall x' \in \mathcal{X} \backslash \{x\}$, *we have that* $\mathbb{P}[\mathsf{Eval}(K, x') = \mathsf{Eval}(K_x, x') : K \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda), K_x = \mathsf{Punc}(K, x)] = 1$, *where* $\mathcal{X}$ *is the input space of the PPRF.*

Definition 2 (Security of PPRF). *We say a PPRF* $\mathit{\Pi} = (\mathsf{KeyGen}, \mathsf{Punc}, \mathsf{Eval})$ *is secure if for every* PPT *adversary* A, *for every* $x \in \mathcal{X}$, *we have that:*

$$|\mathbb{P}[\mathsf{A}(K_x, x, \mathsf{Eval}(K, x)) = 1] - \mathbb{P}[\mathsf{A}(K_x, x, \mathbf{U}_\mathcal{Y}) = 1]| \leq \mathsf{negl}(\lambda)$$

where $\mathcal{X}$ *and* $\mathcal{Y}$ *are the input and output space of the PPRF,* $\mathbf{U}_\mathcal{Y}$ *denotes the uniform distribution over the set* $\mathcal{Y}$, $K \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, *and* $K_x = \mathsf{Punc}(K, x)$.

Finally, we are interested in PPRFs whose punctured keys (and the corresponding evaluation) remain at most linear in the size of the input.

Definition 3 (Succinctness and Efficiency of PPRFs). *We say that a PPRF* $\Pi = (\mathsf{KeyGen}, \mathsf{Punc}, \mathsf{Eval})$ *with input space* $\mathcal{X}$ *is succinct if it satisfies the following conditions:*

- *For every* $x \in \mathcal{X}$*, non-punctured and punctured keys have size* $\mathsf{poly}(\lambda)$ *and* $|x| \cdot \mathsf{poly}(\lambda)$*, respectively;*
- *For every* $x \in \mathcal{X}$*, the algorithm* Eval*, when given as input a punctured key* $K_x = \mathsf{Punc}(\mathsf{KeyGen}(1^\lambda), x)$ *(resp. non-punctured key* $K \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$*), runs in time at most* $\log(|\mathcal{X}|) \cdot \mathsf{poly}(\lambda)$ *and such a computation can be described by a Turing Machine of size* $\mathsf{poly}(\lambda, \log(|\mathcal{X}|))$*, where* $\log(|\mathcal{X}|)$ *is the number of bits required to represent any input in* $\mathcal{X}$*.*

Assuming OWF, there exists a secure PPRF satisfying the above efficiency requirement [6].

3.3 Somewhere Statistically Binding Hashing

A somewhere statistically binding (SSB) hash function [17] with block length ℓ_{block}, output length ℓ_{out}, and opening length ℓ_{open}, consists of four polynomial-time algorithms $\Pi = (\mathsf{Setup}, \mathsf{Hash}, \mathsf{Open}, \mathsf{Verify})$ specified as follows:

- $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, m, i)$: The probabilistic setup algorithm $\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, m, i)$ takes as input the security parameter 1^λ, a block size $1^{\ell_{\mathsf{block}}}$, a message length $m \leq 2^\lambda$, and an index $i \in [m]$, and outputs a key hk. Here, we assume that m and i are encoded in binary, i.e., both $|m|$ and $|i|$ are bounded by $\mathcal{O}(\log m)$.
- $h = \mathsf{Hash}(\mathsf{hk}, (x_i)_{i \in [m']})$: The deterministic hashing algorithm $\mathsf{Hash}(\mathsf{hk}, (x_i)_{i \in [m']})$ takes as input a key hk and an input $(x_i)_{i \in [m']}$ (where $x_i \in \{0,1\}^{\ell_{\mathsf{block}}}$ and $m' \leq m$), and outputs a hash $h \in \{0,1\}^{\ell_{\mathsf{out}}}$.
- $\pi_i = \mathsf{Open}(\mathsf{hk}, (x_i)_{i \in [m']}, i)$: The deterministic opening algorithm $\mathsf{Open}(\mathsf{hk}, (x_i)_{i \in [m']}, i)$ takes as input a key hk, an input $(x_i)_{i \in [m']}$ (where $x_i \in \{0,1\}^{\ell_{\mathsf{block}}}$ and $m' \leq m$), and an index $i \in [m']$, and outputs an opening $\pi_i \in \{0,1\}^{\ell_{\mathsf{open}}}$.
- $b = \mathsf{Verify}(\mathsf{hk}, h, i, x_i, \pi_i)$: The deterministic verification algorithm $\mathsf{Verify}(\mathsf{hk}, h, i, x_i, \pi_i)$ takes as input a key hk, a hash $h \in \{0,1\}^{\ell_{\mathsf{out}}}$, an index $i \in [m']$ (for $m' \leq m$), and input $x_i \in \{0,1\}^{\ell_{\mathsf{block}}}$, and an opening $\pi_i \in \{0,1\}^{\ell_{\mathsf{open}}}$, and outputs a bit b.

Correctness of SSB says that honest openings always verify.

Definition 4 (Correctness of SSB). *We say that a SSB scheme* $\Pi = (\mathsf{Setup}, \mathsf{Hash}, \mathsf{Open}, \mathsf{Verify})$ *is correct if* $\forall \lambda \in \mathbb{N}$*,* $\forall \ell_{\mathsf{block}} \in \mathbb{N}$*,* $\forall m \in \mathbb{N}$ *such that* $m \leq 2^\lambda$*,* $\forall m' \in \mathbb{N}$ *such that* $m' \leq m$*,* $\forall i^* \in [m]$*,* $\forall i \in [m']$*, and* $\forall (x_i)_{i \in [m']} \in \{0,1\}^{\ell_{\mathsf{block}} \cdot m'}$*, we have:*

$$\mathbb{P}\left[\mathsf{Verify}(\mathsf{hk}, h, i, x_i, \pi_i) = 1 : \begin{array}{l} \mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, m, i^*), \\ h = \mathsf{Hash}(\mathsf{hk}, (x_i)_{i \in [m']}), \\ \pi_i = \mathsf{Open}(\mathsf{hk}, (x_i)_{i \in [m']}, i) \end{array}\right] = 1$$

As for security, SSB must satisfy two properties, known as *index hiding* and *somewhere statistically binding*. The first property guarantees the indistinguishability of hashing keys generated with respect to two different indices $i_0 \neq i_1$. The second one states that, when the hashing key is generated for an index i (i.e., $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, m, i)$), there do not exist two distinct openings corresponding to two different i-th messages $x \neq x'$.

Definition 5 (Index hiding of SSB). *We say that a SSB scheme* $\Pi = (\mathsf{Setup}, \mathsf{Hash}, \mathsf{Open}, \mathsf{Verify})$ *satisfies index hiding if for every* PPT *adversary* A*, for every* $\ell_{block} \in \mathbb{N}$*, for every* $m \in \mathbb{N}$*, for every indexes* i_0,$i_1 \in [m]$*, we have:*

$$|\mathbb{P}[\mathsf{A}(1^\lambda, \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, m, i_0)) = 1] - \mathbb{P}[\mathsf{A}(1^\lambda, \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, m, i_1)]| \leq \mathsf{negl}(\lambda).$$

Definition 6 (Somewhere statistically binding of SSB). *We say that a SSB scheme* $\Pi = (\mathsf{Setup}, \mathsf{Hash}, \mathsf{Open}, \mathsf{Verify})$ *is somewhere statistically binding if for every* $\ell_{\mathsf{block}} \in \mathbb{N}$*, for every* $m \in \mathbb{N}$ *such that* $m \leq 2^\lambda$*, for every* $i \in [m]$*, we have:*

$$\mathbb{P}\left[\begin{array}{c} \nexists(h, x, x', \pi, \pi') \in \{0,1\}^{\ell_{\mathsf{out}}+2\ell_{\mathsf{block}}+2\ell_{\mathsf{open}}} \\ s.t.\ x \neq x' \wedge \\ \mathsf{Verify}(\mathsf{hk}, h, i, x, \pi) = \mathsf{Verify}(\mathsf{hk}, h, i, x', \pi') = 1 \end{array}\right] \geq 1 - \mathsf{negl}(\lambda),$$

where $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, m, i)$.

In addition to the above properties, we are interested in succinct and efficient SSB. Such schemes can be built from different assumptions including DDH, ϕ-Hiding, DCR and LWE [17,18].

Definition 7 (Succinctness and Efficiency of SSB hash function). *A SSB scheme* $\Pi = (\mathsf{Setup}, \mathsf{Hash}, \mathsf{Open}, \mathsf{Verify})$ *is succinct and efficient if the output length* ℓ_{out}*, the opening length* ℓ_{open}*, and the size of* hk *(output by* $\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, m, i)$*) are bounded by* $\mathsf{poly}(\lambda, \ell_{\mathsf{block}}, \log m)$*. Moreover, we require that the* Verify *algorithm can be represented by a Turing machine of description size and runtime* $\mathsf{poly}(\lambda, \log m)$*.*

3.4 Indistinguishability Obfuscation for Turing Machines

Indistinguishability Obfuscation (iO) [15] allows to obfuscate circuits, guaranteeing indistinguishability between the obfuscations of any two functionally equivalent circuits (i.e., circuits with identical input–output behavior on all inputs).

In this work, we make us of iO for Turing machines [21,16]. Specifically, iO for Turing machines consists of the following polynomial-time algorithm:

- $\widetilde{M} \xleftarrow{\$} \mathsf{Obf}(1^\lambda, 1^n, t, M)$: The probabilistic obfuscation algorithm Obf takes as input the security parameter 1^λ, a description M of the Turing machine to be obfuscated, and an input length n and a time bound t for M, and outputs a machine $\widetilde{M}$ that can be executed on input x for any time bound $t' \leq t$. We denote by $\widetilde{M}(x, t')$ the output of $\widetilde{M}$ on input x after t' steps of computation. The final output of the obfuscated Turing machine (as determined by the time bound t) is denoted by $\widetilde{M}(x, t)$, or simply $\widetilde{M}(x)$.

Correctness requires that the obfuscated Turing machine $\widetilde{M}$ has the same functionality as the original description M. As for security, an iO obfuscator Obf guarantees the indistinguishability of the obfuscations of any two functionally equivalent Turing machines M_0 and M_1.

Definition 8 (Correctness of iO). *We say an iO obfuscator* Obf *is correct if for every* $\lambda \in \mathbb{N}$*, for every Turing machine* M*, for every input* $x \in \{0,1\}^n$*, for every time bounds* t *and* $t' \leq t$*, we have that:*

$$\mathbb{P}\left[\widetilde{M}(x,t') = y_{t'} : \widetilde{M} \xleftarrow{\$} \mathsf{Obf}\left(1^\lambda, 1^n, t, M\right)\right] = 1,$$

where $y_{t'}$ *is the output of the original Turing machine* M *on input* x *after* t' *steps.*

Definition 9 (Security of iO). *We say an iO obfuscator* Obf *is secure if for every* PPT *adversaries* A*, for every time bounds* t*, and every pairs of machines* M_0 *and* M_1 *of the same size such that for all running steps* $t' \leq t$ *and for all inputs* $x \in \{0,1\}^n$*,* $M_0(x) = M_1(x)$ *when both* M_0 *and* M_1 *are executed for* t' *steps, we have that:*

$$\left|\mathbb{P}\left[\mathsf{A}\left(\mathsf{Obf}\left(1^\lambda, 1^n, t, M_0\right)\right) = 1\right] - \mathbb{P}\left[\mathsf{A}\left(\mathsf{Obf}\left(1^\lambda, 1^n, t, M_1\right)\right) = 1\right]\right| \leq \mathsf{negl}(\lambda).$$

We are interested in iO obfuscators that satisfy the following efficiency and succinctness requirements (as defined in [16]).

Definition 10 (Succinctness and Efficiency of iO). *We say an iO obfuscator* Obf *is succinct and efficient if it produces obfuscations of size* $\mathsf{poly}(|M|, \log t, n, \lambda)$*, where* $|M|$ *is the size of the original Turing machine. We also require that the obfuscated machine* $\widetilde{M}$*, on input* $x \in \{0,1\}^n$ *and for any* $t' \leq t$*, runs in time at most* $\mathsf{poly}(|M|, t', n, \log t, \lambda)$*.*

In simple terms, the obfuscation size is polynomial in the size of the original machine M, the input length n, and the security parameter λ, and poly-logarithmic in the worst-case running time t. The runtime of the obfuscated machine is identical except that can "naturally" depends polynomially in the running steps t' (taken as input).

3.5 Evolving Secret Sharing

In classical non-evolving secret sharing schemes, the set of parties is fixed, and the sharing phase is performed a single time, producing a share for each of the n parties participating in the system. Reconstruction is possible only when combining a set of shares corresponding to a subset $\mathcal{I}$ of parties that is qualified. Whether a subset is qualified depends on the type of secret sharing scheme used, which is usually represented by an access structure. We recall this definition below.

Definition 11 (Access structure). *Let $n \in \mathbb{N}$. A collection of subsets $\mathcal{A} \subseteq 2^{[n]}$ is* monotone *if for every $\mathcal{I} \in \mathcal{A}$, every $\mathcal{I}' \subseteq 2^{[n]}$ such that $\mathcal{I} \subseteq \mathcal{I}'$, we have $\mathcal{I}' \in \mathcal{A}$.*

An access structure $\mathcal{A} \subseteq 2^{[n]}$ is a monotone collection of non-empty subsets. Subsets in $\mathcal{A}$ are called qualified*, whereas subsets not in $\mathcal{A}$ are called* unqualified*.*

In evolving secret sharing (ESS), parties join the system sequentially, and the value n represents the number of parties currently present. Following the definitions of Francati and Venturi [13,14], we focus on the computational setting, where the number of parties n can grow while remaining polynomial in the security parameter, i.e., $n = \mathsf{poly}(\lambda)$. As in [13], we denote by $\mathcal{A}_n$ the access structure corresponding to n parties in the system. We emphasize that the dealer does not know the exact value of n in advance (other than that it is polynomial), nor does it know the access structure $\mathcal{A}_n$ before the n-th party joins the system (i.e., the total number of parties is not known apriori).[11]

Below, we recall the definition of evolving access structure that abstracts the reconstruction rule of such schemes.

Definition 12 (Evolving access structure). *Let $n \in \mathbb{N}$, with $n(\lambda) = \mathsf{poly}(\lambda)$. An evolving access structure $\mathcal{A} = \{\mathcal{A}_n\}$ is a monotone collection of subsets $\mathcal{A}_n \subseteq 2^{[n]}$ such that, for every $n \in \mathbb{N}$, it holds that $\mathcal{A}_n \subseteq \mathcal{A}_{n+1}$.*

For completeness, we note that the computational setting considers only classes of access structures that admit a polynomial-size representation, as discussed in [13, Remark 1].

Following [13,14], we naturally focus on rigid monotone evolving access structures, i.e., subsets that were previously unauthorized remains always unauthorized. This condition is necessary in order to avoid re-generating shares of older parties. We refer to [13] for more details.

Definition 13 (Rigid evolving access structure). *A* rigid *evolving access structure $\mathcal{A} = \{\mathcal{A}_n\}_{n \geq 1}$ is a monotone collection of subsets $\mathcal{A}_n \subseteq 2^{[n]}$ such that, for any $n \in \mathbb{N}$, it holds that $\mathcal{A}_n = \mathcal{A} \cap 2^{[n]}$.*

To highlight rigid access structures, we use the hat symbol $\hat{\mathcal{A}}$ to denote access structures that evolve while preserving rigidity.

Syntax and definitions for ESS schemes. A ESS scheme Π for an evolving access structure $\mathcal{A}$ is composed of the following two polynomial-time algorithms:

- $\sigma_n \xleftarrow{\$} \mathsf{Share}(\mu, (\sigma_i)_{i \in [n-1]}, \mathcal{A}_n)$: The probabilistic sharing algorithm $\mathsf{Share}(\mu, (\sigma_i)_{i \in [n-1]}, \mathcal{A}_n)$ takes as input a message $\mu \in \mathcal{M}$, a collection of $n-1$ shares $\sigma_1, ..., \sigma_{n-1} \in \mathcal{S}$, and an access structure $\mathcal{A}_n$, and outputs the share σ_n for the n-the party.

[11] As mentioned in [13], when the total number of parties n is known in advance (or a safer upper bound), the problem becomes trivial making non-evolving and evolving secret sharing collapse. This is because, a dealer, that knows n and has access to $\mathcal{A}_n$, could simply use a non-evolving secret sharing scheme for the same access structure $\mathcal{A}_n$ and distribute the shares to the parties as they arrive.

– $\mu = \mathsf{Recon}((\sigma_i)_{i\in\mathcal{I}}, \mathcal{I}, \mathcal{A}_n)$: The deterministic reconstruction algorithm Recon $((\sigma_i)_{i\in\mathcal{I}}, \mathcal{I}, \mathcal{A}_n)$ takes as input a collection of shares $(\sigma_i)_{i\in\mathcal{I}}$, along with a subset $\mathcal{I} \subseteq 2^{[n]}$ of the parties and the access structure $\mathcal{A}_n$, where $n = \max(\mathcal{I})$ and outputs a message $\mu \in \mathcal{M}$. Note that the assumption that $n = \max(\mathcal{I})$ is w.l.o.g., as for any $n' > n$ it holds that $\mathcal{A}_n \subseteq \mathcal{A}_{n'}$ by monotonicity of the access structure.

We emphasize that the reconstruction algorithm takes as input a description of the access structure $\mathcal{A}_n$. This is consistent with all previous definitions of evolving secret sharing.
Correctness says that any qualified set $\mathcal{I} \in \mathcal{A}$ can reconstruct the message by combining their shares $(\sigma_i)_{i\in\mathcal{A}}$.

Definition 14 ((Perfect) Correctness of ESS). *We say that a secret sharing scheme ESS over message space $\mathcal{M}$ for the evolving access structure $\mathcal{A}$ is correct if for every message $\mu \in \mathcal{M}$, for every number of parties $n \geq 1$, and for every qualified subset $\mathcal{I} \in \mathcal{A}_n$ it holds that:*

$$\mathbb{P}[\mu = \mathsf{Recon}((\sigma_i)_{i\in\mathcal{I}}, \mathcal{I}, \mathcal{A}_n) : \forall i \leq n : \sigma_i \xleftarrow{\$} \mathsf{Share}(\mu, (\sigma_j)_{j<i}, \mathcal{A}_i)] = 1 - \mathsf{negl}(\lambda).$$

We say that ESS is perfectly correct if the correctness property holds with probability 1.

As for security, ESS guarantees the secrecy of the message against any adversary controlling a set of unqualified parties $\mathcal{U} \notin \mathcal{A}$. In this work, we consider an adaptive security which addresses the problem of keeping the message secret against an adversary that can adaptively increase the number of parties and corrupt them (obtaining their shares). Below, we provide the adaptive definition recently introduced in [12, Definition 15]. Static security can be easily derived by restricting the adversary to submit a single query to the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ of the experiment below.[12]

Definition 15 (Static/Adaptive computational privacy of ESS). *An ESS scheme Π over message space $\mathcal{M}$ for the evolving access structure $\mathcal{A}$ is computationally private in the adaptive setting if $\{\boldsymbol{G}^{\mathsf{priv}}_{\Pi,\mathsf{A}}(\lambda,0)\}_{\lambda\in\mathbb{N}} \approx_c \{\boldsymbol{G}^{\mathsf{priv}}_{\Pi,\mathsf{A}}(\lambda,1)\}_{\lambda\in\mathbb{N}}$, where the game $\{\boldsymbol{G}^{\mathsf{priv}}_{\Pi,\mathsf{A}}(\lambda,b)\}$ is defined as follows:*

– *Initially let $n = 0$.*
– *The adversary A chooses a pair of messages $\mu_0, \mu_1 \in \mathcal{M}$.*
– *The adversary A gets oracle access to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ that, on input an integer $n_k \in \mathbb{N}$ and a set of indices $\mathcal{U}_k$, proceeds as follows:*
  • *If $\bigcup_{k\in[q]} \mathcal{U}_k \in \mathcal{A}_{n+n_k}$, where q is the number of oracle queries made so far, the oracle returns $\perp$ and aborts (i.e., the corrupted parties compose a qualified set).*

[12] When only a single query is submitted, it is easy to see that Definition 15 is equivalent to the static security definitions given in [13, Definition 8], [14, Definition 18], and [12, Definition 11].

- *Otherwise, it computes* $\sigma_{n+i} \xleftarrow{\$} \mathsf{Share}(\mu_b, (\sigma_j)_{j<n+i}, \mathcal{A}_{n+i})$ *for every* $i \in [n_k]$.
- *The oracle internally stores* $(\sigma_i)_{i\in[n+n_k]}$ *and updates* $n = n + n_k$.
- *Finally, it returns* $(\sigma_i)_{i\in\mathcal{U}_k}$.

– *Finally,* A *outputs a bit* $b' \in \{0,1\}$ *which is also the output of the experiment.*

If the adversary submits only a single query to the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ *(i.e.,* $q = 1$*), we say that the ESS scheme is computationally private in the static setting.*

3.6 Shamir's Secret Sharing and Lagrange Interpolation

Let $n, t \in \mathbb{N}$ with $n \geq t$. We briefly recall the classical method by Shamir [24] for implementing t-out-of-n threshold secret sharing. Let $\mathbb{F}$ be a finite field. In order to share a message $\mu \in \mathbb{F}$, we sample a random polynomial $f(X)$ with coefficients in $\mathbb{F}$ and degree $t-1$ such that $f(0) = \mu$. The shares $(\sigma_1, \ldots, \sigma_n)$ for the n parties are then computed by evaluating $f(X)$ at the parties' indices, that is, $\sigma_i = (i, f(i))$. Reconstruction is straightforward: given a set of t shares $(\sigma_i)_{i\in\mathcal{I}}$, it suffices to interpolate the points $(i, f(i))$ to recover $f(X)$, which reveals the message $\mu = f(0)$.

Lagrange Interpolation. Instead of recomputing the polynomial $f(X)$ in its entirety, Lagrange interpolation is often used to recover the message directly. In other words, given any set of t evaluations $(f(i))_{i\in\mathcal{I}}$ and the corresponding distinct evaluation points $\mathcal{I} = \{i_1, \ldots, i_t\}$, Lagrange interpolation allows one to directly compute $f(0)$.

More precisely, for any fixed $i^* \in \mathbb{F}$, with $i^* \notin \mathcal{I}$, the value $f(i^*)$ can be computed from $\mathcal{I} = \{i_1, \ldots, i_t\}$ and $(f(i))_{i\in\mathcal{I}}$ as follows:

$$f(i^*) = \mathsf{Lagrange}(i^*, (f(i))_{i\in\mathcal{I}}, \mathcal{I}) = \sum_{j\in[t]} f(i_j) \cdot \ell_j(i^*), \tag{1}$$

where $\ell_j(i^*) = \prod_{k\in[t]\setminus\{j\}} \frac{i^*-i_k}{i_j-i_k}$. The polynomials $\{\ell_j(X)\}_{j\in[t]}$ are called the Lagrange basis polynomials.

Note that viewing polynomial evaluation through the lens of Lagrange interpolation allows to compute the target value $f(i^*)$ incrementally by summing the terms $f(i_j)\cdot\ell_j(i^*)$. Our constructions will extensively leverage this property in order to program obfuscated Turing machines (through iO) while keeping their size succinct. The latter is achieved via repeated applications of the theorem below, which states that, given a set of evaluations $\{y_1, \ldots, y_t\}$ at indices $\{i_1, \ldots, i_t\}$, whenever one of the evaluations — say y_t — is sampled at random, computing $y^* = f(i^*)$ — where $f(X)$ of degree t is uniquely defined by $\{y_1, \ldots, y_t\}$ and $\{i_1, \ldots, i_t\}$ — through Lagrange interpolation is identically distributed to sampling $y^* = f(i^*)$ directly at random and then interpolating y_t.

Lemma 1. *Let* $\mathbb{F}$ *be a finite field. For every* $n \in \mathbb{N}$*, for every* $t \in \mathbb{N}$ *such that* $t \leq n$*, for every subset* $\mathcal{I} = \{i_1, \ldots, i_{t-1}\} \subset [n]$ *of distinct indices, for every pair*

of indices $i_t, i^* \in [n] \setminus \overline{\mathcal{I}}$ *such that* $i_t \neq i^*$*, for every subset of evaluations* $\overline{\mathcal{Y}} = \{y_1, \ldots, y_{t-1}\} \subseteq \mathbb{F}$ *we have that* $\left\{n, t, \overline{\mathcal{I}}, i_t, i^*, \overline{\mathcal{Y}}, r^*\right\} \equiv \left\{n, t, \overline{\mathcal{I}}, i_t, i^*, \overline{\mathcal{Y}}, y^*\right\}$*, where* y_t *and* r^* *are sampled uniformly at random from* $\mathbb{F}$*, and* $y^* = \mathsf{Lagrange}(i^*, \overline{\mathcal{Y}} \cup \{y_t\}, \overline{\mathcal{I}} \cup \{i_t\})$*.*

Proof. Fix an arbitrary choice of $n, t, \overline{\mathcal{I}}, i_t, i^*, \overline{\mathcal{Y}}$ as in the theorem statement, where $\overline{\mathcal{I}} = \{i_1, \ldots, i_{t-1}\}$ and $\overline{\mathcal{Y}} = \{y_1, \ldots, y_{t-1}\}$. We must show that y^* is uniformly random over $\mathbb{F}$ and independent of the above fixed parameters. By Equation (1), we can write: $y^* = \sum_{j \in [t]} y_j \cdot \ell_j(i^*) = \sum_{j \in [t-1]} y_j \cdot \ell_j(i^*) + y_t \cdot \ell_t(i^*)$.

Given that $\overline{\mathcal{I}}$, $\overline{\mathcal{Y}}$ and i^* are fixed, we observe that $y^* = \alpha + \beta \cdot y_t$ where $\alpha, \beta \in \mathbb{F}$ are constants. Additionally, since i^*, i_t and the indices in $\overline{\mathcal{I}}$ are all distinct, we have that $\beta \neq 0$ and thus the map $f(X) = \alpha + \beta \cdot X$ is a bijection from $\mathbb{F}$ to $\mathbb{F}$. Hence, since y_t is uniform, we have that y^* is uniform and independent of $n, t, \overline{\mathcal{I}}, i_t, i^*, \overline{\mathcal{Y}}$. The theorem follows. □

Note that the above lemma does not assume any distribution on the evaluations, except that one of them is random.

4 Evolving Threshold Access Structures without Weights

Below, we recall the formal definition of an evolving threshold access structure $\hat{\mathcal{A}}^{\mathsf{ethr}}$. We adopt the more general definition provided by [13, Remark 4] where threshold can be arbitrary (i.e., they can increase, decrease, or remain stable). In order to be monotone and rigid, the reconstructing threshold t_n of an authorized set $\mathcal{I}$ is the one associated to the party with the largest index in $\mathcal{I}$, as defined in [13, Remark 4].

Definition 16 (Evolving Threshold Access Structure [13,14]). *An evolving threshold access structure* $\hat{\mathcal{A}}^{\mathsf{ethr}}$ *is defined over a sequence of thresholds* $t_1, \ldots, t_n$ *satisfying the following condition:*

- *(qualified set) a set* $\mathcal{I} = \{i_1, \ldots, i_k, n\}$ *is qualified (i.e.,* $\mathcal{I} \in \hat{\mathcal{A}}^{\mathsf{ethr}}_n$*) if* $t_n \leq |\mathcal{I}|$ *and* $\mathsf{max}(\mathcal{I}) = n$*.*

We note that the above access structure is monotone and rigid by definition. Monotonicity holds since any superset $\mathcal{I}'$ of a qualified set $\mathcal{I}$ is authorized by definition, as $\mathcal{I}'$ necessarily contains $\mathcal{I}$, which satisfies both $t_n \leq |\mathcal{I}|$ and $\mathsf{max}(\mathcal{I}) = n$. Rigidity holds by the constraint $\mathsf{max}(\mathcal{I}) = n$ whose value determines the reconstructing threshold t_n. This implies that any unauthorized set $\mathcal{I}$ such that $\mathsf{max}(\mathcal{I}) = n$ but $|\mathcal{I}| < t_n$ remains unauthorized, since the threshold t_n always applies to $\mathcal{I}$ when n is the party with the highest index.

4.1 Construction from iO for Turing Machines and PPRF

We are now ready to present our construction of ESS for the dynamic threshold evolving access structure $\hat{\mathcal{A}}^{\mathsf{ethr}}$ with share size $\mathsf{poly}(\lambda, \log n)$, where n is the number of parties. In Figure 1 we provide the formalization of our Turing machine whereas in Construction 4.1 we present our ESS scheme.

$M_p^{\mathsf{share}}[K_f, i, \mathsf{pos}, t^*, v](k, t)$:

if $k < i$ **or** $t \leq 0$ **then**
  return $\perp$
if $k = \mathsf{pos}$ **and** $t = t^*$ **then**
  return v
Initialize tape eval as an array of evaluations of size t.
$j = 0$
while $j < t$ **do**
  $\mathsf{eval}[j] = \mathsf{Eval}(K_f, k \| t \| (2^\lambda - j))$
  $j = j + 1$
$j = 0$, $S = 0$
while $j < t$ **do**
  $m = 0$, $L = 1$
  while $m < t$ **do**
    if $m \neq j$ **then**
      $$L = L \cdot \frac{i - (2^\lambda - m)}{(2^\lambda - j) - (2^\lambda - m)} \bmod p$$
    $m = m + 1$
  $S = S + L \cdot \mathsf{eval}[j] \bmod p$
  $j = j + 1$
return S

Fig. 1. The formal definition of the Turing Machine $M_p^{\mathsf{share}}[K_f, i, \mathsf{pos}, t^*, v](k, t)$ required in Construction 4.1. The input space of the machine is $\{0,1\}^{2 \cdot \mathsf{suplog}(\lambda)}$. Given the description, it is syntactically clear that $M_p^{\mathsf{share}}[K_f, i, \mathsf{pos}, t^*, v](k, t)$, for any input $(k, t) \in \{0,1\}^{2 \cdot \mathsf{suplog}(\lambda)}$, the machine always halt. We denote by $\ell^{\mathsf{max}}_{M^{\mathsf{share}}} = 2 \cdot \mathsf{suplog}(\lambda)$ and $t^{\mathsf{max}}_{M^{\mathsf{share}}}$ the maximum input size and maximum running time (over the choice of all inputs $(k, t) \in \{0,1\}^{\ell^{\mathsf{max}}_{M^{\mathsf{share}}}}$). The highlighted parts of the Turing machine are those required to prove the security of the scheme (due to the use of iO), and are therefore not needed for correctness.

Construction 4.1

Consider the following ingredients:

- $\mathbb{F}_p$ be a field where p is a $(\lambda+1)$-bit prime;
- $\mathsf{PPRF} = (\mathsf{KeyGen}, \mathsf{Punc}, \mathsf{Eval})$ be a PPRF such that $\mathsf{Eval} : \{0,1\}^\lambda \times \{0,1\}^{2\cdot\mathsf{suplog}(\lambda)+\lambda} \to \mathbb{F}_p$;[a]
- Obf be an iO obfuscator for the class of Turing machines $\mathcal{M}^{\mathsf{share}} = \{M_p^{\mathsf{share}}[K_f, i, \mathsf{pos}, t^*, v](k,t)\}$ as defined in Figure 1. The values between square brackets are hardcoded values that we will modify during our security proof. The hardcoded values come from the space $\mathsf{KeyGen}(1^\lambda) \times \{0,1\}^{3\cdot\mathsf{suplog}(\lambda)} \times \mathbb{F}_p$.

We build a ESS scheme $\mathsf{SS} = (\mathsf{Share}, \mathsf{Recon})$ over message space $\mathbb{F}_p$ for the evolving threshold access structure $\hat{\mathcal{A}}^{\mathsf{ethr}}$ (Definition 16) as follows:

Sharing: When the n-th party arrives, the dealer shares $\mu \in \mathbb{F}_p$ as follows:

- If $n = 1$ (first party), sample a PPRF key $K_f \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$.
- Let $M_n = M_p^{\mathsf{share}}[K_f, n, 0, 0, 0](\cdot,\cdot)$ be a Turing machine as defined in Figure 1 and let $\ell^{\mathsf{max}}_{M^{\mathsf{share}}}$ and $t^{\mathsf{max}}_{M^{\mathsf{share}}}$ be the maximum input size and running time defined in Figure 1.
- Compute $\widetilde{M}_n \xleftarrow{\$} \mathsf{Obf}(1^\lambda, 1^{\ell^{\mathsf{max}}_{M^{\mathsf{share}}}}, t^{\mathsf{max}}_{M^{\mathsf{share}}}, M_n)$.
- Compute $K_n = f_n(0)$, where $f_n(X) \in \mathbb{F}_p$ is the unique polynomial of degree $t_n - 1$ obtained by interpolating the evaluations $\{\mathsf{Eval}(K_f, n||t_n||(2^\lambda - j))\}_{j\in[0,t_n)}$ and the points $\{2^\lambda - j\}_{j\in[0,t_n)}$.
- Compute $\mathsf{ct}_n = K_n \cdot \mu$ where μ is the message.
- Finally, set the share of the n-th party to $\sigma_n = (\mathsf{ct}_n, \widetilde{M}_n)$.

Reconstruction: The reconstruction algorithm $\mathsf{Recon}((\sigma_i)_{i\in\mathcal{I}}, \mathcal{I}, \hat{\mathcal{A}}^{\mathsf{ethr}}_n)$, given the shares $(\sigma_i)_{i\in\mathcal{I}} = (\mathsf{ct}_i, \widetilde{M}_i)$, and a (minimal) qualified subset $\mathcal{I} = \{i_1, i_2, \ldots, n\} \in \hat{\mathcal{A}}^{\mathsf{ethr}}$ (i.e., $|\mathcal{I}| \geq t_n$ and $\mathsf{max}(\mathcal{I}) = n$), it proceeds as follows:

- For $i \in \mathcal{I}$, execute the obfuscated Turing machine $\mathsf{sh}_i = \widetilde{M}_i(n, t_n)$.
- Compute $K'_n = \mathsf{Lagrange}(0, (\mathsf{sh}_i)_{i\in\mathcal{I}}, \mathcal{I})$.
- Finally, output $\mu' = \mathsf{ct}_n \cdot (K'_n)^{-1}$.

[a] Note that $\mathsf{suplog}(\lambda)$ bits are sufficient to encode in binary any number of parties $n \in \mathsf{poly}(\lambda)$ and any threshold $t_n \leq n$.

Correctness, succinctness, and security of Construction 4.1. Correctness follows by the correctness of iO. The formal proof is deferred to Appendix A.1.

Theorem 4. *If the iO obfuscator* Obf *is correct (Definition 8) then the ESS scheme of Construction 4.1 is perfectly correct.*

As for succinctness, we demonstrate that obfuscating $M_p^{\mathsf{share}}[K_f, i, \mathsf{pos}, t^*, v](k, t)$ produces an obfuscation of size $\mathsf{poly}(\lambda, \log n)$ (yielding a share of the same size). Below, we provide the formal result.

Theorem 5. *If the iO obfuscator* Obf *is succinct and efficient (Definition 10) and* PPRF *is succinct and efficient (Definition 3) then the ESS scheme of Construction 4.1 produces shares of size* $\mathsf{poly}(\lambda, \log n)$ *for every party* $n \geq 1$.

The formal proof is deferred to Appendix A.2.

Finally, the theorem below establishes the security of the scheme whose formal proof appears in Appendix A.3

Theorem 6. *If the iO obfuscator* Obf *is secure (Definition 9), and* PPRF *is secure (Definition 2), then the scheme ESS of Construction 4.1, for the evolving threshold access structures* $\hat{\mathcal{A}}^{\mathsf{ethr}}$*, is computationally private in the adaptive setting (Definition 15).*

5 Evolving Threshold Access Structures with Fixed Weights

We now focus on weighted threshold access structures under the restriction that weights are fixed. In a nutshell, when a party arrives, it is associated with a threshold t_n and a weight w_n, which remain unchanged throughout the entire evolution of the access structure (i.e., the n-th party always has weight w_n). As in $\hat{\mathcal{A}}^{\mathsf{ethr}}$ (Definition 16), the reconstructing threshold t_n is determined by the party with the highest index in the qualified set $\mathcal{I}$.

Definition 17 (Evolving Threshold Access Structure with Fixed Weights). *An evolving threshold access structure with fixed weights* $\hat{\mathcal{A}}^{\mathsf{fw-ethr}}$ *is defined over a sequence of threshold and weight pairs* $(t_1, w_1), \ldots, (t_n, w_n)$ *satisfying the following condition:*

- *(qualified set) a set* $\mathcal{I} = \{i_1, \ldots, i_k, n\}$ *is qualified (i.e.,* $\mathcal{I} \in \hat{\mathcal{A}}_n^{\mathsf{fw-ethr}}$*) if* $t_n \leq \sum_{i \in \mathcal{I}} w_i$ *and* $\mathsf{max}(\mathcal{I}) = n$.

We observe that also for this access structure (as for the one of Definition 16), both monotonicity and rigidity are satisfied by definition.

5.1 Construction from iO for Turing Machines and PPRF

We are now ready to present our construction of ESS for the dynamic threshold evolving access structure with fixed weights $\hat{\mathcal{A}}^{\mathsf{fw-ethr}}$ with share size $\mathsf{poly}(\lambda, \log W)$, where W is the sum of the parties' weights.

We restrict ourselves to access structures $\hat{\mathcal{A}}^{\mathsf{fw-ethr}}$ in which each weight has size at most $|w| = O(\log \lambda)$. In Figure 2 we provide the formalization of our Turing machine whereas in Construction 5.1 we present our ESS scheme.

$M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,i}, W, W^*, w_i, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, q)$:

if $k < i$ **or** $t \leq 0$ **or** $q \leq 0$ **or** $q > w_i$ **then**
  return $\perp$
Initialize tapes eval and points as arrays of evaluations and points of size t.
$j = 0$
if $k = \mathsf{pos}$ **and** $t = t^*$ **and** $q = q_1^*$ **then**
  return (x_1, v_1)
if $k = \mathsf{pos}$ **and** $t = t^*$ **and** $q \leq q_2^*$ **then**
  return $\big(\mathsf{Eval}(K'_p, k\|t\|(W^* + q)),\ \mathsf{Eval}(K_f, k\|t\|(W^* + q))\big)$
if $k = \mathsf{pos}$ **and** $t = t^*$ **then**
  while $j < l^*$ **do**
    $\mathsf{eval}[j] = \mathsf{Eval}(K_f, k\|t\|j)$
    $\mathsf{points}[j] = \mathsf{Eval}(K'_p, k\|t\|j)$
    $j = j + 1$
  if $v_2 \neq \perp$ **then**
    $\mathsf{eval}[j] = v_2$
    $\mathsf{points}[j] = x_2$
    $j = j + 1$
while $j < t$ **do**
  $\mathsf{eval}[j] = \mathsf{Eval}(K_f, k\|t\|(2^\lambda - j))$
  $\mathsf{points}[j] = \mathsf{Eval}(K_p, k\|t\|(2^\lambda - j))$
  $j = j + 1$
$j = 0$, $S = 0$
while $j < t$ **do**
  $m = 0$, $L = 1$
  while $m < t$ **do**
    if $m \neq j$ **then**
      $L = L \cdot \dfrac{\mathsf{Eval}(K_{p,i}, k\|t\|(W + q)) - \mathsf{points}[m]}{\mathsf{points}[j] - \mathsf{points}[m]} \bmod p$
    $m = m + 1$
  $S = S + L \cdot \mathsf{eval}[j] \bmod p$
  $j = j + 1$
return $\big(\mathsf{Eval}(K_{p,i}, k\|t\|(W + q)),\ S\big)$

Fig. 2. The formal definition of the Turing Machine $M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,i}, W, W^*, w_i, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, q)$ required in Construction 5.1. The input space of the machine is $\{0,1\}^{2\cdot\mathsf{suplog}(\lambda)+|w|}$ where $|w|$ is the maximum supported weight size. Given the description, it is syntactically clear that $M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,i}, W, W^*, w_i, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, q)$, for any input $(k, t, q) \in \{0,1\}^{2\cdot\mathsf{suplog}(\lambda)+|w|}$, the machine always halt. We denote by $\ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}} = 2 \cdot \mathsf{suplog}(\lambda) + |w|$ and $t^{\mathsf{max}}_{M^{\mathsf{fw-share}}}$ the maximum input size and maximum running time (over the choice of all inputs $(k, t, q) \in \{0,1\}^{\ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}}}$). The highlighted parts of the Turing machine are those required to prove the security of the scheme (due to the use of iO), and are therefore not needed for correctness.

Construction 5.1

Consider the following ingredients:

- $\mathbb{F}_p$ be a field where p is a $(\lambda+1)$-bit prime;
- $\mathsf{PPRF} = (\mathsf{KeyGen}, \mathsf{Punc}, \mathsf{Eval})$ be a PPRF such that $\mathsf{Eval} : \{0,1\}^\lambda \times \{0,1\}^{2\cdot\mathsf{suplog}(\lambda)+\lambda} \to \mathbb{F}_p$;[a]
- Obf be an iO obfuscator for the class of Turing machines $\mathcal{M}^{\mathsf{fw-share}} = \{M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,i}, W, W^*, w_i, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, q)\}$ as defined in Figure 2. The values between square brackets are the hardcoded values that we will modify during our security proof. The hardcoded values come from the following space $\mathsf{KeyGen}(1^\lambda)^4 \times \{0,1\}^{2\cdot\mathsf{suplog}(\lambda)+|w|+3\cdot\mathsf{suplog}(\lambda)} \times \mathbb{F}_p^4 \times \{0,1\}^{3\cdot\mathsf{suplog}(\lambda)}$.

We build a ESS scheme $\mathsf{SS} = (\mathsf{Share}, \mathsf{Recon})$ over message space $\mathbb{F}_p$ for the evolving threshold access structure $\hat{\mathcal{A}}^{\mathsf{fw-ethr}}$ (Definition 17) for weights of size $|w| = O(\log \lambda)$ as follows:

Sharing: When the n-th party arrives, the dealer proceeds as follows:

- If $n = 1$ (first party), sample four PPRF keys: $K_f \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, $K_p \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, $K'_p \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, and $K_{p,n} \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$.[b]
- Let $M_n = M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,n}, W_{n-1}, 0, w_n, n, 0, 0, 0, 0, 0, \bot, 0, 0, 0](\cdot,\cdot,\cdot)$ be a Turing machine as defined in Figure 2 and let $\ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}}$ and $t^{\mathsf{max}}_{M^{\mathsf{fw-share}}}$ are the maximum input size and running time defined in Figure 2.
- Compute $\widetilde{M}_n \xleftarrow{\$} \mathsf{Obf}(1^\lambda, 1^{\ell^{\mathsf{max}}_{M^{\mathsf{fw-hare}}}}, t^{\mathsf{max}}_{M^{\mathsf{fw-share}}}, M_n)$.
- Compute $K_n = f_n(\mathsf{Eval}(K_p, n||t_n||0))$, where $f_n(X) \in \mathbb{F}_p$ is the unique polynomial of degree $t_n - 1$ obtained by interpolating the evaluations $\{\mathsf{Eval}(K_f, n||t_n||(2^\lambda - j))\}_{j\in[0,t_n)}$ and the points $\{\mathsf{Eval}(K_p, n||t_n||(2^\lambda - j))\}_{j\in[0,t_n)}$.
- Compute $\mathsf{ct}_n = K_n \cdot \mu$ where μ is the message.
- Finally, set the share of the n-th party to $\sigma_n = (\mathsf{ct}_n, \widetilde{M}_n, x_{n,0})$ where $x_{n,0} = \mathsf{Eval}(K_p, n||t_n||0)$.

Reconstruction: The reconstruction algorithm $\mathsf{Recon}((\sigma_i)_{i\in\mathcal{I}}, \mathcal{I}, \hat{\mathcal{A}}_n^{\mathsf{fw-ethr}})$, given the shares $(\sigma_i)_{i\in\mathcal{I}} = (\mathsf{ct}_i, \widetilde{M}_i, x_{i,0})$, and a (minimal) qualified subset $\mathcal{I} = \{i_1, i_2, \ldots, n\} \in \hat{\mathcal{A}}^{\mathsf{fw-ethr}}$ (i.e., $W_\mathcal{I} = \sum_{i\in\mathcal{I}} w_i \geq t_n$ and $\mathsf{max}(\mathcal{I}) = n$), it proceeds as follows:

- For every $i \in \mathcal{I}$ and $q_i \in [w_i]$, execute the obfuscated Turing machine $(x_{i,q_i}, \mathsf{sh}_{i,q_i}) = \widetilde{M}_i(n, t_n, q_i)$.
- Run $K'_n = \mathsf{Lagrange}(x_{n,0}, (\mathsf{sh}_{i,q_i})_{i\in\mathcal{I}, q_i\in[w_i]}, (x_{i,q_i})_{i\in\mathcal{I}, q_i\in[w_i]})$.
- Finally, output $\mu' = \mathsf{ct}_n \cdot (K'_n)^{-1}$.

[a] Note that $\mathsf{suplog}(\lambda)$ bits are sufficient to encode in binary any number of parties $n \in \mathsf{poly}(\lambda)$ and any threshold $t_n \leq n$.

[b] The PPRF key $K_{p'}$ is sampled and hardcoded into $M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,n}, W_{n-1}, 0, w_n, n, 0, 0, 0, 0, 0, \perp, 0, 0, 0]$ solely to simplify the security proof. Indeed, K'_p is used only in the proof and not required for the correctness of the construction.

Correctness, succinctness, and security of Construction 5.1. Differently from Construction 4.1, Construction 5.1 does not achieve perfect correctness. This is because correctness requires that all points, generated as $\{x_{i,q_i} = \mathsf{Eval}(K_{p,i}, n||t_n||(W+q_i))\} \cup \{\mathsf{Eval}(K_{p,i}, n||t_n||(2^\lambda - j))\}$ by the set of Turing Machins $\{\widetilde{M}_i(n, t_n, q_i)\}_{i \in \mathcal{I}}$ are distinct. This property follows from the security of the underlying PPRF. Below, we provide the formal statement, while the proof is given in Appendix A.4.

Theorem 7. *If the iO obfuscator* Obf *is correct (Definition 8) and* PPRF *is secure (Definition 2) then the ESS scheme of Construction 5.1 is correct.*

As for succinctness, we demonstrate that obfuscating $M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,i}, W, W^*, w_i, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*]$ produces an obfuscation of size $\mathsf{poly}(\lambda, \log W_n)$ (yielding a share of the same size) where $\sum_{i \in [n]} w_i = W_n$. Below, we provide the formal result, whose proof appears in Appendix A.5.

Theorem 8. *If the iO obfuscator* Obf *is succinct and efficient (Definition 10) and* PPRF *is succinct and efficient (Definition 3) then the ESS scheme of Construction 5.1 produces shares of size* $\mathsf{poly}(\lambda, \log W_n)$ *where* $\sum_{i \in [n]} w_i = W_n$ *for every party* $n \geq 1$.

Finally, the theorem below establishes the security of the scheme whose formal proof appears in Appendix A.6.

Theorem 9. *If the iO obfuscator* Obf *is secure (Definition 9) and* PPRF *is secure (Definition 2), then the scheme ESS scheme of Construction 5.1, for the evolving threshold access structure with fixed weights* $\hat{\mathcal{A}}^{\mathsf{fw-ethr}}$ *with weights of size at most* $|w| = O(\log \lambda)$*, is computationally private in the adaptive setting (Definition 15).*

Remark 1 (On supporting a priori *unknown but logarithmic weight sizes* $|w| \in O(\log \lambda)$*).* Whereas we presented Construction 5.1 assuming a known upper bound $|w|$ on all weight sizes, it is straightforward to see that the construction naturally supports weights whose size is *a priori* unknown but logarithmic in λ.

This can be achieved by slightly enlarging the hardcoded value space and the input space of the Turing Machine $M_i = M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,i}, W, W^*, w_i, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, q)$. In particular, we can set $q \in \{0,1\}^{\mathsf{suplog}(\lambda)}$ (input of M_i) and $w_i \in \{0,1\}^{\mathsf{suplog}(\lambda)}$ (hardcoded value of M_i). In this way, the Turing Machine is able to handle weights of *a priori* unknown size $|w| \in O(\log \lambda)$, since any weight of logarithmic size can be encoded using $\mathsf{suplog}(\lambda)$.

The proof of correctness, succinctness, and security of the related ESS scheme remains unaffected by this modification.

6 Evolving Threshold Access Structures with Dynamic Weights

Our last construction tackles the most general notion of dynamic thresholds, denoted by $\hat{\mathcal{A}}^{\mathsf{dw-ethr}}$. Here, the weights associated with parties may change over time, and the association in force is determined by the party with the largest index in the qualified set $\mathcal{I}$. In a nutshell, when the n-th party arrives, it is associated with a threshold t_n and a weight function $\mathsf{weight}_n : [n] \to \mathbb{N}$, which specifies the weight of the new n-th party and redefines the weights of all previous parties with index $< n$. Both the threshold t_n and the weight function weight_n are considered whenever n is the highest-index party participating in the reconstruction. The formal definition follows.

Definition 18 (Evolving Threshold Access Structure supporting Dynamic Weights [12]). *An evolving threshold access structure with dynamic weights $\hat{\mathcal{A}}^{\mathsf{dw-ethr}}$ is defined over a sequence of weights functions and threshold pairs $(\mathsf{weight}_1, t_1), \ldots, (\mathsf{weight}_n, t_n)$ satisfying the following condition:*

- *(qualified set) a set $\mathcal{I} = \{i_1, \ldots, i_k, n\}$ is authorized (i.e., $\mathcal{I} \in \hat{\mathcal{A}}^{\mathsf{dw-ethr}}_n$) if $t_n \leq \sum_{j \in \mathcal{I}} \mathsf{weight}_n(j)$ and $\mathsf{max}(\mathcal{I}) = n$.*

We observe that, when party n arrives, the threshold and the weights of all parties $i < n$ may arbitrarily change. However, for any qualified set $\mathcal{I} \in \mathcal{A}_n$, the reconstruction uses the threshold and weight functions corresponding to the maximal index in the set $\mathcal{I}$. This automatically preserves both monotonicity and rigidity.

6.1 Construction from iO for Turing Machines, PPRF, and SSB

In Figure 3 we provides the formalization of our Turing machine whereas in Construction 6.1 we present our last ESS scheme.

As in Construction 5.1, this construction supports the evolving access structure $\hat{\mathcal{A}}^{\mathsf{dw-ethr}}$ (Definition 18) with weights of size $|w| = O(\log \lambda)$. We note that the scheme below assumes an *a priori* known bound $|w| = O(\log \lambda)$. Nevertheless, the scheme is able to support also weights of *a priori* unknown but logarithmic size by applying the same technique described in Remark 1 of Construction 5.1. We recall that this construction achieves static privacy only, due to the use of SSB. We refer the reader to the technical overview for more details.

$M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, W^*, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, h, \pi, w, q)$:

if $k < i$ **or** $t \leq 0$ **then**
  return $\perp$
if $0 = \mathsf{SSB.Verify}(\mathsf{hk}, h, i, w, \pi)$ **or** $q \leq 0$ **or** $q > w$ **then**
  return $\perp$
if $k = \mathsf{pos}$ **and** $t = t^*$ **and** $q = q_1^*$ **then**
  return (x_1, v_1)
if $k = \mathsf{pos}$ **and** $t = t^*$ **and** $q \leq q_2^*$ **then**
  return $\big(\mathsf{Eval}(K'_p, k\|t\|h\|(W^* + q)),\ \mathsf{Eval}(K_f, k\|t\|h\|(W^* + q))\big)$
Initialize tapes eval and points as arrays of evaluations and points of size t
$j = 0$
if $k = \mathsf{pos}$ **and** $t = t^*$ **then**
  while $j < l^*$ **do**
    $\mathsf{eval}[j] = \mathsf{Eval}(K_f, k\|t\|h\|j)$
    $\mathsf{points}[j] = \mathsf{Eval}(K'_p, k\|t\|h\|j)$
    $j = j + 1$
  if $v_2 \neq \perp$ **then**
    $\mathsf{eval}[j] = v_2$
    $\mathsf{points}[j] = x_2$
    $j = j + 1$
while $j < t$ **do**
  $\mathsf{eval}[j] = \mathsf{Eval}(K_f, k\|t\|h\|(2^\lambda - j))$
  $\mathsf{points}[j] = \mathsf{Eval}(K_p, k\|t\|h\|(2^\lambda - j))$
  $j = j + 1$
$j = 0$, $S = 0$
while $j < t$ **do**
  $m = 0$, $L = 1$
  while $m < t$ **do**
    if $m \neq j$ **then**
      $$L = L \cdot \frac{\mathsf{Eval}(K_p, k\|t\|h\|(2^{i\lambda} + q)) - \mathsf{points}[m]}{\mathsf{points}[j] - \mathsf{points}[m]} \bmod p$$
    $m = m + 1$
  $S = S + L \cdot \mathsf{eval}[j] \bmod p$
  $j = j + 1$
return $\big(\mathsf{Eval}(K_p, k\|t\|h\|(2^{i\lambda} + q)),\ S\big)$

Fig. 3. The formal definition of the Turing Machine $M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, W^*, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, h, \pi, w, q)$ required in Construction 6.1. The input space of the machine is $\{0,1\}^{2\cdot\mathsf{suplog}(\lambda)+\ell_{\mathsf{hash}}+\ell_{\mathsf{open}}+2\cdot|w|}$ where ℓ_{open} and ℓ_{hash} are the sizes of opening and hashes produced by SSB, and $|w|$ is the maximum supported weight size. Given the description, it is syntactically clear that $M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, W^*, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, h, \pi, w, q)$, for any input $(k, t, h, \pi, w, q) \in \{0,1\}^{2\cdot\mathsf{suplog}(\lambda)+\ell_{\mathsf{hash}}+\ell_{\mathsf{open}}+2\cdot|w|}$, the machine always halt. We denote by $\ell^{\mathsf{max}}_{M^{\mathsf{dw-share}}} = 2\cdot\mathsf{suplog}(\lambda) + \ell_{\mathsf{hash}} + \ell_{\mathsf{open}} + 2\cdot|w|$ and $t^{\mathsf{max}}_{M^{\mathsf{dw-share}}}$ be its maximum input size and maximum running time (over the choice of all inputs $(k, t, h, \pi, w, q) \in \{0,1\}^{\ell^{\mathsf{max}}_{M^{\mathsf{dw-share}}}}$). The highlighted parts of the Turing machine are those required to prove the security of the scheme (due to the use of iO), and are therefore not needed for correctness.

Construction 6.1

Consider the following ingredients:

- $\mathbb{F}_p$ be a field where p is a (λ^c)-bit prime for constant $c > 1$;
- $\mathsf{SSB} = (\mathsf{Setup}, \mathsf{Hash}, \mathsf{Open}, \mathsf{Verify})$ be an SSB scheme. Let $\ell_{\mathsf{block}}, \ell_{\mathsf{open}}, \ell_{\mathsf{hash}}$ be the length of a block length, proofs π, and hashes of SSB. Since the SSB block will be the encoding of a weight of size $O(\log \lambda)$, we set $\ell_{\mathsf{block}} = |w|$ where $|w| = O(\log \lambda)$ is the maximum supported weight bit size;
- $\mathsf{PPRF} = (\mathsf{KeyGen}, \mathsf{Punc}, \mathsf{Eval})$ be a PPRF such that $\mathsf{Eval} : \{0,1\}^\lambda \times \{0,1\}^{2 \cdot \mathsf{suplog}(\lambda) + \lambda + \ell_{\mathsf{hash}}} \to \mathbb{F}_p$;[a]
- Obf be an iO obfuscator for the class of Turing machines $\mathcal{M}^{\mathsf{dw-share}} = \{M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, W^*, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, h, \pi, w, q)\}$ as defined in Figure 3. The values between square brackets are hardcoded values that we will modify during our security proof. The hardcoded values come from the following space $\mathsf{KeyGen}(1^\lambda)^3 \times \mathsf{Setup}(1^\lambda) \times \{0,1\}^{4 \cdot \mathsf{suplog}(\lambda)} \times \mathbb{F}_p^4 \times \{0,1\}^{3 \cdot \mathsf{suplog}(\lambda)}$.

We build a ESS scheme $\mathsf{SS} = (\mathsf{Share}, \mathsf{Recon})$ over message space $\mathbb{F}_p$ for the evolving threshold access structure $\hat{\mathcal{A}}^{\mathsf{dw-ethr}}$ (Definition 18) with weights of size at most $O(\log \lambda) = |w|$ as follows:

Sharing: When the n-th party arrives, the dealer proceeds as follows:

- If $n = 1$ (first party), sample $K_f \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, $K_p \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, $K'_p \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, and $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, 2^\lambda, 1)$.[b]
- Let $M_n = M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, 0, n, 0, 0, 0, 0, 0, \perp, 0, 0, 0](\cdot, \cdot, \cdot, \cdot, \cdot, \cdot)$ be a Turing machine as defined in Figure 3 and let $\ell^{\mathsf{max}}_{M^{\mathsf{dw-share}}}$ and $t^{\mathsf{max}}_{M^{\mathsf{dw-share}}}$ be the maximum input size and running time defined in Figure 3.
- Compute $\widetilde{M}_n \xleftarrow{\$} \mathsf{Obf}(1^\lambda, 1^{\ell^{\mathsf{max}}_{M^{\mathsf{dw-share}}}}, t^{\mathsf{max}}_{M^{\mathsf{dw-share}}}, M_n)$.
- Compute $h_n = \mathsf{Hash}(\mathsf{hk}, (\mathsf{weight}_n(i))_{i \in [n]})$.
- Compute $K_n = f_n(\mathsf{Eval}(K_p, n||t_n||h_n||0))$, where $f_n(X) \in \mathbb{F}_p$ is the unique polynomial of degree $t_n - 1$ obtained by interpolating the evaluations $\{\mathsf{Eval}(K_f, n||t_n||h_n||(2^\lambda - j))\}_{j \in [0, t_n)}$ and the points $\{\mathsf{Eval}(K_p, n||t_n||h_n||(2^\lambda - j))\}_{j \in [0, t_n)}$.
- Compute $\mathsf{ct}_n = K_n \cdot \mu$ where μ is the message.
- Finally, set the share of the n-th party to $\sigma_n = (\mathsf{hk}, \mathsf{ct}_n, \widetilde{M}_n, x_{n,0})$ where $x_{n,0} = \mathsf{Eval}(K_p, n||t_n||h_n||0)$.

Reconstruction: The reconstruction algorithm $\mathsf{Recon}((\sigma_i)_{i \in \mathcal{I}}, \mathcal{I}, \hat{\mathcal{A}}_n^{\mathsf{dw-ethr}})$, given the shares $(\sigma_i)_{i \in \mathcal{I}} = (\mathsf{hk}, \mathsf{ct}_i, \widetilde{M}_i, x_{i,0})$, and a (minimal) qualified subset $\mathcal{I} = \{i_1, i_2, \ldots, n\} \in \hat{\mathcal{A}}_n^{\mathsf{dw-ethr}}$ (i.e., $W_{\mathcal{I}} = \sum_{i \in \mathcal{I}} \mathsf{weight}_n(i) \geq t_n$ and $\mathsf{max}(\mathcal{I}) = n$), it proceeds as follows:

- Let (weight_n, t_n) be the weights function and threshold pair which is part of the definition of the access structure $\hat{\mathcal{A}}_n^{\mathsf{dw-ethr}}$ taken as input.
- Compute $h_n = \mathsf{Hash}(\mathsf{hk}, (\mathsf{weight}_n(i))_{i\in[n]})$.
- For every $i \in \mathcal{I}$ and $q_i \in [w_i]$, execute the obfuscated Turing machine $(x_{i,q_i}\mathsf{sh}_{i,q_i}) = \widetilde{M}_i(n, t_n, h_n, \pi_i, \mathsf{weight}_n(i), q_i)$ where $\pi_i = \mathsf{Open}(\mathsf{hk}, (\mathsf{weight}_n(i))_{i\in[n]}, i)$.
- Compute $K'_n = \mathsf{Lagrange}(x_{n,0}, (\mathsf{sh}_{i,q_i})_{i\in\mathcal{I},q_i\in[w_i]}, (x_{i,q_i})_{i\in\mathcal{I},q_i\in[w_i]})$.
- Finally, output $\mu' = \mathsf{ct}_n \cdot (K'_n)^{-1}$.

[a] Note that $\mathsf{suplog}(\lambda)$ bits are sufficient to encode in binary any number of parties $n \in \mathsf{poly}(\lambda)$ and any threshold $t_n \leq n$.

[b] The latter PPRF key $K_{p'}$ is sampled and hardcoded into $M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, 0, n, 0, 0, 0, 0, 0, \bot, 0, 0, 0]$ solely to simplify the security proof. Indeed, K'_p is used only in the proof and not required for the correctness of the construction.

As for Construction 5.1, this construction does not achieve perfect correctness. Below, we report the formal statement whose proof appears in Appendix A.7.

Theorem 10. *If the iO obfuscator* Obf *is correct (Definition 8) and* PPRF *is secure (Definition 2) then the ESS scheme of Construction 6.1 is correct.*

As usual, succinctness follows by demonstrating that $M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, 0, n, 0, 0, 0, 0, 0, \bot, 0, 0, 0]$ produces an obfuscation of size $\mathsf{poly}(\lambda, \log W_n)$ (yielding a share of the same size) where $\sum_{i\in[n]} \mathsf{weight}_n(i) = W_n$. Below, we provide the formal result whose proof appears in Appendix A.8.

Theorem 11. *If the iO obfuscator* Obf *is succinct and efficient (Definition 10),* PPRF *is succinct and efficient (Definition 3), and* SSB *is succinct and efficient (Definition 7) then the ESS scheme of Construction 6.1 produces shares of size* $\mathsf{poly}(\lambda, \log W_n)$ *where* $\sum_{i\in[n]} \mathsf{weight}_n(i) = W_n$ *for every party* $n \geq 1$.

Finally, the theorem below establishes the security of the scheme whose formal proof appears in Appendix A.9. We recall that this construction only achieves static security.

Theorem 12. *If the iO obfuscator* Obf *is secure (Definition 9)* PPRF *is secure (Definition 2), and* SSB *is index hiding (Definition 5) and somewhere perfectly binding (Definition 6), then the scheme ESS scheme of Construction 6.1, for the evolving threshold access structure with dynamic weights* $\hat{\mathcal{A}}^{\mathsf{dw-ethr}}$ *with weights of size at most* $|w| = O(\log \lambda)$*, is a computationally private in the static setting (Definition 15).*

Acknowledgements

Danilo Francati was supported by project SERICS (PE00000014), under the MUR National Recovery and Resilience Plan funded by the European Union-NextGenerationEU. Daniele Venturi is member of the Gruppo Nazionale Calcolo

Scientifico Istituto Nazionale di Alta Matematica (GNCS-INdAM). His research was supported by project SERICS (PE00000014) and by project PARTHENON (B53D23013000006), under the MUR National Recovery and Resilience Plan funded by the European Union-NextGenerationEU, and by project BEAT, funded by Sapienza University of Rome.

References

1. Alon, B., Beimel, A., Ben David, T., Omri, E., Paskin-Cherniavsky, A.: New upper bounds for evolving secret sharing via infinite branching programs. In: Boyle, E., Mahmoody, M. (eds.) Theory of Cryptography. pp. 548–580. Springer Nature Switzerland, Cham (2025)
2. Avitabile, G., Döttling, N., Magri, B., Sakkas, C., Wohnig, S.: Signature-based witness encryption with compact ciphertext. In: Advances in Cryptology – ASIACRYPT 2024: 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9–13, 2024, Proceedings, Part I. p. 3–31. Springer-Verlag, Berlin, Heidelberg (2024). https://doi.org/10.1007/978-981-96-0875-1_1, https://doi.org/10.1007/978-981-96-0875-1_1
3. Beimel, A., Othman, H.: Evolving ramp secret-sharing schemes. In: Catalano, D., De Prisco, R. (eds.) Security and Cryptography for Networks. pp. 313–332. Springer International Publishing, Cham (2018)
4. Beimel, A., Othman, H.: Evolving ramp secret sharing with a small gap. In: Canteaut, A., Ishai, Y. (eds.) Advances in Cryptology – EUROCRYPT 2020. pp. 529–555. Springer International Publishing, Cham (2020)
5. Blakley, G.R.: Safeguarding cryptographic keys. Proceedings of AFIPS 1979 National Computer Conference **48**, 313–317 (1979)
6. Boyle, E., Goldwasser, S., Ivan, I.: Functional signatures and pseudorandom functions. In: Krawczyk, H. (ed.) Public-Key Cryptography – PKC 2014. pp. 501–519. Springer Berlin Heidelberg, Berlin, Heidelberg (2014)
7. D'Arco, P., De Prisco, R., De Santis, A.: Secret sharing schemes for infinite sets of participants: A new design technique. Theoretical Computer Science **859**, 149–161 (2021). https://doi.org/https://doi.org/10.1016/j.tcs.2021.01.019, https://www.sciencedirect.com/science/article/pii/S0304397521000347
8. David, T.B., Paskin-Cherniavsky, A.: Tight lower bounds and new upper bounds for evolving CDS. Cryptology ePrint Archive, Paper 2025/292 (2025), https://eprint.iacr.org/2025/292
9. Desmedt, Y., Dutta, S., Morozov, K.: Evolving perfect hash families: A combinatorial viewpoint of evolving secret sharing. In: Mu, Y., Deng, R.H., Huang, X. (eds.) Cryptology and Network Security. pp. 291–307. Springer International Publishing, Cham (2019)
10. Devadas, L., Jain, A., Waters, B., Wu, D.J.: Succinct witness encryption for batch languages and applications. In: Hanaoka, G., Yang, B.Y. (eds.) Advances in Cryptology – ASIACRYPT 2025. pp. 130–162. Springer Nature Singapore, Singapore (2026)
11. Dutta, S., Roy, P.S., Fukushima, K., Kiyomoto, S., Sakurai, K.: Secret sharing on evolving multi-level access structure. In: You, I. (ed.) Information Security Applications. pp. 180–191. Springer International Publishing, Cham (2020)

12. Francati, D., Giammusso, S., Venturi, D.: Taming adaptive security and new access structures in evolving secret sharing. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 67–98. Springer (2025)
13. Francati, D., Venturi, D.: Evolving secret sharing made short. In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology – ASIACRYPT 2024. pp. 69–99. Springer Nature Singapore, Singapore (2025)
14. Francati, D., Venturi, D.: Evolving secret sharing revisited: computational security and succinctness: D. francati, d. venturi. Designs, Codes and Cryptography pp. 1–53 (2025)
15. Garg, S., Gentry, C., Halevi, S., Raykova, M., Sahai, A., Waters, B.: Candidate indistinguishability obfuscation and functional encryption for all circuits. In: 2013 IEEE 54th Annual Symposium on Foundations of Computer Science. pp. 40–49 (2013). https://doi.org/10.1109/FOCS.2013.13
16. Garg, S., Srinivasan, A.: A simple construction of io for turing machines. In: Beimel, A., Dziembowski, S. (eds.) Theory of Cryptography. pp. 425–454. Springer International Publishing, Cham (2018)
17. Hubacek, P., Wichs, D.: On the communication complexity of secure function evaluation with long output. In: Proceedings of the 2015 Conference on Innovations in Theoretical Computer Science. p. 163–172. ITCS '15, Association for Computing Machinery, New York, NY, USA (2015). https://doi.org/10.1145/2688073.2688105, https://doi.org/10.1145/2688073.2688105
18. Jafargholi, Z., Kamath, C., Klein, K., Komargodski, I., Pietrzak, K., Wichs, D.: Be adaptive, avoid overcommitting. In: Katz, J., Shacham, H. (eds.) Advances in Cryptology – CRYPTO 2017. pp. 133–163. Springer International Publishing, Cham (2017)
19. Komargodski, I., Naor, M., Yogev, E.: How to share a secret, infinitely. In: Hirt, M., Smith, A. (eds.) Theory of Cryptography. pp. 485–514. Springer Berlin Heidelberg, Berlin, Heidelberg (2016)
20. Komargodski, I., Paskin-Cherniavsky, A.: Evolving secret sharing: Dynamic thresholds and robustness. In: Kalai, Y., Reyzin, L. (eds.) Theory of Cryptography. pp. 379–393. Springer International Publishing, Cham (2017)
21. Koppula, V., Lewko, A.B., Waters, B.: Indistinguishability obfuscation for turing machines with unbounded memory. In: Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing. p. 419–428. STOC '15, Association for Computing Machinery, New York, NY, USA (2015). https://doi.org/10.1145/2746539.2746614, https://doi.org/10.1145/2746539.2746614
22. Mazor, N.: A lower bound on the share size in evolving secret sharing. In: 4th Conference on Information-Theoretic Cryptography (ITC 2023). Schloss Dagstuhl-Leibniz-Zentrum für Informatik (2023). https://doi.org/10.4230/LIPIcs.ITC.2023.2
23. Paskin-Cherniavsky, A.: How to infinitely share a secret more efficiently. Cryptology ePrint Archive, Paper 2016/1088 (2016), https://eprint.iacr.org/2016/1088
24. Shamir, A.: How to share a secret. Commun. ACM **22**(11), 612–613 (Nov 1979)
25. Xing, C., Yuan, C.: Evolving secret sharing schemes based on polynomial evaluations and algebraic geometry codes. Cryptology ePrint Archive, Paper 2021/1115 (2021), https://eprint.iacr.org/2021/1115

A Supporting Proofs

A.1 Proof of Theorem 4

Proof. Let $\lambda \in \mathbb{N}$ and $\mathcal{I} = \{i_1, i_2, \ldots, n\} \in \hat{\mathcal{A}}^{\mathsf{ethr}}$, i.e., $|\mathcal{I}| \geq t_n$ and $\mathsf{max}(\mathcal{I}) = n$. Let us show $(K'_n)^{-1} \cdot K_n \cdot \mu = \mu$. First, we have $K_n = \mathsf{Lagrange}(0, (\mathsf{Eval}(K_f, n||t_n||(2^\lambda - j)))_{j\in[0,t_n)}, (2^\lambda - j)_{j\in[0,t_n)})$.

Each party $i \in \mathcal{I}$ holds $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$, where $\mathsf{ct}_i = K_i \cdot \mu$, with $K_i = f_i(0)$ and $\widetilde{M_i} \xleftarrow{\$} \mathsf{Obf}(1^\lambda, 1^{\ell^{\mathsf{max}}_{M^{\mathsf{share}}}}, t^{\mathsf{max}}_{M^{\mathsf{share}}}, M_i)$ with $M_i = M_p^{\mathsf{share}}[K_f, i, 0, 0, 0](\cdot, \cdot)$. During reconstruction, for each party $i \in \mathcal{I}$, we run the obfuscated program $\mathsf{sh}_i = \widetilde{M_i}(n, t_n)$. By correctness of Obf, this outputs the values generated by $M_p^{\mathsf{share}}[K_f, i, \mathsf{pos}, t^*, v](k, t)$ which runs the code in figure 1 and has harcoded values: $K_f \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, $i = i$, $\mathsf{pos} = t^* = v = 0$. Clearly $n \geq i$, and $t_n > 0$. Then, the circuit computes for every $j \in [0, t_n)$ the evaluations $\mathsf{eval}[j] = \mathsf{Eval}(K_f, n||t_n||(2^\lambda - j))$. Finally the circuit computes $\mathsf{Lagrange}(i, (\mathsf{eval}[j])_{j\in[0,t_n)}, (2^\lambda - j)_{j\in[0,t_n)})$. Hence, for every $i \in \mathcal{I}$, the parties obtain $\mathsf{sh}_i = f_n(i)$, and since $\mathcal{I} \in \hat{\mathcal{A}}^{\mathsf{ethr}}$, we are guaranteed that parties collectively have at least t_n shares $(\mathsf{sh}_i)_{i\in\mathcal{I}}$. Thus, parties can compute $K'_n = \mathsf{Lagrange}(0, (\mathsf{sh}_i)_{i\in\mathcal{I}}, \mathcal{I})$, which is exactly $f_n(0)$. Finally, since $K'_n = f_n(0) = K_n$, we have that $(K_n)^{-1} \cdot K_n \cdot \mu = \mu$.

A.2 Proof of Theorem 5

Proof. We need to prove that the size of each share $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$ is $\mathsf{poly}(\lambda, \log i)$. First, we have that $\mathsf{ct}_i = K_i \cdot \mu$: Hence, $|\mathsf{ct}_i| \in \mathsf{poly}(\lambda)$. To conclude the proof, it remains to show that $|\widetilde{M_i}|$ is in $\mathsf{poly}(\lambda, \log i)$.

Recall that $\widetilde{M_i}$ is computed as $\widetilde{M_i} \xleftarrow{\$} \mathsf{Obf}(1^\lambda, 1^{\ell^{\mathsf{max}}_{M^{\mathsf{share}}}}, t^{\mathsf{max}}_{M^{\mathsf{share}}}, M_i)$ with $M_i = M_p^{\mathsf{share}}[K_f, i, \mathsf{pos}, t^*, v]$. By the succinctness and efficiency requirements on Obf, $|\widetilde{M_i}|$ is in $\mathsf{poly}(|M_p^{\mathsf{share}}|, \log t^{\mathsf{max}}_{M^{\mathsf{share}}}, \ell^{\mathsf{max}}_{M^{\mathsf{share}}}, \lambda)$, where:

1. $|M_p^{\mathsf{share}}|$ is the description size of the Turing Machine;
2. $t^{\mathsf{max}}_{M^{\mathsf{share}}}$ is the maximum runtime of the Turing Machine;
3. $\ell^{\mathsf{max}}_{M^{\mathsf{share}}}$ is the maximum input size of the Turing Machine.

According to Figure 1, we have $\ell^{\mathsf{max}}_{M^{\mathsf{share}}} = 2 \cdot \mathsf{suplog}(\lambda)$. Below, we measure the remaining two parameters.

Regarding the maximum runtime $t^{\mathsf{max}}_{M^{\mathsf{share}}}$:

- Comparisons with $i, 0, \mathsf{pos}, t^*$ can be executed in time $\mathsf{poly}(\mathsf{suplog}(\lambda))$;
- $\mathsf{Eval}(K_f, \cdot)$ can be executed in $(2 \cdot \mathsf{suplog}(\lambda) + \lambda) \cdot \mathsf{poly}(\lambda) = \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$;
- Since $t \leq n$ and $|n| \leq \mathsf{suplog}(\lambda)$, the first while-loop has runtime $2^{\mathsf{suplog}(\lambda)} \cdot \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$;
- Modular arithmetic in $\mathbb{F}_p$ can be executed in time $\mathsf{poly}(\log p) = \mathsf{poly}(\lambda)$ since p is a prime of $\lambda + 1$ bits;
- Since $t \leq n$, the last while-loop (line 11) has runtime in $\left(2^{\mathsf{suplog}(\lambda)}\right)^2 \cdot \mathsf{poly}(\lambda) = 2^{2 \cdot \mathsf{suplog}(\lambda)} \cdot \mathsf{poly}(\lambda)$.

Hence, the maximum runtime is bounded by

$$t^{\mathsf{max}}_{M^{\mathsf{share}}} = O\left(2^{\mathsf{suplog}(\lambda)}\right) \cdot \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)).$$

Regarding the description size $|M_p^{\mathsf{share}}|$, we start observing that, as described in Construction 4.1, the sizes of hardwired values (which are part of the description of the Turing Machine) are $|K_f| = |\mathsf{KeyGen}(1^\lambda)| = \mathsf{poly}(\lambda)$, $|i| = |\mathsf{pos}| = |t^*| = \mathsf{suplog}(\lambda)$, and $|v| = \log p = \log(\lambda + 1)$ since $v \in \mathbb{F}_p$.

Then, the computation performed by the Turing Machine can be described using the following size:

- Comparisons with $i, 0, \mathsf{pos}, t^*$ can be described in size $\mathsf{poly}(\mathsf{suplog}(\lambda))$;
- $\mathsf{Eval}(K_f, \cdot)$ can be described in $(2 \cdot \mathsf{suplog}(\lambda) + \lambda) \cdot \mathsf{poly}(\lambda) = \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$;
- Since $t \leq n$ and $|n| \in \mathsf{suplog}(\lambda)$, the first while-loop has description size $\mathsf{suplog}(\lambda) + \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$;
- Modular arithmetic in $\mathbb{F}_p$ can be described in size $\mathsf{poly}(\log p) = \mathsf{poly}(\lambda)$;
- Since $t \leq n$ and $|n| \in \mathsf{suplog}(\lambda)$, the second while-loop (line 11) can be described in size $2 \cdot \mathsf{suplog}(\lambda) + \mathsf{poly}(\lambda)$.

We conclude that the description size is bounded by

$$|M_p^{\mathsf{share}}| = \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)).$$

By combining the bounds of $|M_p^{\mathsf{share}}|$, $t^{\mathsf{max}}_{M^{\mathsf{share}}}$, and $\ell^{\mathsf{max}}_{M^{\mathsf{share}}}$, we conclude:

$$\begin{aligned}
|\widetilde{M}_i| &= \mathsf{poly}(|M_p^{\mathsf{share}}|, \log t^{\mathsf{max}}_{M^{\mathsf{share}}}, \ell^{\mathsf{max}}_{M^{\mathsf{share}}}, \lambda) \\
&= \mathsf{poly}(\mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)), O(\log(2^{\mathsf{suplog}(\lambda)})) \cdot \\
&\qquad \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)), 2 \cdot \mathsf{suplog}(\lambda), \lambda) \\
&= \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)).
\end{aligned}$$

Finally, since $i \in [n]$ for an apriori unknown $n \in \mathsf{poly}(\lambda)$ (and thus $\log i = O(\log \lambda)$), $\mathsf{suplog}(\lambda) = \log^c(\lambda)$ for some constant $c > 1$, and all $\mathsf{suplog}(\lambda)$ values are chosen so as to allow the Turing machine to perform its computation correctly for every pair (k, t) whose values can depend on n (i.e., worst-case size is $O(\log n)$), for an apriori unknown value $n \in \mathsf{poly}(\lambda)$, we can rewrite the equation above as follows:

$$|\widetilde{M}_i| = \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)) = \mathsf{poly}(\lambda, \log i) \text{ where } i \in [n].$$

This concludes the proof.

A.3 Proof of Theorem 6

Restatement of Theorem 6. *If the iO obfuscator* Obf *is secure (Definition 9), and* PPRF *is secure (Definition 2), then the scheme ESS of Construction 4.1, for the evolving threshold access structures* $\hat{\mathcal{A}}^{\mathsf{ethr}}$*, is computationally private in the adaptive setting (Definition 15).*

Proof. Let $\mu_0, \mu_1 \in \mathcal{M}$ be the two messages chosen by the adversary A in the game defining the adaptive computational privacy of SS, $\boldsymbol{G}^{\mathsf{priv}}_{\mathsf{SS},\mathsf{A}}(\lambda, b)$. W.l.o.g. we assume that on each adversary's oracle query to $\mathcal{O}_{\mathsf{share}}(n_g, \mathcal{U})$, $|\mathcal{U}| \leq 1$.

Consider the following hybrid experiments.

$\mathbf{H}_0(\lambda, b)$: This experiment is identical to $\boldsymbol{G}^{\mathsf{priv}}_{\mathsf{SS},\mathsf{A}}(\lambda, b)$.

$\mathbf{H}^{i^*}_1(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}^{i^*}_{19}(\lambda, b)$ (with $\mathbf{H}^0_{19}(\lambda, b) = \mathbf{H}_0(\lambda, b)$), except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot, \cdot)$ such that, on input (n_g, z^*), if party $z^* \leq i^*$ we change the machine used to generate $\widetilde{M}_{z^*}$ so that, on input $(k, t) = (i^*, t_{i^*})$, it outputs the explicitly hardwired value $\mathsf{sh}^{t_{i^*}}_{z^*}$ (defined as $\mathsf{Lagrange}(z^*, \{\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j))\}_{j\in[0,t_{i^*})}, \{2^\lambda - j\}_{j\in[0,t_{i^*})})$), and otherwise behaves as before. Hence, we replace the machine M^{share}_p we use to compute $\widetilde{M}_{z^*}$ by

$$M_0 = M^{\mathsf{share}}_p[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}^{t_{i^*}}_{z^*}] \quad \text{instead of} \quad M_1 = M^{\mathsf{share}}_p[K_f, z^*, 0, 0, 0].$$

$\mathbf{H}^{i^*}_2(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}^{i^*}_1(\lambda, b)$, except that the challenger samples $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||2^\lambda\})$ and we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot, \cdot)$ such that, on input (n_g, z^*), the challenger replaces the machine used to compute the obfuscation $\widetilde{M}_{z^*}$ by

$$M_0 = \begin{cases} M^{\mathsf{share}}_p[K_{f,*}, z^*, i^*, t_{i^*}, \mathsf{sh}^{t_{i^*}}_{z^*}] \text{ if } z^* \leq i^* \\ M^{\mathsf{share}}_p[K_{f,*}, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

instead of

$$M_1 = \begin{cases} M^{\mathsf{share}}_p[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}^{t_{i^*}}_{z^*}] \text{ if } z^* \leq i^* \\ M^{\mathsf{share}}_p[K_f, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

$\mathbf{H}^{i^*}_3(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}^{i^*}_2(\lambda, b)$, except that the challenger replaces the value $\mathsf{Eval}(K_f, i^*||t_{i^*}||2^\lambda)$ used in defining the polynomial f_{i^*} by a uniformly random value $r_{i^*} \xleftarrow{\$} \mathbb{Z}_p$. The key $K_{i^*} = f_{i^*}(0)$, and all the values $\mathsf{sh}^{t_{i^*}}_{z_j} = f_{i^*}(z_j)$ hardcoded into $\widetilde{M}_{z_j}$ by the oracle $\mathcal{O}_{\mathsf{share}}(\cdot, \cdot)$ for all $z_j \leq i^*$ are defined consistently as Lagrange interpolations of the resulting polynomial. Specifically,

$$K_{i^*} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{-x_m}{x_j - x_m},$$

and

$$\mathsf{sh}^{t_{i^*}}_{z_j} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{z_j - x_m}{x_j - x_m},$$

with

$$y_j = \begin{cases} r_{i^*} \text{ if } j = 0 \\ \mathsf{Eval}(K_f, i^* || t_{i^*} || (2^\lambda - j)) \text{ if } j \in [1, t_{i^*}) \end{cases}$$

and $x_j = 2^\lambda - j$ for all $j \in [0, t_{i^*})$.

$\mathbf{H}_4^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_3^{i^*}(\lambda, b)$, except that the challenger samples $K_{i^*} \xleftarrow{\$} \{0,1\}^\lambda$ instead of $K_{i^*} = f_{i^*}(0)$. All the values $\mathsf{sh}_{z_j}^{t_{i^*}} = f_{i^*}(z_j)$ hardcoded into $\widetilde{M}_{z_j}$ by the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ for all $z_j \leq i^*$, are defined consistently as Lagrange interpolations of the resulting polynomial. Specifically,

$$\mathsf{sh}_{z_j}^{t_{i^*}} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{z_j - x_m}{x_j - x_m},$$

with

$$y_j = \begin{cases} K_{i^*} \text{ if } j = 0 \\ \mathsf{Eval}(K_f, i^* || t_{i^*} || (2^\lambda - j)) \text{ if } j \in [1, t_{i^*}) \end{cases}$$

and

$$x_j = \begin{cases} 0 \text{ if } j = 0 \\ 2^\lambda - j \text{ for all } j \in [1, t_{i^*}) \end{cases}$$

$\mathbf{H}_5^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_4^{i^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ such that, on input (n_g, z^*), the challenger replaces the machine used to compute the obfuscation $\widetilde{M}_{z^*}$ by

$$M_0 = \begin{cases} M_p^{\mathsf{share}}[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_f, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

instead of

$$M_1 = \begin{cases} M_p^{\mathsf{share}}[K_{f,*}, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_{f,*}, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

We will now run a series of hybrids: $\mathbf{H}_6^{i^*,j^*}(\lambda, b), \ldots, \mathbf{H}_9^{i^*,j^*}(\lambda, b)$ for $i^* \in [\mathsf{poly}(\lambda)]$, and $j^* \in [1, t_{i^*})$:

- $\mathbf{H}_6^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [1, t_{i^*})$: This is identical to $\mathbf{H}_9^{i^*,j^*-1}(\lambda, b)$ with $\mathbf{H}_9^{i^*,-1}(\lambda, b) = \mathbf{H}_5^{i^*}(\lambda, b)$, except that the challenger samples $K_{f,*} = \mathsf{Punc}(K_f, \{i^* || t_{i^*} || (2^\lambda - j^*)\})$ and we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ such that, on input (n_g, z^*), the challenger replaces the machine used to compute the obfuscation $\widetilde{M}_{z^*}$ by

$$M_0 = \begin{cases} M_p^{\mathsf{share}}[K_{f,*}, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_{f,*}, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

instead of

$$M_1 = \begin{cases} M_p^{\mathsf{share}}[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_f, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

– $\mathbf{H}_7^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [1, t_{i^*})$: This is identical to $\mathbf{H}_6^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ as follows. We restrict attention to the subsequence of oracle queries $\mathcal{Q} = (q_1, \ldots, q_m)$ made to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, where each query q_j has parameters (n_j, z_j) with $z_j \leq i^*$; all other queries with $z_j > i^*$ are left unchanged.
When we reach the query $q_{j^*} \in \mathcal{Q}$ (i.e., the j^*-th query with $z_t \leq i^*$) the challenger replaces the value $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ used in defining the polynomial f_{i^*} by a uniformly random value $r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$.
All the values $(\mathsf{sh}_{z_j}^{t_{i^*}} = f_{i^*}(z_j))_{j \geq j^*}$ hardcoded into $\widetilde{M}_{z_j}$ by the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ are defined consistently as Lagrange interpolations of the resulting polynomial. Specifically,

$$\mathsf{sh}_{z_j}^{t_{i^*}} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{z_j - x_m}{x_j - x_m},$$

with

$$y_j = \begin{cases} K_{i^*} \text{ if } j = 0 \\ \mathsf{sh}_{z_j}^{t_{i^*}} \text{ if } 0 < j < j^* \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

and

$$x_j = \begin{cases} 0 \text{ if } j = 0 \\ z_j \text{ if } 0 < j < j^* \\ 2^\lambda - j \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

– $\mathbf{H}_8^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [1, t_{i^*})$: This is identical to $\mathbf{H}_7^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ as follows. We restrict attention to the subsequence of oracle queries $\mathcal{Q} = (q_1, \ldots, q_m)$ made to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, where each query q_j has parameters (n_j, z_j) with $z_j \leq i^*$; all other queries with $z_j > i^*$ are left unchanged.
When we reach the query $q_{j^*} \in \mathcal{Q}$ (i.e., the j^*-th query with $z_t \leq i^*$) the challenger samples $\mathsf{sh}_{z_{j^*}}^{t_{i^*}} \xleftarrow{\$} \mathbb{Z}_p$ and sets the hardcoded share in $\widetilde{M}_{z_{j^*}}$ to $\mathsf{sh}_{z_{j^*}}^{t_{i^*}}$. It then defines $f_{i^*}(x)$ as the unique degree $t_{i^*} - 1$ polynomial obtained by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} K_{i^*} \text{ if } j = 0 \\ \mathsf{sh}_{z_j}^{t_{i^*}} \text{ if } 0 < j \leq j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} 0 \text{ if } j = 0 \\ z_j \text{ if } 0 < j \leq j^* \\ 2^\lambda - j \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

Consequently, we change how the challenger computes $(\mathsf{sh}_{z_j}^{t_{i^*}} = f_{i^*}(z_j))_{j>j^*}$, (if any), to be hardcoded into the obfuscations $(\widetilde{M}_{z_j})_{j>j^*}$. In particular,

$$\mathsf{sh}_{z_j}^{t_{i^*}} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{z_j - x_m}{x_j - x_m}.$$

– $\mathbf{H}_9^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [1, t_{i^*})$: This is identical to $\mathbf{H}_8^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ such that, on input (n_g, z^*), the challenger replaces the machine used to compute the obfuscation $\widetilde{M}_{z^*}$ by

$$M_0 = \begin{cases} M_p^{\mathsf{share}}[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_f, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

instead of

$$M_1 = \begin{cases} M_p^{\mathsf{share}}[K_{f,*}, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_{f,*}, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

$\mathbf{H}_{10}^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_9^{i^*,t_{i^*}-1}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ such that, on input (n_g, z^*), if $z^* = i^*$ the ciphertext of party i^* is changed to $\mathsf{ct}_{i^*} \xleftarrow{\$} \mathbb{Z}_p$ instead of $\mathsf{ct}_{i^*} := K_{i^*} \cdot \mu_b$.

We will now run a series of hybrids: $\mathbf{H}_{11}^{i^*,j^*}(\lambda, b), \ldots, \mathbf{H}_{14}^{i^*,j^*}(\lambda, b)$ for $i^* \in [\mathsf{poly}(\lambda)]$, and $j^* \in (t_{i^*}, 1]$:

– $\mathbf{H}_{11}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t_{i^*}, 1]$: This is identical to $\mathbf{H}_{14}^{i^*,j^*-1}(\lambda, b)$ with $\mathbf{H}_{14}^{i^*,-1}(\lambda, b) = \mathbf{H}_{10}^{i^*}(\lambda, b)$, except that the challenger samples $K_{f,*} = \mathsf{Punc}(K_f, \{i^* || t_{i^*} || (2^\lambda - j^*)\})$ and we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot;\cdot)$ such that, on input (n_g, z^*), the challenger replaces the machine used to compute the obfuscation $\widetilde{M}_{z^*}$ by

$$M_0 = \begin{cases} M_p^{\mathsf{share}}[K_{f,*}, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_{f,*}, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

instead of

$$M_1 = \begin{cases} M_p^{\mathsf{share}}[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_f, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

– $\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t_{i^*}, 1]$: This is identical to $\mathbf{H}_{11}^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ as follows. We

restrict attention to the subsequence of oracle queries $\mathcal{Q} = (q_1, \ldots, q_m)$ made to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, where each query q_j has parameters (n_j, z_j) with $z_j \leq i^*$; all other queries with $z_j > i^*$ are left unchanged.
When we reach the query $q_{j^*} \in \mathcal{Q}$ (i.e., the j^*-th query with $z_t \leq i^*$) the challenger replaces the value $\mathsf{sh}_{z_{j^*}}^{t_{i^*}}$ used in defining the polynomial f_{i^*} by a uniformly random value $r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$.
All the values $(\mathsf{sh}_{z_j}^{t_{i^*}})_{j \geq j^*}$ hardcoded into $\widetilde{M}_{z_j}$ by the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ are defined consistently as Lagrange interpolations of the resulting polynomial, defined by:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^* || t_{i^*} || (2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{sh}_{z_j}^{t_{i^*}} \text{ if } 0 < j < j^* \\ K_{i^*} \text{ if } j = 0 \end{cases}$$

and

$$x_j = \begin{cases} 2^\lambda - j \text{ if } j \in [j^*, t_{i^*}) \\ z_j \text{ if } 0 < j < j^* \\ 0 \text{ if } j = 0 \end{cases}$$

– $\mathbf{H}_{13}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t_{i^*}, 1]$: This is identical to $\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ as follows. We restrict attention to the subsequence of oracle queries $\mathcal{Q} = (q_1, \ldots, q_m)$ made to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, where each query q_j has parameters (n_j, z_j) with $z_j \leq i^*$; all other queries with $z_j > i^*$ are left unchanged.
When we reach the query $q_{j^*} \in \mathcal{Q}$ (i.e., the j^*-th query with $z_t \leq i^*$) the challenger replaces the value r_{i^*,j^*} used in defining the polynomial f_{i^*} by $\mathsf{Eval}(K_f, i^* || t_{i^*} || (2^\lambda - j^*))$.
All the values $(\mathsf{sh}_{z_j}^{t_{i^*}})_{j \geq j^*}$ hardcoded into $\widetilde{M}_{z_j}$ by the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ are defined consistently as Lagrange interpolations of the resulting polynomial, defined by:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^* || t_{i^*} || (2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \\ \mathsf{sh}_{z_j}^{t_{i^*}} \text{ if } 0 < j < j^* \\ K_{i^*} \text{ if } j = 0 \end{cases}$$

and

$$x_j = \begin{cases} 0 \text{ if } j = 0 \\ z_j \text{ if } 0 < j < j^* \\ 2^\lambda - j \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

– $\mathbf{H}_{14}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t_{i^*}, 1]$: This is identical to $\mathbf{H}_{13}^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ such that, on input

(n_g, z^*), the challenger replaces the machine used to compute th obfuscation $\widetilde{M}_{z^*}$ by

$$M_0 = \begin{cases} M_p^{\mathsf{share}}[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_f, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

instead of

$$M_1 = \begin{cases} M_p^{\mathsf{share}}[K_{f,*}, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_{f,*}, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

$\mathbf{H}_{15}^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_{14}^{i^*,1}(\lambda, b)$, except that the challenger samples $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||2^\lambda\})$ and we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ such that, on input (n_g, z^*), the challenger replaces the machine used to compute the obfuscation $\widetilde{M}_{z^*}$ by

$$M_0 = \begin{cases} M_p^{\mathsf{share}}[K_{f,*}, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_{f,*}, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

instead of

$$M_1 = \begin{cases} M_p^{\mathsf{share}}[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_f, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

$\mathbf{H}_{16}^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_{15}^{i^*}(\lambda, b)$, except that we change how the challenger computes K_{i^*}. In particular, instead of $K_{i^*} \xleftarrow{\$} \{0,1\}^\lambda$, the challenger samples a uniformly random r_{i^*} and computes

$$K_{i^*} = f_{i^*}(0) = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{-x_m}{x_j - x_m},$$

with

$$y_j = \begin{cases} r_{i^*} \text{ if } j = 0 \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j >\in [1, t_{i^*}) \end{cases}$$

and $x_j = (2^\lambda - j)$ for all $j \in [0, t_{i^*})$. Consequently all values $\mathsf{sh}_{z_j}^{t_{i^*}}$ hardcoded into $\widetilde{M}_{z_j}$ by the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ are defined consistently as Lagrange interpolations of the resulting polynomial.

$\mathbf{H}_{17}^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_{16}^{i^*}(\lambda, b)$, except that we change how the challenger computes K_{i^*}. In particular, the challenger computes $K_{i^*} = \mathsf{Lagrange}(0, \{\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j))\}_{j \in [0, t_{i^*})}, \{2^\lambda - j\}_{j \in [0, t_{i^*})})$. Moreover all values $\mathsf{sh}_{z_j}^{t_{i^*}}$ hardcoded into $\widetilde{M}_{z_j}$ by the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ are defined consistently as Lagrange interpolations of the resulting polynomial.

$\mathbf{H}_{18}^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_{17}^{i^*}(\lambda, b)$, except that we change

the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ such that, on input (n_g, z^*), the challenger replaces the machine used to compute the obfuscation $\widetilde{M}_{z^*}$ by

$$M_0 = \begin{cases} M_p^{\mathsf{share}}[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_f, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

instead of

$$M_1 = \begin{cases} M_p^{\mathsf{share}}[K_{f,*}, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \leq i^* \\ M_p^{\mathsf{share}}[K_{f,*}, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

$\mathbf{H}_{19}^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_{18}^{i^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ such that, on input (n_g, z^*), if party $z^* \leq i^*$ we change the machine used to generate $\widetilde{M}_{z^*}$ so that, on input $(k, t) = (i^*, t_{i^*})$ the machine behaves normally instead of outputting the explicitly hardwired value $\mathsf{sh}_{z^*}^{t_{i^*}}$. In particular we let:

$$M_0 = M_p^{\mathsf{share}}[K_f, z^*, 0, 0, 0] \quad \text{instead of} \quad M_1 = M_p^{\mathsf{share}}[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}]$$

Lemma 2. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{19}^{i^*-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_1^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. Let $\mathcal{Q} = (q_1, \ldots, q_m)$ be the subsequence of oracle queries made to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, where each query q_j has parameters (n_j, z_j) with $z_j \leq i^*$; for every $z \in [0, |\mathcal{Q}|]$, let $\boldsymbol{H}_1^{i^*,z}(\lambda, b)$ be the experiment where we change the first z answers to the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ with an input (n_g, z^*) such that $z^* < i^*$.

Clearly, $\{\mathbf{H}_1^{i^*,0}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{19}^{i^*-1}(\lambda, b))\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_1^{i^*,|\mathcal{Q}|}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_1^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the lemma, it suffices to show that, for all $z \in [|\mathcal{Q}|]$, we have $\{\mathbf{H}_1^{i^*,z-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_1^{i^*,z}(\lambda, b)\}_{\lambda\in\mathbb{N}}$. The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_1^{i^*,z-1}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_1^{i^*,z}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_{z^*} with z^* being the party corrupted in the z-th query to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$.

Let us argue that

$$M_0 = M_p^{\mathsf{share}}[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}]$$

and

$$M_1 = M_p^{\mathsf{share}}[K_f, z^*, 0, 0, 0]$$

are functionally equivalent.

Assuming that there is an input (k, t) for which M_0, M_1 produce different outputs, then $k \geq z^*$ and $t > 0$ must hold. Otherwise they both output $\perp$.

We distinguish 2 cases:

- $k = i^*$ and $t = t_{i^*}$: In this case, in M_0 we output $v = \mathsf{sh}_{z^*}^{t_{i^*}}$, while in M_1 we output S. Notice that $\mathsf{sh}_{z^*}^{t_{i^*}}$ and S are computed exactly at the same way, i.e., as the Lagrange interpolation in z^* of a degree $t_{i^*} - 1$ polynomial evaluated at the t_{i^*} distinct points $(2^\lambda - j)$ for $j \in [0, t_{i^*})$.

– $k \neq i^*$ or $t \neq t_{i^*}$: In this case both M_0 and M_1 will output S.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\mathbf{H}_1^{i^*,z-1}(\lambda, b)$ and $\mathbf{H}_1^{i^*,z}(\lambda, b)$ are indistinguishable. It follows that $\mathbf{H}_1^{i^*}(\lambda, b)$ and $\mathbf{H}_{19}^{i^*-1}(\lambda, b)$ are indistinguishable.

Lemma 3. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_1^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_2^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The proof uses the hybrid argument. In particular, let q_{max} be the maximum number of queries to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ made by the adversary. For every $z \in [0, q_{\mathsf{max}}]$, let $\mathbf{H}_2^{i^*,z}(\lambda, b)$ be the experiment where we change the first z answers to the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$.

Clearly, $\{\mathbf{H}_2^{i^*,0}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_1^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_2^{i^*,q_{\mathsf{max}}}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_2^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the lemma, it suffices to show that, for all $z \in [q_{\mathsf{max}}]$, we have $\{\mathbf{H}_2^{i^*,z-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_2^{i^*,z}(\lambda, b)\}_{\lambda\in\mathbb{N}}$. The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_2^{i^*,z-1}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_2^{i^*,z}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_{z^*} with z^* being the party corrupted in the z-th query to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$.

Let us argue that

$$M_0 = \begin{cases} M_p^{\mathsf{share}}[K_{f,*}, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \in \mathcal{U}^* \\ M_p^{\mathsf{share}}[K_{f,*}, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

and

$$M_1 = \begin{cases} M_p^{\mathsf{share}}[K_f, z^*, i^*, t_{i^*}, \mathsf{sh}_{z^*}^{t_{i^*}}] \text{ if } z^* \in \mathcal{U}^* \\ M_p^{\mathsf{share}}[K_f, z^*, 0, 0, 0] \text{ otherwise} \end{cases}$$

are functionally equivalent.

Assuming that there is an input (k, t) for which M_0, M_1 produce different outputs, then $k \geq z^*$ and $t > 0$ must hold. Otherwise they both output $\perp$.

We distinguish 2 major cases:

– $z^* > i^*$: Since we assumed $k \geq z^*$, $k > i^*$ holds, and $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ will never be called as the machine will abort;
– $z^* \leq i^*$: here we distinguish two cases:
  • $k = i^*$ and $t = t_{i^*}$: In this case, neither in M_0 or in M_1 $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ is never called, as both circuits output v;
  • $k \neq i^*$ or $t \neq t_{i^*}$: In this case both M_0 and M_1 will output S, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\mathbf{H}_2^{i^*,z-1}(\lambda, b)$ and $\mathbf{H}_2^{i^*,z}(\lambda, b)$ are indistinguishable. It follows that $\mathbf{H}_1^{i^*}(\lambda, b)$ and $\mathbf{H}_2^{i^*}(\lambda, b)$ are indistinguishable.

Lemma 4. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_2^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_3^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||2^\lambda$. We can show how to turn any distinguisher D between the two hybrids into an adversary attacking the pseudorandomness at punctured points of PPRF (Definition 2) as follows.

- The reduction receives the messages $\mu_0, \mu_1 \in \mathcal{M}$ from the $\boldsymbol{G}^{\mathsf{priv}}_{\mathsf{SS},\mathsf{D}}$ adversary D.
- The reduction sends to the PPRF security game the point $i^*||t_{i^*}||2^\lambda$ and receives $K_* = \mathsf{Punc}(K, \{i^*||t_{i^*}||2^\lambda\})$ and either a uniform value $\bar{u} \xleftarrow{\$} \mathbb{Z}_p$ or the value $\bar{u} = \mathsf{Eval}(K, i^*||t_{i^*}||2^\lambda)$, where K is the PPRF key generated by the PPRF challenger.
- At each oracle query to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, on input (n_g, i) the oracle computes the answer σ_i as follows:
  - If $i > i^*$: the reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i||t_i||(2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and sets:
    - $\mathsf{ct}_i = K_i \cdot \mu$,
    - $M_i = M_p^{\mathsf{share}}[K_*, i, 0, 0, 0]$

    Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.
  - If $i = i^*$: the reduction computes $K_{i^*} = f_{i^*}(0)$ and $\mathsf{sh}_{i^*}^{t_{i^*}} = f_{i^*}(i^*)$ as

$$K_{i^*} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_m}{x_j - x_m},$$

    and

$$\mathsf{sh}_{i^*}^{t_{i^*}} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{i^* - x_m}{x_j - x_m},$$

    with

$$y_j = \begin{cases} \bar{u} \text{ if } j = 0 \\ \mathsf{Eval}(K_*, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [1, t_{i^*}) \end{cases}$$

    and $x_j = 2^\lambda - j$ for all $j \in [0, t_{i^*})$. Then it sets:
    - $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu$,
    - $M_{i^*} = M_p^{\mathsf{share}}[K_*, i^*, i^*, t_{i^*}, \mathsf{sh}_{i^*}^{t_{i^*}}]$

    Finally $\sigma_{i^*} = (\mathsf{ct}_{i^*}, \widetilde{M_{i^*}})$.
  - If $i < i^*$: the reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i||t_i||(2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and $\mathsf{sh}_i^{t_{i^*}} = f_{i^*}(i)$ as

$$\mathsf{sh}_i^{t_{i^*}} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{i - x_m}{x_j - x_m},$$

    Then, it sets:

$*$ $\mathsf{ct}_i \xleftarrow{\$} \mathbb{Z}_p$
$*$ $M_i = M_p^{\mathsf{share}}[K_*, i, i^*, t_{i^*}, \mathsf{sh}_i^{t_{i^*}}]$

Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.

– Finally, the reduction outputs whatever D outputs.

Lemma 5. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_3^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_4^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by Lemma 1. We can show how to turn any distinguisher D between the two hybrids into an adversary attacking the security of Lemma 1 as follows.

– The reduction receives the messages $\mu_0, \mu_1 \in \mathcal{M}$ from the $\boldsymbol{G}_{\mathsf{SS},\mathsf{D}}^{\mathsf{priv}}$ adversary D.
– The reduction computes $K_* = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||2^\lambda\})$;
– The reduction sends to the security game of Lemma 1:
  • $n = i^*$;
  • $t = t_{i^*}$;
  • $\mathcal{I} = \{2^\lambda - j\}$ for $j \in [1, t_{i^*})$ with $|\mathcal{I}| = t_{i^*} - 1$;
  • $i_{t-1} = 2^\lambda$;
  • $i^* = 0$;
  • $\mathcal{Y} = \{\mathsf{Eval}(K_*, i^*||t_{i^*}||(2^\lambda - j))\}$ for $j \in [1, t_{i^*})$ with $|\mathcal{Y}| = t_{i^*} - 1$;

  Then, the Lemma 1 challenger randomly samples y_{t-1} and computes r_b either as a random value or as $f(i^*) = \sum_{j=0}^{t-1} y_j \ell_{x_j}(i^*)$, where $\ell_{x_j}(x) = \prod_{\substack{0\le m\le t-1 \\ m\neq x_j}} \frac{x - x_m}{x_j - x_m}$ (for $i \in \mathcal{I}$). Then, it forwards r_b to the reduction. Then, the reduction computes $f_{i^*}(x)$ as the unique polynomial of degree $t_{i^*} - 1$ obtained by interpolating the values $(y_j)_{j\in[0,t_{i^*})}$:

$$y_j = \begin{cases} r_b \text{ if } j = 0 \\ \mathsf{Eval}(K_*, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [1, t_{i^*}) \end{cases}$$

  at the points

$$x_j = \begin{cases} 0 \text{ if } j = 0 \\ 2^\lambda - j \text{ if } j \in [1, t_{i^*}) \end{cases}$$

– At each oracle query to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, on input (n_g, i) the oracle computes the answer σ_i as follows:
  • If $i > i^*$: the reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i||t_i||(2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and sets:
    $*$ $\mathsf{ct}_i = K_i \cdot \mu$,
    $*$ $M_i = M_p^{\mathsf{share}}[K_*, i, 0, 0, 0]$

    Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.

- If $i \leq i^*$: the reduction computes $\mathsf{sh}_i^{t_{i^*}}$ as:

$$\mathsf{sh}_i^{t_{i^*}} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{i - x_m}{x_j - x_m},$$

  Then:
  - If $i < i^*$: the reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i||t_i||(2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and sets:
    - $\mathsf{ct}_i \xleftarrow{\$} \mathbb{Z}_p$
    - $M_i = M_p^{\mathsf{share}}[K_*, i, i^*, t_{i^*}, \mathsf{sh}_i^{t_{i^*}}]$

    Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.
  - If $i = i^*$: the reduction sets:
    - $\mathsf{ct}_{i^*} = r_b \cdot \mu$
    - $M_{i^*} = M_p^{\mathsf{share}}[K_*, i^*, i^*, t_{i^*}, \mathsf{sh}_{i^*}^{t_{i^*}}]$

    Finally $\sigma_{i^*} = (\mathsf{ct}_{i^*}, \widetilde{M_{i^*}})$.
- Finally the reduction outputs whatever D outputs.

Lemma 6. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_4^{i^*}(\lambda, b)\}_{\lambda \in \mathbb{N}} \approx_c \{\mathbf{H}_5^{i^*}(\lambda, b)\}_{\lambda \in \mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 3, with M_0 and M_1 inverted.

Lemma 7. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [1, t_{i^*})$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_9^{i^*, j^*-1}(\lambda, b)\}_{\lambda \in \mathbb{N}} \approx_c \{\mathbf{H}_6^{i^*, j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 3.

Lemma 8. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [1, t_{i^*})$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_6^{i^*, j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}} \approx_c \{\mathbf{H}_7^{i^*, j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||(2^\lambda - j^*)$. We can show how to turn any distinguisher D between the two hybrids into an adversary attacking the pseudorandomness at punctured points of PPRF (Definition 2) as follows.

- The reduction receives the messages $\mu_0, \mu_1 \in \mathcal{M}$ from the $\boldsymbol{G}_{\mathsf{SS},\mathsf{D}}^{\mathsf{priv}}$ adversary D.
- The reduction sends to the PPRF security game the point $i^*||t_{i^*}||(2^\lambda - j^*)$ and receives $K_* = \mathsf{Punc}(K, \{i^*||t_{i^*}||(2^\lambda - j^*)\})$ and either a uniform value $\bar{u} \xleftarrow{\$} \mathbb{Z}_p$ or the value $\bar{u} = \mathsf{Eval}(K, i^*||t_{i^*}||2^\lambda - j^*)$, where K is the PPRF key generated by the PPRF challenger.
- At each oracle query to $\mathcal{O}_{\mathsf{share}}(\cdot, \cdot)$, on input (n_g, i) the oracle computes the answer σ_i as follows:

- If $i > i^*$: the reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i||t_i||(2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and sets:
  * $\mathsf{ct}_i = K_i \cdot \mu$,
  * $M_i = M_p^{\mathsf{share}}[K_*, i, 0, 0, 0]$

  Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.
- Otherwise, the reduction samples $K_{i^*} \xleftarrow{\$} \mathbb{Z}_p$ and j^*-1 values $(\mathsf{sh}_{i_j}^{t_{i^*}})_{j \in [1, j^*)}$ randomly, then for all the queries in which $i \leq i^*$:
  * For the queries $q \in [1, j^*)$:
    · If $i < i^*$: The reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i||t_i||(2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and sets: $\mathsf{ct}_i \xleftarrow{\$} \mathbb{Z}_p$, $M_i = M_p^{\mathsf{share}}[K_*, i, i^*, t_{i^*}, \mathsf{sh}_{i_q}^{t_{i^*}}]$
    · If $i = i^*$: the reduction sets: $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu$, $M_{i^*} = M_p^{\mathsf{share}}[K_*, i^*, i^*, t_{i^*}, \mathsf{sh}_{i_q}^{t_{i^*}}]$
    · Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.
  * Otherwise, for the queries $q \in [j^*, q_{\mathsf{max}}]$: the reduction computes $f_{i^*}(x)$ as the unique polynomial of degree $t_{i^*} - 1$ obtained by interpolating the values $(y_j)_{j \in [0, t_{i^*})}$:

$$y_j = \begin{cases} K_{i^*} \text{ if } j = 0 \\ \mathsf{sh}_{i_j}^{t_{i^*}} \text{ if } 0 < j < j^* \\ \bar{u} \text{ if } j = j^* \\ \mathsf{Eval}(K_*, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

  at the points

$$x_j = \begin{cases} 0 \text{ if } j = 0 \\ i_j \text{ if } 0 < j < j^* \\ 2^\lambda - j \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

  where $(\mathsf{sh}_{i_j}^{t_{i^*}})_{j \in [1, j^*)}$ and $(i_j)_{j \in [1, j^*)}$ are the randomly sampled shares and the indices of the corrupted parties during the queries $q \in [1, j^*)$ respectively. The reduction also computes

$$\mathsf{sh}_i^{t_{i^*}} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{i - x_m}{x_j - x_m},$$

  Then:
    · If $i < i^*$: the reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i||t_i||(2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and sets: $\mathsf{ct}_i \xleftarrow{\$} \mathbb{Z}_p$, $M_i = M_p^{\mathsf{share}}[K_*, i, i^*, t_{i^*}, \mathsf{sh}_i^{t_{i^*}}]$

· If $i = i^*$: the reduction sets: $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu$, $M_{i^*} = M_p^{\mathsf{share}}[K_*, i^*, i^*, t_{i^*}, \mathsf{sh}_{i^*}^{t_{i^*}}]$.
· Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.

– Finally, the reduction outputs whatever D outputs.

Lemma 9. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [1, t_{i^*})$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_7^{i^*,j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}} \equiv \{\mathbf{H}_8^{i^*,j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}}$.*

Proof. The lemma follows by Lemma 1. We can show how to turn any distinguisher D between the two hybrids into an adversary attacking the security of Lemma 1 as follows.

– The reduction receives the messages $\mu_0, \mu_1 \in \mathcal{M}$ from the $\boldsymbol{G}_{\mathsf{SS},\mathsf{D}}^{\mathsf{priv}}$ adversary D.
– The reduction computes $K_* = \mathsf{Punc}(K_f, \{i^* || t_{i^*} || (2^\lambda - j^*)\})$;
– At each oracle query to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, on input (n_g, i) the oracle computes the answer σ_i as follows:
  • If $i > i^*$: the reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i || t_i || (2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and sets:
    ∗ $\mathsf{ct}_i = K_i \cdot \mu$,
    ∗ $M_i = M_p^{\mathsf{share}}[K_*, i, 0, 0, 0]$
    Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.
  • Otherwise, the reduction samples $K_{i^*} \xleftarrow{\$} \mathbb{Z}_p$ and j^*-1 values $(\mathsf{sh}_{i_j}^{t_{i^*}})_{j \in [1,j^*)}$ randomly, then for all the queries in which $i \leq i^*$:
    ∗ For the queries $q \in [1, j^*)$:
      · If $i < i^*$: the reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i || t_i || (2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and sets: $\mathsf{ct}_i \xleftarrow{\$} \mathbb{Z}_p$, $M_i = M_p^{\mathsf{share}}[K_*, i, i^*, t_{i^*}, \mathsf{sh}_{i_q}^{t_{i^*}}]$
      · If $i = i^*$: the reduction sets: $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu$, $M_{i^*} = M_p^{\mathsf{share}}[K_*, i^*, i^*, t_{i^*}, \mathsf{sh}_{i_q}^{t_{i^*}}]$
      · Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.
    ∗ At the query $q = j^*$: in order to simulate $\mathsf{sh}_i^{t_{i^*}}$ the reduction sends to the security game of Lemma 1:
      · $n = i^*$;
      · $t = t_{i^*}$;
      ·
      $$\mathcal{I} = \{x_j \mid x_j = \begin{cases} 0 \text{ if } j = 0 \\ i_j \text{ if } 0 < j < j^* \\ 2^\lambda - j \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases} \}$$
      for $j \in [0, t_{i^*})$ with $|\mathcal{I}| = t_{i^*} - 1$, where $(i_j)_{j \in [1,j^*)}$ are the parties corrupted during the queries $q \in [1, j^*)$;
      · $i_{t-1} = 2^\lambda - j^*$;

· $i^* = i$;
·

$$\mathcal{Y} = \{y_j \mid y_j = \begin{cases} K_{i^*} \text{ if } j = 0 \\ \mathsf{sh}_{i_j}^{t_{i^*}} \text{ if } 0 < j < j^* \\ \mathsf{Eval}(K_*, i^* || t_{i^*} || (2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases} \}$$

where $(\mathsf{sh}_{i_j}^{t_{i^*}})_{j \in [1, j^*)}$ are the randomly sampled shares during the queries $q \in [1, j^*)$, with $|\mathcal{Y}| = t_{i^*} - 1$;

Then, the Lemma 1 challenger randomly samples y_{t-1} and computes r_b either as a random value or as $f(i^*) = \sum_{j=0}^{t-1} y_j \ell_{x_j}(i^*)$, where $\ell_{x_j}(x) = \prod_{\substack{0 \leq m \leq t-1 \\ m \neq x_j}} \frac{x - x_m}{x_j - x_m}$ (for $i \in \mathcal{I}$). Then, it forwards r_b to the reduction. Then:

· If $i < i^*$: the reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i || t_i || (2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and sets: $\mathsf{ct}_i \xleftarrow{\$} \mathbb{Z}_p$, $M_i = M_p^{\mathsf{share}}[K_*, i, i^*, t_{i^*}, r_b]$
· If $i = i^*$: the reduction sets $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu$, $M_{i^*} = M_p^{\mathsf{share}}[K_*, i^*, i^*, t_{i^*}, r_b]$.
· Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.

∗ For the queries $q \in [j^* + 1, q_{\mathsf{max}}]$: the reduction computes

$$\mathsf{sh}_i^{t_{i^*}} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{i - x_m}{x_j - x_m},$$

with

$$y_j = \begin{cases} K_{i^*} \text{ if } j = 0 \\ \mathsf{sh}_{i_j}^{t_{i^*}} \text{ if } 0 < j < j^* \\ r_b \text{ if } j = j^* \\ \mathsf{Eval}(K_*, i^* || t_{i^*} || (2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

and

$$x_j = \begin{cases} 0 \text{ if } j = 0 \\ i_j \text{ if } 0 < j < j^* \\ i \text{ if } j = j^* \\ 2^\lambda - j \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

where $(\mathsf{sh}_{i_j}^{t_{i^*}})_{j < j^*}$ and $(i_j)_{j < j^*}$ are the randomly sampled shares and the parties corrupted during the queries $q \in [1, j^*)$. Then

· If $i < i^*$: the reduction computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_*, i || t_i || (2^\lambda - j))$ at the points $(2^\lambda - j)$ for all $j \in [0, t_i)$ and sets: $\mathsf{ct}_i \xleftarrow{\$} \mathbb{Z}_p$, $M_i = M_p^{\mathsf{share}}[K_*, i, i^*, t_{i^*}, \mathsf{sh}_i^{t_{i^*}}]$

· If $i = i^*$: the reduction sets $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu$, $M_{i^*} = M_p^{\mathsf{share}}[K_*, i^*, i^*, t_{i^*}, \mathsf{sh}_{i^*}^{t_{i^*}}]$.
· Finally $\sigma_i = (\mathsf{ct}_i, \widetilde{M_i})$.

– Finally the reduction outputs whatever D outputs.

Lemma 10. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [1, t_{i^*})$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_8^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_9^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 7, with M_0 and M_1 inverted.

Lemma 11. *For every $i^* \in [\mathsf{poly}(\lambda)]$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_9^{i^*,t_{i^*}-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{10}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by observing that the two hybrids differ only in the answer of $\mathcal{O}_{\mathsf{share}}$ on input (n_g, i^*); in the former hybrid $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu_b$ is a one-time-pad encryption under a uniform key K_{i^*}, while in the latter ct_{i^*} is sampled uniformly at random, which is identically distributed to a one-time-pad encryption.

Lemma 12. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in (t_{i^*}, 1]$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{14}^{i^*,j^*-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{11}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 10, with M_0 and M_1 inverted.

Lemma 13. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in (t_{i^*}, 1]$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{11}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Lemma 1. The reduction for showing $\{\mathbf{H}_{11}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ is analogous to the one used in Lemma 9. We omit it for brevity.

Lemma 14. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in (t_{i^*}, 1]$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{13}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||(2^\lambda - j^*)$. The reduction for showing $\{\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{13}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ is analogous to the one used in Lemma 8. We omit it for brevity.

Lemma 15. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in (t_{i^*}, 1]$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{13}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{14}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 7, with M_0 and M_1 inverted.

Lemma 16. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{14}^{i^*,1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{15}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 6, with M_0 and M_1 inverted.

Lemma 17. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{15}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{16}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Lemma 1. The reduction for showing $\{\mathbf{H}_{15}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{16}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ is analogous to the one used in Lemma 5. We omit it for brevity.

Lemma 18. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{16}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{17}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||2^\lambda$. The reduction for showing $\{\mathbf{H}_{16}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{17}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ is analogous to the one used in Lemma 4. We omit it for brevity.

Lemma 19. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{17}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{18}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 3, with M_0 and M_1 inverted.

Lemma 20. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{18}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{19}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 2, with M_0 and M_1 inverted.

Lemma 21. *$\{\mathbf{H}_{19}^{n}(\lambda, 0)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{19}^{n}(\lambda, 1)\}_{\lambda\in\mathbb{N}}$, where $n \in \mathsf{poly}(\lambda)$ is the number of parties chosen by the adversary.*

Proof. The lemma follows by simply observing that the distribution of the shares $(\sigma_u)_{u\in\mathcal{U}}$ where $\mathcal{U} = \bigcup_{q\in[q_{\mathsf{max}}]} \mathcal{U}_q$ is now independent of the message.

A.4 Proof of Theorem 7

Proof. Let $\lambda \in \mathbb{N}$ and $\mathcal{I} = \{i_1, i_2, \ldots, n\} \in \hat{\mathcal{A}}^{\mathsf{fw-ethr}}$, i.e., $W_{\mathcal{I}} = \sum_{i\in\mathcal{I}} w_i \geq t_n$ and $\mathsf{max}(\mathcal{I}) = n$. Let us show $(K'_n)^{-1} \cdot K_n \cdot \mu = \mu$. We have that $K_n = \mathsf{Lagrange}(x_{n,0}, \{\mathsf{Eval}(K_f, n||t_n||(2^\lambda - j))\}_{j\in[0,t_n)}, \{\mathsf{Eval}(K_p, n||t_n||(2^\lambda - j))\}_{j\in[0,t_n)})$, with $x_{n,0} = \mathsf{Eval}(K_p, n||t_n||0)$. Each party $i \in \mathcal{I}$ holds $\sigma_i = (\mathsf{ct}_i, \widetilde{M}_i, x_{i,0})$, where $\mathsf{ct}_i = K_i \cdot \mu$, with $K_i = f_i(x_{i,0})$, $\widetilde{M}_i \xleftarrow{\$} \mathsf{Obf}(1^\lambda, 1^{\ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}}}, t^{\mathsf{max}}_{M^{\mathsf{fw-share}}}, M_i)$ with M_i equal to $M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,i}, W_{i-1}, 0, w_i, i, 0, 0, 0, 0, 0, \bot, 0, 0, 0](\cdot,\cdot,\cdot)$ for $W_i = \sum_{j\in[i]} w_j$ and $W_0 = 0$, and $x_{i,0} = \mathsf{Eval}(K_p, i||t_i||0)$. During reconstruction, for each party $i \in \mathcal{I}$ and for each $q_i \in [w_i]$ we run the obfuscated program $(x_{i,q_i}, \mathsf{sh}_{i,q_i}) = \widetilde{M}_i(n, t_n, q_i)$. By correctness of Obf, this outputs the values generated by

$$M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,i}, W, W^*, w_i, i, \mathsf{pos}, t^*, x_1, v_1, x_2, v_2, l^*, q_1^*, q_2^*](k, t, q)$$

which runs the code in Figure 2 and has hardcoded values: $K_f \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, $K_p \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, $K'_p \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, $K_{p,i} \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$, $W = W_{i-1}, W^* = 0$, $w_i = w_i$, $i = i$,$\mathsf{pos} = t^* = x_1 = x_2 = 0$, $v_2 = \perp$, $l^* = q_1^* = q_2^* = 0$. Clearly $n \geq i$, $t_n > 0$ and $0 < q \leq w_i$. Since $n > 0$ and $t_n > 0$, the circuit jumps to computing for every $j \in [0, t_n)$ the evaluations $\mathsf{eval}[j] = \mathsf{Eval}(K_f, n||t_n||(2^\lambda - j))$ and the points $\mathsf{points}[j] = \mathsf{Eval}(K_p, n||t_n||(2^\lambda - j))$. Finally the circuit computes $\mathsf{Lagrange}(x_{i,q_i}, \{\mathsf{eval}[j]\}_{j\in[0,t_n)}, \{\mathsf{points}[j]\}_{j\in[0,t_n)})$, where $x_{i,q_i} = \mathsf{Eval}(K_{p,i}, n||t_n||(W_{i-1} + q_i))$. Hence, for every party $i \in \mathcal{I}$ and for every $q_i \in [w_i]$, the parties obtain $\mathsf{sh}_{i,q_i} = f_n(x_{i,q_i})$, and since $\mathcal{I} \in \hat{\mathcal{A}}^{\mathsf{fw-ethr}}$, we are guaranteed that parties collectively have at least t_n shares $(\mathsf{sh}_{i,q_i})_{i\in\mathcal{I},q_i\in[w_i]}$. Thus, parties can compute $K'_n = \mathsf{Lagrange}(x_{n,0}, (\mathsf{sh}_{i,q_i})_{i\in\mathcal{I},q_i\in[w_i]}, (x_{i,q_i})_{i\in\mathcal{I},q_i\in[w_i]})$, which is exactly $f_n(x_{n,0})$ whenever the interpolation denominators $\{\mathsf{points}[j]\}_{j\in[0,t_n)}$ (used during the computation of the parties' shares) and $(x_{i,q_i})_{i\in\mathcal{I},q_i\in[w_i]}$ (used during the reconstruction of K'_n) are all nonzero,[13] i.e., no collision generated by the PPRF evaluation indexed by keys $\{K_p\} \cup \{K_{p,i}\}_{i\in\mathcal{I}}$. Once, $f_n(x_{n,0})$ is computed, it is easy to see that $K'_n = f_n(x_{n,0}) = f_n(\mathsf{Eval}(K_p, n||t_n||0)) = K_n$ and $(K'_n)^{-1} \cdot K_n \cdot \mu = \mu$. Hence, to concludes the proof it is sufficient to show that the PPRFs (indexed by $\{K_p\} \cup \{K_{p,i}\}_{i\in\mathcal{I}}$) do not generate collisions on distinct inputs, except with negligible probability. Let us consider the set of PPRF inputs that appear in the reconstruction:

- the t_n interpolation base points: $\alpha_j := \{n||t_n||(2^\lambda - j)\}$ for $j \in [0, t_n)$. Note that these points are fed to the PPRF indexed by K_p;
- the share x-coordinates of the parties in the authorized subset $\mathcal{I}$: $\beta_{i,q_i} := \{n||t_n||(W_{i-1} + q_i)\}$ for $i \in \mathcal{I}, q_i \in [w_i]$. Note that each β_{i,q_i} is fed to the PPRF indexed by $K_{p,i}$.

We note that the PPRF inputs $n||t_n||0$ will never collide with neither α_j for any $j \in [0, t_n)$ (since by assumption $t_n = \mathsf{poly}(\lambda)$) or with β_{i,q_i} since $W_{i-1} \geq 0$ and $q_i > 0$ for every $i \in \mathcal{I}$, $q_i \in [w_i]$.

Let $\mathcal{Q} := \{\alpha_j : j \in [0, t_n)\} \cup \{\beta_{i,q_i} : i \in \mathcal{I}, q_i \in [w_i]\}$. Notice $|\mathcal{Q}| = t_n + \sum_{i\in\mathcal{I}} w_i \leq t_n + W = \mathsf{poly}(\lambda)$. For any pair of functions $G_K : \{0,1\}^* \to \mathbb{Z}_p$, $G_{K^*} : \{0,1\}^* \to \mathbb{Z}_p$, let us define the event:

$$\mathsf{Bad}(G_K, G_{K^*}) := [\exists u, v \in \mathcal{Q} \mid u \neq v \wedge G_K(u) = G_{K^*}(v)].^{14}$$

We observe that:

- In the real world, $G_K(\cdot) = \mathsf{Eval}(K, \cdot)$ for $K \in \{K_p\} \cup \{K_{p,i}\}_{i\in\mathcal{I}}$;

[13] It suffices that the interpolation denominators be pairwise distinct within each respective set, i.e., no collisions occur among the elements of $\{\mathsf{points}[j]\}_{j\in[0,t_n)}$ and none among the elements of $(x_{i,q_i})_{i\in\mathcal{I},\, q_i\in[w_i]}$. Collisions between the two sets do not affect correctness, as they are never used jointly in a single interpolation. However, for simplicity, we assume the stronger condition that the two sets are mutually disjoint, which simplifies the proof.

[14] Note that we do not define a collision event when $K \neq K^*$ but $u = v$, since this case is not relevant to our setting. Indeed, our construction never evaluates two different PPRFs on the same input $u \in \mathcal{Q}$.

– In the ideal world, $G_K(\cdot) = R(\cdot)$ for an uniform random function and every $K \in \{K_p\} \cup \{K_{p,i}\}_{i \in \mathcal{I}}$.

In order to prove that $\Pr[\mathsf{Bad}(\mathsf{Eval}(K,\cdot), \mathsf{Eval}(K^*,\cdot))]$ is negligible for every $(K, K^*) \in (\{K_p\} \cup \{K_{p,i}\}_{i \in \mathcal{I}})^2$ (i.e., the PPRF indexed by K have collision with the PPRF indexed by K^* with negligible probability),[15] we can build a PPRF PPT distinguisher $\mathsf{D}^{\neq}$ for the case $K \neq K^*$, and a distinguisher $\mathsf{D}^{=}$ for the case $K = K^*$.

If $K \neq K^*$, the distinguisher $\mathsf{D}^{\neq}$ proceeds as follows:

1. Compute $\mathcal{Q}$ from $(\mathcal{I}, \hat{\mathcal{A}}^{\mathsf{fw-ethr}}, t_n)$;
2. Sample $K^* \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$;
3. For every $u \in \mathcal{Q}$, query the PPRF oracle $\mathcal{O}(\cdot)$ and store $z_u = \mathcal{O}(u)$;
4. For every $u \in \mathcal{Q}$, compute $z^*_u = \mathsf{Eval}(K^*, u)$ and store z^*_u;
5. If there exists $u, v \in \mathcal{Q}$ with $z_u = z^*_v$ and $u \neq v$, it outputs 1, otherwise it outputs 0.

That is, $\mathsf{D}^{\neq}$ outputs 1 iff $\mathsf{Bad}(\mathcal{O}(\cdot), \mathsf{Eval}(K^*,\cdot))$ holds. Let Real denote the experiment where $\mathcal{O}(\cdot) = \mathsf{Eval}(K,\cdot)$ and Rand denote the experiment where $\mathcal{O}(\cdot) = R(\cdot)$. Then, by construction

$$\Pr[\mathsf{D}^{\neq,\mathsf{Real}}(1^\lambda) = 1] = \Pr[\mathsf{Bad}(\mathsf{Eval}(K,\cdot), \mathsf{Eval}(K^*,\cdot))],$$

and

$$\Pr[\mathsf{D}^{\neq,\mathsf{Rand}}(1^\lambda) = 1] = \Pr[\mathsf{Bad}(R(\cdot), \mathsf{Eval}(K^*,\cdot))].$$

Therefore $\mathsf{D}^{\neq}$'s PPRF distinguishing advantage is exactly:

$$\mathsf{Adv}_{\mathsf{D}^{\neq},\mathsf{PPRF}} := |\Pr[\mathsf{Bad}(\mathsf{Eval}(K,\cdot), \mathsf{Eval}(K^*,\cdot))] - \Pr[\mathsf{Bad}(R(\cdot), \mathsf{Eval}(K^*,\cdot))]|.$$

By applying the union bound, using the fact that $R(\cdot)$ outputs random elements from $\mathbb{F}_p$ with prime p of size $\lambda + 1$, and that $|\mathcal{Q}| \in \mathsf{poly}(\lambda)$, we conclude that $\Pr[\mathsf{Bad}(R(\cdot), \mathsf{Eval}(K^*,\cdot))]$ is negligible. Hence, if we assume towards contradiction that $\Pr[\mathsf{Bad}(\mathsf{Eval}(K,\cdot), \mathsf{Eval}(K^*,\cdot))]$ is not negligible, we would have that $\mathsf{D}^{\neq}$ is a non-negligible PPRF distinguisher against $\mathsf{Eval}(K,\cdot)$.

On the other hand, if $K = K^*$, the distinguisher $\mathsf{D}^{=}$ proceeds as follows:

1. Compute $\mathcal{Q}$ from $(\mathcal{I}, \hat{\mathcal{A}}^{\mathsf{fw-ethr}}, t_n)$.
2. For every $u \in \mathcal{Q}$, query the PPRF oracle $\mathcal{O}(\cdot)$ and store $z_u = \mathcal{O}(u)$;
3. If there exists $u, v \in \mathcal{Q}$ with $z_u = z_v$ and $u \neq v$, it outputs 1, otherwise it outputs 0.

Using a similar argument to that used for the case $K \neq K^*$, we conclude that:

$$\mathsf{Adv}_{\mathsf{D}^{=},\mathsf{PPRF}} := |\Pr[\mathsf{Bad}(\mathsf{Eval}(K,\cdot), \mathsf{Eval}(K,\cdot))] - \Pr[\mathsf{Bad}(R(\cdot), R(\cdot))]| \leq \mathsf{negl}(\lambda).$$

[15] Observe that the pairs of PPRF keys $(K, K^*) \in (\{K_p\} \cup \{K_{p,i}\}_{i \in \mathcal{I}})^2$ are polynomial in the security parameters since $|\mathcal{I}| \leq t_n \leq n \in \mathsf{poly}(\lambda)$.

A.5 Proof of Theorem 8

Proof. We need to prove that the size of each share $\sigma_i = (\mathsf{ct}_i, \widetilde{M}_i, x_{i,0})$ is $\mathsf{poly}(\lambda, \log W_i)$.

First, we have that $\mathsf{ct}_i = K_i \cdot \mu$ and $x_{i,0} \in \mathbb{F}_p$. Hence, $|\mathsf{ct}_i| = |x_{i,0}| = \log(p) = \log \lambda + 1 = O(\lambda)$. To conclude the proof, it remains to show that $|\widetilde{M}_i|$ is in $\mathsf{poly}(\lambda, \log W_i)$.

Recall that $\widetilde{M}_i$ is computed as $\widetilde{M}_i \xleftarrow{\$} \mathsf{Obf}(1^\lambda, 1^{\ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}}}, t^{\mathsf{max}}_{M^{\mathsf{fw-share}}}, M_i)$ with

$$M_i = M_p^{\mathsf{fw-share}}[K_f, K_p, K'_p, K_{p,i}, W_{i-1}, 0, w_i, i, 0, 0, 0, 0, 0, \bot, 0, 0, 0].$$

By the succinctness and efficiency requirements on Obf, $|\widetilde{M}_i|$ is bounded by $\mathsf{poly}(|M_p^{\mathsf{fw-share}}|, \log t^{\mathsf{max}}_{M^{\mathsf{fw-share}}}, \ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}}, \lambda)$, where:

- $|M_p^{\mathsf{fw-share}}|$ is the description size of the Turing Machine;
- $t^{\mathsf{max}}_{M^{\mathsf{fw-share}}}$ is the maximum runtime of the Turing Machine;
- $\ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}}$ is the maximum input size of the Turing Machine.

According to Figure 2, we have $\ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}} = 2{\cdot}\mathsf{suplog}(\lambda){+}|w|$ where $O(\log \lambda) = |w|$. Below, we measure the remaining two parameters.

Regarding the maximum runtime $t^{\mathsf{max}}_{M^{\mathsf{fw-share}}}$:

- Comparisons with $i, 0, w_i, \mathsf{pos}, t^*, q_1^*, q_2^*$ can be executed in time $\mathsf{poly}(\mathsf{suplog}(\lambda), |w|) = \mathsf{poly}(\mathsf{suplog}(\lambda))$;
- $\mathsf{Eval}(K_f, \cdot)$,$\mathsf{Eval}(K_p, \cdot)$,$\mathsf{Eval}(K'_p, \cdot)$, and $\mathsf{Eval}(K_{p,i}, \cdot)$ can all be executed in $(2 \cdot \mathsf{suplog}(\lambda) + \lambda) \cdot \mathsf{poly}(\lambda) = \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$.
- Since $l^* \leq W$ and $|W| \leq \mathsf{suplog}(\lambda)$ (recall the maximum weight size is $|w| = O(\log \lambda)$), the first while-loop (line 10) has runtime at most $2^{\mathsf{suplog}(\lambda)} \cdot \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$;
- Since $t \leq t_n \leq W$ (recall the maximum weight size is $|w| = O(\log \lambda)$), the second while-loop (line 18) has runtime $2^{\mathsf{suplog}(\lambda)} \cdot \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$;
- Since $t \leq t_n \leq W$ (recall the maximum weight size is $|w| = O(\log \lambda)$), the last while-loop (line 23) has runtime in $(2^{2 \cdot \mathsf{suplog}(\lambda)}) \cdot \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$;
- Modular arithmetic in $\mathbb{F}_p$ can be executed in time $\mathsf{poly}(\log p) = \mathsf{poly}(\lambda)$ since p is a prime of $\lambda + 1$ bits.

Hence, the maximum runtime is bounded by

$$t^{\mathsf{max}}_{M^{\mathsf{fw-share}}} = O\left(2^{\mathsf{suplog}(\lambda)}\right) \cdot \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)).$$

Regarding the description size $|M_p^{\mathsf{fw-share}}|$, we start by observing that, as described in Construction 5.1, the sizes of the hardwired values (which are part of the description of the Turing Machine) are the following:

- $|K_f| = |K_p| = |K'_p| = |K_{p,i}| = |\mathsf{KeyGen}(1^\lambda)| = \mathsf{poly}(\lambda)$;
- $|W| = |W^*| = |t^*| = |i| = |l^*| = |q_1^*| = |q_2^*| = \mathsf{suplog}(\lambda)$;
- $|w_i| = |w| = O(\log \lambda)$;
- $|x_1| = |v_1| = |x_2| = |v_2| = \log p = \log(\lambda + 1)$ since those values belong to $\mathbb{F}_p$.

Then, the computation performed by the Turing Machine can be described using the following size:

- Comparisons with $i, 0, w_i, \mathsf{pos}, t^*, q_1^*, q_2^*$ can be described in size $\mathsf{poly}(\mathsf{suplog}(\lambda))$.
- $\mathsf{Eval}(K_f, \cdot)$,$\mathsf{Eval}(K_p, \cdot)$,$\mathsf{Eval}(K'_p, \cdot)$, and $\mathsf{Eval}(K_{p,i}, \cdot)$ can all be described in $(2 \cdot \mathsf{suplog}(\lambda) + \lambda) \cdot \mathsf{poly}(\lambda) = \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$;
- Since $l^* \leq W$, the first while-loop (line 10) has description size $\mathsf{suplog}(\lambda) + \mathsf{poly}(\mathsf{suplog}(\lambda), \lambda)$;
- Since $t \leq t_n \leq W$, the second while-loop (line 18) has description size $\mathsf{suplog}(\lambda) + \mathsf{poly}(\mathsf{suplog}(\lambda), \lambda)$;
- Since $t \leq t_n \leq W$, the while-loop (line 23) can be described in size $2 \cdot \mathsf{suplog}(\lambda) + \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$;
- Modular arithmetic in $\mathbb{F}_p$ can be described in size $\mathsf{poly}(\log p) = \mathsf{poly}(\lambda)$.

We conclude that the description size is bounded by

$$|M_p^{\mathsf{fw-share}}| = \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)).$$

By combining the bounds of $|M_p^{\mathsf{fw-share}}|$, $t^{\mathsf{max}}_{M^{\mathsf{fw-share}}}$, and $\ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}}$, we conclude:

$$\begin{aligned}
|\widetilde{M_i}| &= \mathsf{poly}(|M_p^{\mathsf{fw-share}}|, \log t^{\mathsf{max}}_{M^{\mathsf{fw-share}}}, \ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}}, \lambda) \\
&= \mathsf{poly}(\mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)), \log(O(2^{\mathsf{suplog}(\lambda)}) \cdot \\
&\qquad \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))), \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)), \lambda) \\
&= \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)).
\end{aligned}$$

Finally, since $i \in [n]$ for an apriori unknown $n \in \mathsf{poly}(\lambda)$ (thus $\log i = O(\log \lambda)$), each weight is of size $|w| = O(\log \lambda)$ (thus $|W_i| = |\sum_{j \in [i]} w_j| \leq |i \cdot w| \in \mathsf{suplog}(\lambda)$), and $\mathsf{suplog}(\lambda) = \log^c(\lambda)$ for constant $c > 1$, and all $\mathsf{suplog}(\lambda)$ values are chosen so as to allow the Turing machine to perform its computation correctly for every pair (k, t, q_i) whose values can depend on n (i.e., worst-case size is $O(\log n)$), for an apriori unknown value $n \in \mathsf{poly}(\lambda)$, we can rewrite the equation above as follows:

$$|\widetilde{M_i}| = \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda)) = \mathsf{poly}(\lambda, \log W_i) \text{ where } i \in [n].$$

This concludes the proof.

A.6 Proof of Theorem 9

Restatement of Theorem 9. *If the iO obfuscator* Obf *is secure (Definition 9) and* PPRF *is secure (Definition 2), then the scheme ESS scheme of Construction 5.1, for the evolving threshold access structure with fixed weights* $\hat{\mathcal{A}}^{\mathsf{fw-ethr}}$ *with weights of size at most* $|w| = O(\log \lambda)$*, is computationally private in the adaptive setting (Definition 15).*

Proof. Let $\mu_0, \mu_1 \in \mathcal{M}$ be the two messages chosen by the adversary A in the game defining the adaptive computational privacy of SS, $\boldsymbol{G}^{\mathsf{priv}}_{\mathsf{SS},\mathsf{A}}(\lambda, b)$. W.l.o.g. we assume that on each adversary's oracle query to $\mathcal{O}_{\mathsf{share}}(n_g, \mathcal{U})$, $|\mathcal{U}| \leq 1$. Given a party i^*, we denote as $\mathcal{Q} = (q_1, \ldots, q_m)$ the subsequence of oracle queries made to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, where each query q_j has parameters (n_j, z_j) with $z_j \leq i^*$. Moreover, we let $W^*_{z_j} = \sum_{k=1}^{j} w_{z_k}$ be the sum of the weights of the parties requested in $\mathcal{Q}$, namely only those where $z_j \leq i^*$, up to (and including) the query in which party z_j is requested (with $W^*_0 = 0$).

Consider the following hybrid experiments.

$\mathbf{H}_0(\lambda, b)$: This experiment is identical to $\boldsymbol{G}^{\mathsf{priv}}_{\mathsf{SS},\mathsf{A}}(\lambda, b)$.

$\mathbf{H}^{i^*}_1(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}^{i^*-1}_{29}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows. We change the machine used to generate $\widetilde{M}_{z_j}$ so that, on input $(k, t, q) = (i^*, t_{i^*}, \cdot)$, $\widetilde{M}_{z_j}$ enters the if at line 9 in Figure 2. Moreover we set $W^* = W^*_{z_{j-1}}$. In particular, the machine $M^{\mathsf{fw-share}}_p$ we use to compute $\widetilde{M}_{z_j}$ is replaced by M_0 instead of M_1 where:

$q_j \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q^*_1	q^*_2
$j \in [m]$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_j}	W_{z_j-1}	$W^*_{z_{j-1}}$	w_{z_j}	z_j	i^*	t_{i^*}	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_j}	W_{z_j-1}	0	w_{z_j}	z_j	0	0	0	0	0	$\perp$	0	0	0

Table 1. $M^{\mathsf{fw-share}}_p$ for all $z_j \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

We will now run a series of hybrids: $\mathbf{H}^{i^*,j^*}_2(\lambda, b), \ldots, \mathbf{H}^{i^*,j^*}_{13}(\lambda, b)$ for $i^* \in [\mathsf{poly}(\lambda)]$, and $j^* \in [0, t_{i^*})$:

- $\mathbf{H}^{i^*,j^*}_2(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t_{i^*})$: This is identical to $\mathbf{H}^{i^*,j^*-1}_{13}(\lambda, b)$ (with $\mathbf{H}^{i^*,-1}_{13}(\lambda, b) = \mathbf{H}^{i^*}_1(\lambda, b)$), except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows. When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W^*_{z_{j-1}}$ and we change the machine $M^{\mathsf{fw-share}}_p$ used to compute $\widetilde{M}_{z_j}$ so that $\widetilde{M}_{z_j}$ enters the if at line 5 in Figure 2, namely, on input $(k, t, q) = (i^*, t_{i^*}, w^*)$, it outputs the explicitly hardwired value $\mathsf{sh}^{t_{i^*}}_{z_j, w^*}$, and otherwise behaves as before. Hence, we replace the machine $M^{\mathsf{fw-share}}_p$ used to compute $\widetilde{M}_{z_j}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{C}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t < j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
$t = j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	0	$\perp$	j^*	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	w^*-1
$t > j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	0

Table 2. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

With:

- $x_1 = \mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1} + w^*))$,
- $v_1 = \mathsf{sh}^{t_{i^*}}_{z_j,w^*}$ where:

$$\mathsf{sh}^{t_{i^*}}_{z_j,w^*} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{\mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1}+w^*)) - x_m}{x_j - x_m}$$

with

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

and

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

for all $j \in [0, t_{i^*})$,

- $q_1^* = w^*$.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_3^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t_{i^*})$: This is identical to $\mathbf{H}_2^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to *all the queries* as follows.

- For all the queries q_t we hardcode into the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$:
  - $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||(2^\lambda - j^*)\})$,
  - $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||(2^\lambda - j^*)\})$,
- Only for the queries in $\mathcal{Q}$, when we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W^*_{z_{j-1}}$ and we hardcode into the machines $M_p^{\mathsf{fw-share}}$ used to compute $(\widetilde{M}_{z_t})_{t\geq j, q_t\in\mathcal{Q}}$:
  - $x_2 = \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$,
  - $v_2 = \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$,

so that $\widetilde{M}_{z_t}$ enters the if at line 14 in Figure 2, namely, on input $(k, t, q) = (i^*, t_{i^*}, \cdot)$, the j^*-th evaluation of the polynomial is hardcoded into the machine rather than computed within the while loop at line 18.

To recap, for each query $q_t = (n_t, z_t)$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t < j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
$t = j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	0	$\perp$	j^*	w^*	w^*-1
$t > j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	0

Table 3. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the machine $M_p^{\mathsf{fw-share}}$ is replaced as follows.

$q_t \notin \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z_t$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0

Table 4. $M_p^{\mathsf{fw-share}}$ for all $z_t > i^*$

– $\mathbf{H}_4^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t^*_{i^*})$: This is identical to $\mathbf{H}_3^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular the evaluation $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ is replaced by a random point $r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_{j-1}}$ and we define $\mathsf{sh}^{t_{i^*}}_{z_j,w^*} = f_{i^*}(\mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1} + w^*)))$ to be hardcoded into $\widetilde{M}_{z_j}$ (as v_1) consistently as Lagrange interpolation of the resulting polynomial.
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value v_2 hardcoded into $\widetilde{M}_{z_t}$ to $v_2 = r_{i^*,j^*}$.
- If $j^* = 0$ and we reach a query q_t such that $z_t = i^*$, we define $K_{i^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||0))$ consistently as Lagrange interpolation of the resulting polynomial.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}^{i^*,j^*}_5(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t^*_{i^*}]$: This is identical to $\mathbf{H}^{i^*,j^*}_4(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $x_{j^*} = \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$ is replaced by a random point $s_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ s_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_{j-1}}$, and we define $\mathsf{sh}^{t_{i^*}}_{z_j,w^*} = f_{i^*}(\mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1} + w^*))$ to be hardcoded into $\widetilde{M}_{z_j}$ (as v_1) consistently as Lagrange interpolation of the resulting polynomial.
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value x_2 hardcoded into $\widetilde{M}_{z_t}$ to $x_2 = s_{i^*,j^*}$.
- If $j^* = 0$ and we reach a query q_t such that $z_t = i^*$, we define $K_{i^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||0))$ consistently as Lagrange interpolation of the resulting polynomial.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}^{i^*,j^*}_6(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t^*_{i^*}]$: This is identical to $\mathbf{H}^{i^*,j^*}_5(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the challenger samples a random point $r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ that will be interpreted as follows:

- If $j^* = 0$: r_{i^*,j^*} is interpreted as the key K_{i^*} (instead of computing it as Lagrange interpolation in $(K_p, i^*||t_{i^*}||0)$);
- Otherwise, r_{i^*,j^*} is interpreted as $\mathsf{sh}^{t_{i^*}}_{z_j,w^*}$ hardcoded into $\widetilde{M}_{z_j}$ (as v_1) computed when we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$.

Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||0) \text{ if } j = j^* = 0 \\ \mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1} + w^*)) \text{ if } j = j^* \neq 0 \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_{j-1}}$, and we change the value v_1 hardcoded into $\widetilde{M}_{z_j}$ to the randomly sampled $\mathsf{sh}^{t_{i^*}}_{z_j,w^*}$.
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value v_2 hardcoded into $\widetilde{M}_{z_t}$ to $v_2 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_2 - x_m}{x_j - x_m}$., where we recall the $x_2 = s_{i^*,j^*}$ from the previous hybrid.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_7^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t^*_{i^*})$: This is identical to $\mathbf{H}_6^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to *all the queries* as follows. For the queries in $\mathcal{Q}$, when we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W^*_{z_{j-1}}$ and for every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t < j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
$t = j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t*	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*	w^*	w^*-1
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*	w^*	w^*-1
$t > j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0

Table 5. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the machine $M_p^{\mathsf{fw-share}}$ is replaced as follows.

$q_t \notin \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z_t$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0

Table 6. $M_p^{\mathsf{fw-share}}$ for all $z_t > i^*$

– $\mathbf{H}_8^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t^*_{i^*})$: This is identical to $\mathbf{H}_7^{i^*,j^*}(\lambda, b)$ except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to *all the queries* as follows. We compute:
- $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||j^*\})$,
- If $j^* = 0$: $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||0\})$, else $K_{p,*} = K_p$,
- $K'_{p,*} = \mathsf{Punc}(K'_p, \{i^*||t_{i^*}||j^*\})$

For the queries in $\mathcal{Q}$, when we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W^*_{z_{j-1}}$, and we compute $K_{p,z_j,*} = \mathsf{Punc}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1}+w^*))$. For every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t < j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z-t}
$t = j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	$K_{p,z_t,*}$	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*	w^*	w^*-1
$t > j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0

Table 7. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the machine $M_p^{\mathsf{fw-share}}$ is replaced as follows.

$q_t \notin \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z_t$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0

Table 8. $M_p^{\mathsf{fw-share}}$ for all $z_t > i^*$

– $\mathbf{H}_9^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t^*_{i^*})$: This is identical to $\mathbf{H}_8^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point

$$x_{j^*} = \begin{cases} \mathsf{Eval}(K_p, i^*||t_{i^*}||0) \text{ if } j^* = 0 \\ \mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||W_{z_j-1} + w^*) \text{ otherwise} \end{cases}$$

(where z_j is the party requested at the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$) is replaced by a random point $x_{j^*} \xleftarrow{\$} \mathbb{Z}_p$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ x_{j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

where r_{i^*,j^*} is the random point sampled in $\mathbf{H}_6^{i^*,j^*}(\lambda, b)$ and interpreted as K_{i^*} if $j^* = 0$ and $\mathsf{sh}^{t_{i^*}}_{z_j,w^*}$ otherwise. As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_j}$ and we define x_1 to be hardcoded into $\widetilde{M}_{z_j}$ consistently as x_{j^*};
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value v_2 hardcoded into $\widetilde{M}_{z_t}$ consistently as Lagrange interpolation of the resulting polynomial.
- If $j^* = 0$ and we reach a query q_t such that $z_t = i^*$, we modify the share σ_{i^*} to include $x_{i^*,0} = x_{j^*}$ rather than $\mathsf{Eval}(K_p, i^*||t_{i^*}||0)$.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_{10}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t^*_{i^*})$: This is identical to $\mathbf{H}_9^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $x_{j^*} \xleftarrow{\$} \mathbb{Z}_p$ is replaced by $x_{j^*} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*)$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*) \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}.$$

where r_{i^*,j^*} is the random point sampled in $\mathbf{H}_6^{i^*,j^*}(\lambda, b)$ and interpreted as K_{i^*} if $j^* = 0$ and $\mathsf{sh}_{z_j,w^*}^{t_{i^*}}$ otherwise. As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_j}$ and we define x_1 to be hardcoded into $\widetilde{M}_{z_j}$ consistently as x_{j^*};
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value v_2 hardcoded into $\widetilde{M}_{z_t}$ consistently as Lagrange interpolation of the resulting polynomial.
- If $j^* = 0$ and we reach a query q_t such that $z_t = i^*$, we modify the share σ_{i^*} to include $x_{i^*,0} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||0)$ rather than x_{j^*}.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_{11}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t^*_{i^*})$: This is identical to $\mathbf{H}_{10}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ is replaced by $\mathsf{Eval}(K_f, i^*||t_{i^*}||j^*)$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||j^*) \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*] \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}.$$

As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_j}$ and we define v_1 to be hardcoded into $\widetilde{M}_{z_j}$ consistently as $\mathsf{Eval}(K_f, i^*||t_{i^*}||j^*)$;
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value v_2 hardcoded into $\widetilde{M}_{z_t}$ consistently as Lagrange interpolation of the resulting polynomial.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t^*_{i^*})$: This is identical to $\mathbf{H}_{11}^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to *all the queries* as follows. For the queries in $\mathcal{Q}$, when we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W^*_{z_{j-1}}$ and for every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t < j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_{t-1}}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z-t}
$t = j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*	w^*	w^*-1
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	$K_{p,z_t,*}$	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*	w^*	w^*-1
$t > j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0

Table 9. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the machine $M_p^{\mathsf{fw-share}}$ is replaced as follows.

$q_t \notin \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z_t$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0

Table 10. $M_p^{\mathsf{fw-share}}$ for all $z_t > i^*$

– $\mathbf{H}_{13}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in [0, t^*_{i^*})$: This is identical to $\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows. For the queries in $\mathcal{Q}$, when we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W^*_{z_{j-1}}$ and for every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M_{z_t}}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t < j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
$t = j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*+1	0	w^*
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*	w^*	w^*-1
$t > j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*+1	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0

Table 11. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

$\mathbf{H}_{14}^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_{13}^{i^*,t_{i^*}-1}(\lambda, b)$, except we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to *all the queries* as follows. We

compute $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||0\})$ and for every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$ by M_0 instead of M_1 where:

$q_j \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall j \in [m]$	$\mathbf{M_0}$	$K_{f,*}$	K_p	K'_p	K_{p,z_j}	W_{z_j-1}	$W^*_{z_j-1}$	w_{z_j}	z_j	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_j}$	0	w_{z_j}
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_j}	W_{z_j-1}	$W^*_{z_j-1}$	w_{z_j}	z_j	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_j}$	0	w_{z_j}

Table 12. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the machine $M_p^{\mathsf{fw-share}}$ is replaced as follows.

$q_t \notin \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z_t$	$\mathbf{M_0}$	$K_{f,*}$	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0

Table 13. $M_p^{\mathsf{fw-share}}$ for all $z_t > i^*$

$\mathbf{H}_{15}^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_{14}^{i^*}(\lambda, b)$, except that we change the answer of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ such that, on input $(n_t, z_t) = (\cdot, i^*)$ the ciphertext of party i^* is changed to $\mathsf{ct}_{i^*} \xleftarrow{\$} \mathbb{Z}_p$ instead of $\mathsf{ct}_{i^*} := K_{i^*} \cdot \mu_b$.

$\mathbf{H}_{16}^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_{15}^{i^*}(\lambda, b)$, except we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to *all the queries* as follows. For every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$ by M_0 instead of M_1 where:

$q_j \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall j \in [m]$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_j}	W_{z_j-1}	$W^*_{z_j-1}$	w_{z_j}	z_j	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_j}$	0	w_{z_j}
	$\mathbf{M_1}$	$K_{f,*}$	K_p	K'_p	K_{p,z_j}	W_{z_j-1}	$W^*_{z_j-1}$	w_{z_j}	z_j	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_j}$	0	w_{z_j}

Table 14. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the machine $M_p^{\mathsf{fw-share}}$ is replaced as follows.

$q_t \notin \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z_t$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	$K_{f,*}$	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0

Table 15. $M_p^{\mathsf{fw-share}}$ for all $z_t > i^*$

We will now run a series of hybrids: $\mathbf{H}_{17}^{i^*,j^*}(\lambda, b), \ldots, \mathbf{H}_{28}^{i^*,j^*}(\lambda, b)$ for $i^* \in [\mathsf{poly}(\lambda)]$, and $j^* \in (t_{i^*}, 0]$:

– $\mathbf{H}_{17}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t_{i^*}^*, 0]$: This is identical to $\mathbf{H}_{28}^{i^*,j^*-1}(\lambda, b)$, (with $\mathbf{H}_{28}^{i^*,-1}(\lambda, b) = \mathbf{H}_{16}^{i^*}(\lambda, b)$), except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows. For the queries in $\mathcal{Q}$, when we reach the query q_j such that $W_{z_{j-1}}^* < j^* \leq W_{z_j}^*$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W_{z_{j-1}}^*$ and for every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t > j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W_{z_{t-1}}^*$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W_{z_{t-1}}^*$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	0
$t = j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W_{z_{t-1}}^*$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}_{z_t,w^*}^{t_{i^*}}$	x_2	v_2	j^*-1	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W_{z_{t-1}}^*$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	w^*
$t < j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W_{z_{t-1}}^*$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W_{z_t}^*$	0	w_{z_t}
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W_{z_{t-1}}^*$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W_{z_t}^*$	0	w_{z_t}

Table 16. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

With:

- $x_1 = \mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*)$,
- $\mathsf{sh}_{z_j,w^*}^{t_{i^*}} = \mathsf{Eval}(K_f, i^*, ||t_{i^*}||j^*)$,
- $x_2 \xleftarrow{\$} \mathbb{Z}_p$,
- $v_2 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{x_2 - x_m}{x_j - x_m}$, with

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*] \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

and

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*] \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_{18}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t_{i^*}^*, 0]$: This is identical to $\mathbf{H}_{17}^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to *all the queries* as follows. We compute:
- $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||j^*\})$,
- if $j^* = 0$, $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||0\})$, else $K_{p,*} = K_p$,
- $K'_{p,*} = \mathsf{Punc}(K'_p, \{i^*||t_{i^*}||j^*\})$

For the queries in $\mathcal{Q}$, when we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W^*_{z_{j-1}}$ and we compute $K_{p,z_j,*} = \mathsf{Punc}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1}+w^*))$. For every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t > j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
$t = j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	$K_{p,z_t,*}$	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
$t < j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}

Table 17. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the machine $M_p^{\mathsf{fw-share}}$ is replaced as follows.

$q_t \notin \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z_t$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0

Table 18. $M_p^{\mathsf{fw-share}}$ for all $z_t > i^*$

– $\mathbf{H}_{19}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t_{i^*}^*, 0]$: This is identical to $\mathbf{H}_{18}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $y_{j^*} = \mathsf{Eval}(K_f, i^*||t_{i^*}||j^*)$ is replaced by $y_{j^*} = r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*] \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_j}$ and we define v_1 to be hardcoded into $\widetilde{M}_{z_j}$ consistently as r_{i^*,j^*};
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value v_2 hardcoded into $\widetilde{M}_{z_t}$ consistently as Lagrange interpolation of the resulting polynomial.
- If $j^* = 0$ and we reach a query q_t such that $z_t = i^*$, we set $K_{i^*} = r_{i^*,j^*}$.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_{20}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t^*_{i^*}, 0]$: This is identical to $\mathbf{H}_{19}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $x_{j^*} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*)$ is replaced by $x_{j^*} = s_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ s_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_j}$ and we define x_1 to be hardcoded into $\widetilde{M}_{z_j}$ consistently as s_{i^*,j^*};
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value v_2 hardcoded into $\widetilde{M}_{z_t}$ consistently as Lagrange interpolation of the resulting polynomial.
- If $j^* = 0$ and we reach a query q_t such that $z_t = i^*$, we modify the share σ_{i^*} to include $x_{i^*,0} = s_{i^*,j^*}$ rather than $\mathsf{Eval}(K'_p, i^*||t_{i^*}||0)$.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_{21}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t^*_{i^*}, 0]$: This is identical to $\mathbf{H}_{20}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $x_{j^*} = s_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ is replaced by:

$$x_{j^*} = \begin{cases} \mathsf{Eval}(K_p, i^*||t_{i^*}||0) \text{ if } j^* = 0 \\ \mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1} + w^*)) \text{ otherwise} \end{cases}$$

(where z_j is the party queries at the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$). Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^* || t_{i^*} || j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^* || t_{i^*} || (2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^* || t_{i^*} || j) \text{ if } j \in [0, j^*) \\ x_{j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^* || t_{i^*} || (2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_j}$ and we define x_1 to be hardcoded into $\widetilde{M}_{z_j}$ consistently as x_{j^*};
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value v_2 hardcoded into $\widetilde{M}_{z_t}$ consistently as Lagrange interpolation of the resulting polynomial. In particular:

$$v_2 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_2 - x_m}{x_j - x_m}$$

- If $j^* = 0$ and we reach a query q_t such that $z_t = i^*$, we modify the share σ_{i^*} to include $x_{i^*,0} = \mathsf{Eval}(K_p, i^* || t_{i^*} || 0)$ rather than s_{i^*,j^*}.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}^{i^*,j^*}_{22}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t^*_{i^*}, 0]$: This is identical to $\mathbf{H}^{i^*,j^*}_{21}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to *all the queries* as follows. For the queries in $\mathcal{Q}$, when we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W^*_{z_{j-1}}$ and for every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t > j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
$t = j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	$K_{p,z_t,*}$	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
$t < j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}

Table 19. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the machine $M_p^{\mathsf{fw-share}}$ is replaced as follows.

$q_t \notin \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z_t$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0

Table 20. $M_p^{\mathsf{fw-share}}$ for all $z_t > i^*$

– $\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t^*_{i^*}, 0]$: This is identical to $\mathbf{H}_{22}^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to *all the queries* as follows. We compute:
- $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||(2^\lambda - j^*)\})$,
- $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||(2^\lambda - j^*)\})$

For the queries in $\mathcal{Q}$, when we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W^*_{z_{j-1}}$ and for every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M_{z_t}}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t > j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
$t = j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
$t < j$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}

Table 21. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the machine $M_p^{\mathsf{fw-share}}$ is replaced as follows.

$q_t \notin \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z_t$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0

Table 22. $M_p^{\mathsf{fw-share}}$ for all $z_t > i^*$

– $\mathbf{H}_{24}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t_{i^*}^*, 0]$: This is identical to $\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the challenger samples $r_{i^*,j^*}, s_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ and the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j[0, j^*) \\ s_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_{j-1}}$, and we change the value v_1 hardcoded into $\widetilde{M}_{z_j}$ to $v_1 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{x_1 - x_m}{x_j - x_m}$, where we recall $x_1 = \mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1} + w^*))$ from hybrid $\mathbf{H}_{21}^{i^*,j^*}(\lambda, b)$.
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the values x_2, v_2 hardcoded into $\widetilde{M}_{z_t}$ to $v_2 = r_{i^*,j^*}$ and $x_2 = s_{i^*,j^*}$.
- If $j^* = 0$ and we reach a query q_t such that $z_t = i^*$, we define $K_{i^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||0))$ consistently as Lagrange interpolation of the resulting polynomial. In particular,

$$K_{i^*} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{\mathsf{Eval}(K_p, i^*||t_{i^*}||0) - x_m}{x_j - x_m}.$$

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_{25}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t_{i^*}^*, 0]$: This is identical to $\mathbf{H}_{24}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $x_{j^*} = s_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ is replaced by $x_{j^*} = \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_{j-1}}$, and we change the value v_1 hardcoded into $\widetilde{M}_{z_j}$ to $v_1 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{x_1 - x_m}{x_j - x_m}$, where we recall $x_1 = \mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1} + w^*))$ from hybrid $\mathbf{H}_{21}^{i^*,j^*}(\lambda, b)$.
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value x_2 hardcoded into $\widetilde{M}_{z_t}$ to $x_2 = \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$.
- If $j^* = 0$ and we reach a query q_t such that $z_t = i^*$, we define $K_{i^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||0))$ consistently as Lagrange interpolation of the resulting polynomial.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_{26}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t^*_{i^*}, 0]$: This is identical to $\mathbf{H}_{25}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the evaluation $y_{j^*} = r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ is replaced by $y_{j^*} = \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

As a consequence, we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows:

- When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists for $j^* = 0$), we denote $w^* = j^* - W^*_{z_{j-1}}$, and we change the value v_1 hardcoded into $\widetilde{M}_{z_j}$ to $v_1 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{x_1 - x_m}{x_j - x_m}$, where we recall $x_1 = \mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1} + w^*))$ from hybrid $\mathbf{H}_{21}^{i^*,j^*}(\lambda, b)$.
- For all the queries q_t with $t \geq j$ (or for all the queries if $j^* = 0$), we change the value v_2 hardcoded into $\widetilde{M}_{z_t}$ to $v_2 = \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$.
- If $j^* = 0$ and we reach a query q_t such that $z_t = i^*$, we define $K_{i^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||0))$ consistently as Lagrange interpolation of the resulting polynomial.

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

– $\mathbf{H}_{27}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t^*_{i^*}, 0]$: This is identical to $\mathbf{H}_{26}^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to *all the queries* as follows. For the queries in $\mathcal{Q}$, when we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote

$w^* = j^* - W^*_{z_{j-1}}$ and for every query $q_t \in \mathcal{Q}$ we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_t}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t > j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_{t-1}}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*-1	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_{t-1}}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
$t = j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_{t-1}}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	0	$\perp$	j^*-1	w^*	w^*-1
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_{t-1}}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	v_2	v_2	j^*-1	w^*	w^*-1
$t < j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_{t-1}}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_{t-1}}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}

Table 23. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the machine $M_p^{\mathsf{fw-share}}$ is replaced as follows.

$q_t \notin \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z_t$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	0	w_{z_t}	z_t	0	0	0	0	0	$\perp$	0	0	0

Table 24. $M_p^{\mathsf{fw-share}}$ for all $z_t > i^*$

– $\mathbf{H}_{28}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)], j^* \in (t_{i^*}^*, 0]$: This is identical to $\mathbf{H}_{27}^{i^*,j^*}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows. When we reach the query q_j such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$), we denote $w^* = j^* - W^*_{z_{j-1}}$ and we change the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_j}$ so that $\widetilde{M}_{z_j}$ no longer enters the if at line 5, namely, on input $(k, t, q) = (i^*, t_{i^*}, w^*)$, the machine behaves normally rather than outputting the explicitly hardwired value $\mathsf{sh}^{t_{i^*}}_{z_j,w^*}$. Hence, we replace the machine $M_p^{\mathsf{fw-share}}$ used to compute $\widetilde{M}_{z_j}$ by M_0 instead of M_1 where:

$q_t \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$t > j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*-1	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
$t = j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	j^*-1	0	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z_t,w^*}$	0	$\perp$	j^*-1	w^*	w^*-1
$t < j$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	K_{p,z_t}	W_{z_t-1}	$W^*_{z_t-1}$	w_{z_t}	z_t	i^*	t_{i^*}	0	0	0	$\perp$	$W^*_{z_t}$	0	w_{z_t}

Table 25. $M_p^{\mathsf{fw-share}}$ for all $z_t \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

$\mathbf{H}_{29}^{i^*}(\lambda, b)$ **for** $i^* \in [\mathsf{poly}(\lambda)]$: This is identical to $\mathbf{H}_{28}^{i^*,0}(\lambda, b)$, except that we change the answers of the oracle $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$ to the queries in $\mathcal{Q}$ as follows. We change the machine used to generate $\widetilde{M}_{z_j}$ so that, on input $(k, t, q) = (i^*, t_{i^*}, \cdot)$, $\widetilde{M}_{z_j}$ no longer enters the if at line 9 in Figure 2. Moreover we set $W^* = 0$. In particular, the machine $M_p^{\mathsf{fw-share}}$ we use to compute $\widetilde{M}_{z_j}$ is replaced by M_0 instead of M_1 where:

$q_j \in \mathcal{Q}$	$\mathbf{M}$	K_f	K_p	K'_p	$K_{p,i}$	W	W^*	w_i	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$j \in [m]$	$\mathbf{M_0}$	K_f	K_p	K'_p	K_{p,z_j}	W_{z_j-1}	0	w_{z_j}	z_j	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	K_{p,z_j}	W_{z_j-1}	$W^*_{z_j-1}$	w_{z_j}	z_j	i^*	t_{i^*}	0	0	0	$\perp$	0	0	0

Table 26. $M_p^{\mathsf{fw-share}}$ for all $z_j \leq i^*$

For all remaining queries, namely those with $z_j > i^*$, the oracle $\mathcal{O}_{\mathsf{share}}$ behaves exactly as in the previous hybrid.

Lemma 22. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{29}^{i^*-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_1^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The proof uses the hybrid argument. Let $\mathcal{Q} = (q_1, \ldots, q_m)$ be the subsequence of oracle queries made to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, where each query q_j has parameters (n_j, z_j) with $z_j \leq i^*$; for every $z \in [0, |\mathcal{Q}|]$, let $\boldsymbol{H}_1^{i^*,z}(\lambda, b)$ be the experiment where we change the first z answers in $\mathcal{Q}$.

Clearly, $\{\mathbf{H}_1^{i^*,0}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{29}^{i^*-1}(\lambda, b))\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_1^{i^*,|\mathcal{Q}|}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_1^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the lemma, it suffices to show that, for all $z \in [|\mathcal{Q}|]$, we have $\{\mathbf{H}_1^{i^*,z-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_1^{i^*,z}(\lambda, b)\}_{\lambda\in\mathbb{N}}$. The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_1^{i^*,z-1}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_1^{i^*,z}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_{z^*} with z^* being the party corrupted in the z-th query in $\mathcal{Q}$.

Let us argue that M_0 and M_1 are functionally equivalent.

Assuming that there is an input (k, t, q) for which M_0, M_1 produce different outputs, then $k \geq z^*$, $t > 0$, and $0 < q \leq w_{z^*}$ must hold. Otherwise they both output $\perp$. We distinguish 2 cases:

- $k = i^*$ and $t = t_{i^*}$: In this case, for every $0 < q \leq w_{z^*}$ in M_0 we do not enter any different branch of code compared to M_1, since no condition is fully satisfied, i.e., $q \neq q_1^*$ and $q > q_2^*$ (recall that $q_1^* = q_2^* = 0$); moreover $j \geq l^*$ (recall that $l^* = 0$), and $v_2 = \perp$. So both M_0 and M_1 will output $(\mathsf{Eval}(K_{z^*}, k||t||(W + q)), S)$.
- $k \neq i^*$ or $t \neq t_{i^*}$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_{p,z^*}, k||t||(W + q)), S)$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\mathbf{H}_1^{i^*,z-1}(\lambda, b)$ and $\mathbf{H}_1^{i^*,z}(\lambda, b)$ are indistinguishable. It follows that $\mathbf{H}_1^{i^*}(\lambda, b)$ and $\mathbf{H}_{29}^{i^*-1}(\lambda, b)$ are indistinguishable.

Lemma 23. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0, 1\}$, it holds that $\{\mathbf{H}_{13}^{i^*,j^*-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_2^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The only difference between $\{\mathbf{H}_2^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{13}^{i^*,j^*-1}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_{z_j} with z_j being the party queried in the query q_j such that $W_{z_{j-1}}^* < j^* \leq W_{z_j}^*$. When $j^* = 0$, no such query exists, hence $\mathbf{H}_{13}^{i^*,-1} \equiv \mathbf{H}_1^{i^*} \equiv \mathbf{H}_2^{i^*,0}$. When $j^* > 0$, the lemma follows by security of Obf.

Let us argue that M_0 and M_1 are functionally equivalent.

Assuming that there is an input (k, t, q) for which M_0, M_1 produce different outputs, then $k \geq z_j$, $t > 0$, and $0 < q \leq w_{z_j}$ must hold. Otherwise they both output $\perp$. We distinguish 3 cases:

- $k = i^*$, $t = t_{i^*}$, and $q = w^*$: In this case, in M_0 we output (x_1, v_1), which corresponds to $(\mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||W_{z_j-1} + w^*)), \mathsf{sh}_{z_j,w^*}^{t_{i^*}})$, while in M_1 we output $(\mathsf{Eval}(K_{p,z_j}, k||t||(W_{z_j-1} + q)), S)$. Notice that $\mathsf{sh}_{z_j,w^*}^{t_{i^*}}$ and S are computed exactly at the same way, i.e., as the Lagrange interpolation in $\mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||(W_{z_j-1} + w^*))$ of a degree $t_{i^*} - 1$ polynomial evaluated at the t_{i^*} distinct points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

and

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

for all $j \in [0, t_{i^*})$.
- $k = i^*$, $t = t_{i^*}$, and $q \leq w^* - 1$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K'_p, k||t||(W_{z_{j-1}}^* + q)), \mathsf{Eval}(K_f, k||t||(W_{z_{j-1}}^* + q)))$.

– $k \neq i^*$ or $t \neq t_{i^*}$ or $q > w^*$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_{p,z_j}, k||t||(W_{z_j-1}+q)), S)$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\{\mathbf{H}_2^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{13}^{i^*,j^*-1}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ are indistinguishable.

Lemma 24. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_2^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_3^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The proof uses the hybrid argument. In particular, for every query $q \in [q_{\mathsf{max}}]$, where q_{max} is the maximum number of queries made by the adversary, we let $\mathbf{H}_3^{i^*,j^*,q}(\lambda,b)$ be the experiment where we change how the challenger computes the shares for the first q queries. Clearly, $\{\mathbf{H}_3^{i^*,j^*,0}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_2^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_3^{i^*,j^*,q_{\mathsf{max}}}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_3^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the lemma, it suffices to show that, for all $q \in [q_{\mathsf{max}}]$, we have $\{\mathbf{H}_3^{i^*,j^*,q-1}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_3^{i^*,j^*,q}(\lambda,b)\}_{\lambda\in\mathbb{N}}$. The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_3^{i^*,j^*,q-1}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_3^{i^*,j^*,q}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_z with z being the party queried in the q-th query.

Let us argue that M_0 and M_1 are functionally equivalent.

Firstly, we analyse the case where the q-th query is in $\mathcal{Q}$, namely, $z \leq i^*$. We let the q-th query be $q_x \in \mathcal{Q}$, and the query $q_j \in \mathcal{Q}$ be the query such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$, so that $t > j$ for every query $q_t \in \mathcal{Q}$). In this case, assuming that there is an input (k,t,q) for which M_0, M_1 produce different outputs, then $k \geq z_x$, $t > 0$, and $0 < q \leq w_{z_x}$ must hold. Otherwise they both output $\perp$. We distinguish 2 major cases:

– $k = i^*$ and $t = t_{i^*}$:
  - If $x < j$: In this case, for every $0 < q \leq w_{z_x}$ neither in M_0 or in M_1 $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ and $\mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$ are never called, as both circuits output $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||(W^*_{z_{x-1}}+q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||(W^*_{z_{x-1}}+q)))$. Notice that $2^\lambda - j^*$ and $W^*_{z_{x-1}} + q$ never intersect, since by assumption $W_n = \mathsf{poly}(\lambda)$.
  - If $x = j$: In this case, neither in M_0 or in M_1 $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ and $\mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$ are never called, as both circuits output:
    * (x_1, v_1) for $q = w^*$,
    * $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||(W^*_{z_{x-1}}+q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||(W^*_{z_{x-1}}+q)))$ for $0 < q < w^*$,
    * $(\mathsf{Eval}(K_{p,z_x}, k||t||(W_{z_x-1}+q)), S)$ for $w^* < q \leq w_{z_x}$, where $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ and $\mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$ are already set within x_2 and v_2;
  - If $x > j$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_{p,z_x}, k||t||(W_{z_x-1}+q)), S)$, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ or $\mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$, whose values are already set within x_2 and v_2;

– $k \neq i^*$ or $t \neq t_{i^*}$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_{p,z_x}, k||t||(W_{z_x-1}+q)), S)$, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ or $\mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$.

Secondly, we analyse the case where the q-th query is not in $\mathcal{Q}$, namely, $z > i^*$. In this case, for every input where $k = i^*$ and $t = t_{i^*}$ (i.e., the inputs corresponding to the point where K_f and K_p have been punctured), both M_0 and M_1 will abort because $z > k$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\mathbf{H}_3^{i^*,j^*,q-1}(\lambda, b)$ and $\mathbf{H}_3^{i^*,j^*,q}(\lambda, b)$ are indistinguishable. It follows that $\mathbf{H}_3^{i^*,j^*}(\lambda, b)$ and $\mathbf{H}_2^{i^*,j^*}(\lambda, b)$ are indistinguishable.

Lemma 25. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_3^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_4^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||(2^\lambda - j^*)$. We can show how to turn any distinguisher D between the two hybrids into an adversary attacking the pseudorandomness at punctured points of PPRF (Definition 2) as follows:

- The reduction receives the messages $\mu_0, \mu_1 \in \mathcal{M}$ from the $\boldsymbol{G}_{\mathsf{SS},\mathsf{D}}^{\mathsf{priv}}$ adversary D.
- The reduction sends to the PPRF security game the point $i^*||t_{i^*}||(2^\lambda - j^*)$ and receives $K^* = \mathsf{Punc}(K, \{i^*||t_{i^*}||(2^\lambda - j^*)\})$ and either a uniform value $\bar{u} \xleftarrow{\$} \mathbb{Z}_p$ or the value $\bar{u} = \mathsf{Eval}(K, i^*||t_{i^*}||(2^\lambda - j^*))$, where K is the PPRF key generated by the PPRF challenger.
- The reduction computes $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||(2^\lambda - j^*)\})$, and $K'_p \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$.
- At each oracle query q_t to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, on input (n_t, z_t):
  - If $q_t \notin \mathcal{Q}$: the reduction computes $x_{z_t,0} = \mathsf{Eval}(K_{p,*}, z_t||t_{z_t}||0)$, $K_{z_t} = \mathsf{Lagrange}(x_{z_t,0}, \{\mathsf{Eval}(K_*, z_t||t_{z_t}||(2^\lambda - j))\}_{j\in[0,t_{z_t})}, \{\mathsf{Eval}(K_p, z_t||t_{z_t}||(2^\lambda - j))\}_{j\in[0,t_{z_t})})$, and $K_{p,z_t} \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$ and sets:

    $$M_{z_t} = M_p^{\mathsf{fw-share}}[K^*, K_{p,*}, K'_p, K_{p,z_t}, W_{z_t-1}, 0, w_{z_t}, z_t, 0, 0, 0, 0, 0, \perp, 0, 0, 0],$$

    and $\mathsf{ct}_{z_t} = K_{z_t} \cdot \mu_b$. Finally $\sigma_{z_t} = (\mathsf{ct}_{z_t}, \widetilde{M_{z_t}}, x_{z_t,0})$.
  - If $q_t \in \mathcal{Q}$:
    - If $W^*_{z_t} < j^*$: the reduction sets $M_{z_t} = M_p^{\mathsf{fw-share}}[K^*, K_{p,*}, K'_p, K_{p,z_t}, W_{z_t-1}, W^*_{z_t-1}, w_{z_t}, z_t, i^*, t_{i^*}, 0, 0, 0, \perp, W^*_{z_t}, 0, w_{z_t}]$. Moreover, if $z_t < i^*$, the reduction sets $\mathsf{ct}_{z_t} \xleftarrow{\$} \mathbb{Z}_p$ and $x_{z_t,0} = \mathsf{Eval}(K_{p,*}, z_t||t_{z_t}||0)$, while if $z_t = i^*$, it sets $K_{z_t} = \mathsf{Eval}(K^*, i^*||t_{i^*}||0)$, $\mathsf{ct}_{z_t} = K_{z_t} \cdot \mu_b$, and $x_{z_t,0} = \mathsf{Eval}(K'_p, z_t||t_{z_t}||0)$. Finally $\sigma_{z_t} = (\mathsf{ct}_{z_t}, \widetilde{M_{z_t}}, x_{z_t,0})$.
    - If $W^*_{z_{t-1}} < j^* \le W^*_{z_t}$: we denote $w^* = j^* - W^*_{z_{t-1}}$, and the reduction computes $f_{i^*}(x)$ as the unique polynomial of degree $t_{i^*} - 1$ obtained by interpolating the values:

      $$y_j = \begin{cases} \mathsf{Eval}(K^*, i^*||t_{i^*}||j), \text{ if } j \in [0, j^*) \\ \bar{u} \text{ if } j = j^* \\ \mathsf{Eval}(K^*, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

at the points

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

Then, the reduction proceeds by setting:

$$x_1 = \mathsf{Eval}(K_{p,z_t}, i^*||t_{i^*}||(W_{z_t-1} + w^*)),$$

$$\mathsf{sh}^{t_{i^*}}_{z_t,w^*} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{\mathsf{Eval}(K_{p,z_t}, i^*||t_{i^*}||(W_{z_t-1} + w^*)) - x_m}{x_j - x_m},$$

$$x_2 = \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*)),$$

and $v_2 = \bar{u}$.
Then, the reduction sets

$$M_{z_t} = M_p^{\mathsf{fw-share}}[K^*, K_{p,*}, K'_p, K_{p,z_t}, W_{z_t-1}, W^*_{z_t-1}, w_{z_t}, z_t, i^*, t_{i^*}, x_1, \mathsf{sh}^{t_{i^*}}_{z_t,w^*}, x_2, v_2, j^*, w^*, w^* - 1].$$

Moreover, if $z_t < i^*$, the reduction sets $\mathsf{ct}_{z_t} \xleftarrow{\$} \mathbb{Z}_p$ and $x_{z_t,0} = \mathsf{Eval}(K_{p,*}, z_t||t_{z_t}||0)$, while if $z_t = i^*$, it set $K_{z_t} = \mathsf{Eval}(K^*, i^*||t_{i^*}||0)$, $\mathsf{ct}_{z_t} = K_{z_t} \cdot \mu_b$, and $x_{z_t,0} = \mathsf{Eval}(K'_p, z_t||t_{z_t}||0)$. Finally $\sigma_{z_t} = (\mathsf{ct}_{z_t}, \widetilde{M_{z_t}}, x_{z_t,0})$.

∗ For all the other queries in $\mathcal{Q}$, the reduction sets

$$M_{z_t} = M_p^{\mathsf{fw-share}}[K^*, K_{p,*}, K'_p, K_{p,z_t}, W_{z_t-1}, W^*_{z_t-1}, w_{z_t}, z_t, i^*, t_{i^*}, 0, 0, x_2, v_2, j^*, 0, 0],$$

where x_2 and v_2 are those computed at the previous step. Moreover, if $z_t < i^*$, the reduction sets $\mathsf{ct}_{z_t} \xleftarrow{\$} \mathbb{Z}_p$ and $x_{z_t,0} = \mathsf{Eval}(K_{p,*}, z_t||t_{z_t}||0)$, while if $z_t = i^*$, if $j^* = 0$ the reduction computes $x_{z_t,0} = \mathsf{Eval}(K_{p,*}, i^*||t_{i^*}||0)$ and $K_{z_t} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{x_{z_t,0} - x_m}{x_j - x_m}$, (where y_j and x_j are those defined at the previous step), otherwise (if $j^* \neq 0$) it sets $x_{z_t,0} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||0)$ and $K_{z_t} = \mathsf{Eval}(K^*, i^*||t_{i^*}||0)$ and then $\mathsf{ct}_{z_t} = K_{z_t} \cdot \mu_b$. Finally $\sigma_{z_t} = (\mathsf{ct}_{z_t}, \widetilde{M_{z_t}}, x_{z_t,0})$.

– Finally, the reduction outputs whatever D outputs.

Lemma 26. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_4^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_5^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||(2^\lambda - j^*)$. The reduction for showing $\{\mathbf{H}_4^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_5^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ is similar to the one used in Lemma 25 where K^* and $\bar{u}$ are used as $K_{p,*}$ and x_2 (and hence x_{j^*}) respectively (instead of $K_{f,*}$ and v_2), while the point v_2 (and hence y_{j^*}) is randomply sampled. We omit it for brevity.

Lemma 27. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_5^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_6^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by Lemma 1. We can show how to turn any distinguisher D between the two hybrids into an adversary attacking the security of Lemma 1 as follows:

- The reduction receives the messages $\mu_0, \mu_1 \in \mathcal{M}$ from the $\boldsymbol{G}^{\mathsf{priv}}_{\mathsf{SS},\mathsf{D}}$ adversary D.
- The reduction computes $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||(2^\lambda - j^*)\})$, $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||(2^\lambda - j^*)\})$, and $K'_p \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$.
- At each oracle query q_t to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, on input (n_t, z_t):
  - If $q_t \notin \mathcal{Q}$: the reduction computes $x_{z_t,0} = \mathsf{Eval}(K_p, z_t||t_{z_t}||0)$, $K_{z_t} = \mathsf{Lagrange}(x_{z_t,0}, \{\mathsf{Eval}(K_f, z_t||t_{z_t}||(2^\lambda - j))\}_{j\in[0,t_{z_t})}, \{\mathsf{Eval}(K_p, z_t||t_{z_t}||(2^\lambda - j))\}_{j\in[0,t_{z_t})})$, and $K_{p,z_t} \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$ and sets:

    $$M_{z_t} = M_p^{\mathsf{fw-share}}[K_{f,*}, K_{p,*}, K'_p, K_{p,z_t}, W_{z_t-1}, 0, w_{z_t}, z_t, 0, 0, 0, 0, 0, \perp, 0, 0, 0],$$

    and $\mathsf{ct}_{z_t} = K_{z_t} \cdot \mu_b$. Finally $\sigma_{z_t} = (\mathsf{ct}_{z_t}, \widetilde{M_{z_t}}, x_{z_t,0})$.
  - If $q_t \in \mathcal{Q}$:
    - If $W^*_{z_t} < j^*$: the reduction sets

      $$M_{z_t} = M_p^{\mathsf{fw-share}}[K_{f,*}, K_{p,*}, K'_p, K_{p,z_t}, W_{z_t-1}, W^*_{z_{t-1}}, w_{z_t}, z_t, i^*, t_{i^*}, 0, 0, 0, \perp, W^*_{z_t}, 0, w_{z_t}].$$

      Moreover, if $z_t < i^*$, the reduction sets $\mathsf{ct}_{z_t} \xleftarrow{\$} \mathbb{Z}_p$ and $x_{z_t,0} = \mathsf{Eval}(K_p, z_t||t_{z_t}||0)$, while if $z_t = i^*$, it set $K_{z_t} = \mathsf{Eval}(K_f, i^*||t_{i^*}||0)$, $\mathsf{ct}_{z_t} = K_{z_t} \cdot \mu_b$, and $x_{z_t,0} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||0)$. Finally $\sigma_{z_t} = (\mathsf{ct}_{z_t}, \widetilde{M_{z_t}}, x_{z_t,0})$.
    - If $W^*_{z_{t-1}} < j^* \le W^*_{z_t}$: we denote $w^* = j^* - W^*_{z_{t-1}}$, and the reduction sends to the security game of Lemma 1:
      - $n = i^*$;
      - $t = t_{i^*}$;
      - $$\mathcal{I} = \{x_j \mid x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

        with $|\mathcal{I}| = t_{i^*} - 1$;
      - $i_{t-1} = \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$;
      - $i^* = \mathsf{Eval}(K_{p,z_t}, i^*||t_{i^*}||(W_{z_t-1} + w^*))$;
      - $$\mathcal{Y} = \{y_j \mid y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

        with $|\mathcal{Y}| = t_{i^*} - 1$;

The Lemma 1 challenger randomly samples y_{t-1} and computes r_b either as a random value or as $f(i^*) = \sum_{j=0}^{t-1} y_j \ell_j(i^*)$, where $\ell_j(x) = \prod_{\substack{0 \leq m \leq t-1 \\ m \neq j}} \frac{x - x_m}{x_j - x_m}$. Then, it forwards r_b to the reduction. The reduction proceeds by setting $x_1 = \mathsf{Eval}(K_{p,z_t}, i^* || t_{i^*} || (W_{z_t - 1} + w^*))$, $v_1 = r_b$, $x_2 \xleftarrow{\$} \mathbb{Z}_p$, $v_2 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_2 - x_m}{x_j - x_m}$ (where $y_{j^*} = r_b$, $x_{j^*} = x_1$, while the values $(y_j)_{j \in [0, t_{i^*}) \setminus \{j^*\}}$ and $(x_j)_{j \in [0, t_{i^*}) \setminus \{j^*\}}$ are those in $\mathcal{Y}$ and in $\mathcal{I}$ respectively). Then, the reduction sets:

$$M_{z_t} = M_p^{\mathsf{fw-share}}[K_{f,*}, K_{p,*}, K'_p, K_{p,z_t}, W_{z_t - 1}, W^*_{z_{t-1}}, w_{z_t}, z_t, i^*, t_{i^*}, x_1, v_1, x_2, v_2, j^*, w^*, w^* - 1].$$

Moreover, if $z_t < i^*$, the reduction sets $\mathsf{ct}_{z_t} \xleftarrow{\$} \mathbb{Z}_p$ and $x_{z_t,0} = \mathsf{Eval}(K_p, z_t || t_{z_t} || 0)$, while if $z_t = i^*$, it set $K_{z_t} = \mathsf{Eval}(K_f, i^* || t_{i^*} || 0)$, $\mathsf{ct}_{z_t} = K_{z_t} \cdot \mu_b$, and $x_{z_t,0} = \mathsf{Eval}(K'_p, i^* || t_{i^*} || 0)$. Finally $\sigma_{z_t} = (\mathsf{ct}_{z_t}, \widetilde{M_{z_t}}, x_{z_t,0})$.

∗ For all the other queries in $\mathcal{Q}$: if $j^* = 0$, the the reduction sends to the security game of Lemma 1:
- $n = i^*$;
- $t = t_{i^*}$;
- $\mathcal{I} = \{\mathsf{Eval}(K_p, i^* || t_{i^*} || (2^\lambda - j))\}_{j \in [j^* + 1, t_{i^*})}$, with $|\mathcal{I}| = t_{i^*} - 1$;
- $i_{t-1} = \mathsf{Eval}(K_p, i^* || t_{i^*} || 2^\lambda)$;
- $i^* = \mathsf{Eval}(K_p, i^* || t_{i^*} || 0)$;
- $\mathcal{Y} = \{\mathsf{Eval}(K_f, i^* || t_{i^*} || (2^\lambda - j))\}_{j \in [j^* + 1, t_{i^*})}$, with $|\mathcal{Y}| = t_{i^*} - 1$;

The Lemma 1 challenger randomly samples y_{t-1} and computes r_b either as a random value or as $f(i^*) = \sum_{j=0}^{t-1} y_j \ell_j(i^*)$, where $\ell_j(x) = \prod_{\substack{0 \leq m \leq t-1 \\ m \neq j}} \frac{x - x_m}{x_j - x_m}$. Then, it forwards r_b to the reduction. The reduction proceeds by setting $x_2 \xleftarrow{\$} \mathbb{Z}_p$, $v_2 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_2 - x_m}{x_j - x_m}$ (where $y_{j^*} = r_b$, $x_{j^*} = \mathsf{Eval}(K_p, i^* || t_{i^*} || 0)$, while the values $(y_j)_{j \in [1, t_{i^*})}$ and $(x_j)_{j \in [1, t_{i^*})}$ are those in $\mathcal{Y}$ and in $\mathcal{I}$ respectively). Then, the reduction sets:

$$M_{z_t} = M_p^{\mathsf{fw-share}}[K_{f,*}, K_{p,*}, K'_p, K_{p,z_t}, W_{z_t - 1}, W^*_{z_{t-1}}, w_{z_t}, z_t, i^*, t_{i^*}, 0, 0, x_2, v_2, j^*, 0, 0].$$

Moreover, if $z_t < i^*$, the reduction sets $x_{z_t,0} = \mathsf{Eval}(K_p, z_t || t_{z_t} || 0)$ and $\mathsf{ct}_{z_t} \xleftarrow{\$} \mathbb{Z}_p$, while if $z_t = i^*$, it sets $x_{z_t,0} = \mathsf{Eval}(K_p, i^* || t_{i^*} || 0)$, $K_{z_t} = r_b$, and $\mathsf{ct}_{z_t} = K_{z_t} \cdot \mu_b$. Finally $\sigma_{z_t} = (\mathsf{ct}_{z_t}, \widetilde{M_{z_t}}, x_{z_t,0})$
While if $j^* \neq 0$: the reduction sets

$$M_{z_t} = M_p^{\mathsf{fw-share}}[K_{f,*}, K_{p,*}, K'_p, K_{p,z_t}, W_{z_t - 1}, W^*_{z_{t-1}}, w_{z_t}, z_t, i^*, t_{i^*}, 0, 0, x_2, v_2, j^*, 0, 0],$$

where x_2 and v_2 are the ones set at the step where $W^*_{z_{t-1}} < j^* \leq W^*_{z_t}$. Moreover, if $z_t < i^*$, the reduction sets $x_{z_t,0} = \mathsf{Eval}(K_p, z_t||t_{z_t}||0)$ and $\mathsf{ct}_{z_t} \xleftarrow{\$} \mathbb{Z}_p$, while if $z_t = i^*$, it sets $x_{z_t,0} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||0)$, $K_{z_t} = \mathsf{Eval}(K_f, i^*||t_{i^*}||0)$, and $\mathsf{ct}_{z_t} = K_{z_t} \cdot \mu_b$. Finally $\sigma_{z_t} = (\mathsf{ct}_{z_t}, \widetilde{M_{z_t}}, x_{z_t,0})$.

– Finally the reduction outputs whatever D outputs.

Lemma 28. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_6^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_7^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf, since, as shown in Lemma 24, $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ and $\mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$ are never called. We omit the proof here for brevity.

Lemma 29. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_7^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_8^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The proof uses the hybrid argument. In particular, for every $q \in [q_{\mathsf{max}}]$, where q_{max} is the maximum number of queries made by the adversary, we let $\mathbf{H}_8^{i^*,j^*,q}(\lambda, b)$ be the experiment where we change how the challenger computes the shares for the first q queries. Clearly, $\{\mathbf{H}_8^{i^*,j^*,0}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_7^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_8^{i^*,j^*,q_{\mathsf{max}}}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_8^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the lemma, it suffices to show that, for all $q \in [q_{\mathsf{max}}]$, we have $\{\mathbf{H}_8^{i^*,j^*,q-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_8^{i^*,j^*,q}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.

The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_8^{i^*,j^*,q-1}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_8^{i^*,j^*,q}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_z with z being the party queried in the q-th query.

Firstly, we analyse the case where the q-th query is in $\mathcal{Q}$, namely $z \leq i^*$. We define q_j as the query in $\mathcal{Q}$ such that $W^*_{z_{j-1}} < j^* \leq W^*_{z_j}$ (note that no such query exists when $j^* = 0$, in that case we can assume $j = 0$), and we denote $w^* = j^* - W^*_{z_{j-1}}$. We let the q-th query be $q_x \in \mathcal{Q}$.

In this case, assuming that there is an input (k, t, q) for which M_0, M_1 produce different outputs, then $k \geq z_x$, $t > 0$, and $0 < q \leq w_{z_x}$ must hold. Otherwise they both output $\perp$. We distinguish 2 major cases:

– If $k = i^*$ and $t = t_{i^*}$:
  - If $x < j$: In this case, for every $0 < q \leq w_{z_x}$ neither in M_0 or in M_1 $\mathsf{Eval}(K_f, i^*||t_{i^*}||j^*)$ and $\mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*)$ are never called, as both circuits output $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||(W^*_{z_{x-1}} + q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||(W^*_{z_{x-1}} + q)))$ and for every $q \in [w_{z_x}]$ we have that $W^*_{z_{x-1}} + q \leq W^*_{z_x} \leq W^*_{z_{j-1}}$ (since $x < j$), and $W^*_{z_{j-1}} < j^*$ by definition, hence $W^*_{z_{x-1}} + q < j^*$ for every $q \in [w_{z_x}]$.
  - If $x = j$: In this case, neither in M_0 or in M_1 $\mathsf{Eval}(K_f, i^*||t_{i^*}||j^*)$ and $\mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*)$ are never called, as both circuits output:
    * (x_1, v_1) for $q = w^*$;

* $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||(W^*_{z_{x-1}}+q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||(W^*_{z_{x-1}}+q)))$ for $0 < q < w^*$;
* $(\mathsf{Eval}(K_{p,z_x}, k||t||(W_{z_x}+q)), S)$ for $w^* < q \leq w_{z_x}$, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||j^*)$ or $\mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*)$ because l^* is equal to j^* (hence we exit the while-loop), and without ever calling $\mathsf{Eval}(K_p, i^*||t_{i^*}||0)$ or $\mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||W_{z_j-1}+w^*)$ because $W_{z_x-1}+q > 0$ for every $q \in [w_{z_x}]$ and $W_{z_j-1}+w^* < W_{z_j-1}+q$ for $w^* < q \leq w_{z_x}$;

- If $x > j$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_{p,z_x}, k||t||(W_{z_x-1}+q)), S)$, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||j^*)$ or $\mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*)$ because l^* is equal to j^*, and without ever calling $\mathsf{Eval}(K_p, i^*||t_{i^*}||0)$ or $\mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||W_{z_j-1}+w^*)$ because $W_{z_x-1}+q > 0$ and $K_{p,z_j} \neq K_{p,z_x}$ since $x > j$.

– $k \neq i^*$ or $t \neq t_{i^*}$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_{p,z_x}, k||t||(W_{z_x-1}+q)), S)$, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||j^*)$ or $\mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*)$ or $\mathsf{Eval}(K_{p,z_j}, i^*||t_{i^*}||W_{z_j-1}+w^*)$ (or $\mathsf{Eval}(K_p, i^*||t_{i^*}||0)$).

Secondly, we analyse the case where the q-th query is not in $\mathcal{Q}$, namely $z > i^*$. In this case, for every input $k = i^*$ and $t = t_{i^*}$ (i.e., the inputs corresponding to the points where K_f,K_p,K'_p,K_{p,z_j} have been punctured), both M_0 and M_1 will abort because $z > k$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus, conditioned on E (only if $j^* > 0$), $\mathbf{H}_8^{i^*,j^*,q-1}(\lambda, b)$ and $\mathbf{H}_8^{i^*,j^*,q}(\lambda, b)$ are indistinguishable.

Lemma 30. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_8^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_9^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||0$ if $j^* = 0$, and $i^*||t_{i^*}||(W_{z_j-1}+w^*)$ if $j^* > 0$. The reduction for showing $\{\mathbf{H}_8^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_9^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ is similar to the one used in Lemma 25, where K^* and $\bar{u}$ are used as $K_{p,*}$ and $x_{i^*,0}$ (if $j^* = 0$) and as $K_{p,z_j,*}$ and x_1 (if $j^* > 0$) (rather than $K_{f,*}$ and v_2). Moreover x_2 is sampled randomly and v_2 is computed as Lagrange interpolation. We omit it for brevity.

Lemma 31. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_9^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{10}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||j^*$. The reduction for showing $\{\mathbf{H}_9^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{10}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ is similar to the one used in Lemma 25, where K^* and $\bar{u}$ are used as $K'_{p,*}$ and $x_{i^*,0}$ (if $j^* = 0$) and as $K'_{p,*}$ and x_1 (if $j^* > 0$) (rather than $K_{f,*}$ and v_2). Moreover x_2 is sampled randomly and v_2 is computed as Lagrange interpolation. We omit it for brevity.

Lemma 32. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{10}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{11}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||j^*$. The reduction for showing $\{\mathbf{H}_{10}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{11}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ is similar to the one used in Lemma 25 where K^* and $\bar{u}$ are used as $K_{f,*}$ and K'_{i^*} (if $j^*=0$) and as $K_{f,*}$ and v_1 (if $j^*>0$) (rather than $K_{f,*}$ and v_2). Moreover x_2 is sampled randomly and v_2 is computed as Lagrange interpolation. We omit it for brevity.

Lemma 33. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{11}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{12}^{i^*,j^*}(\lambda,b))\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 29 with M_0 and M_1 inverted.

Lemma 34. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in [0, t_{i^*})$, and for every $b \in \{0,1\}$, it holds that $\mathbf{H}_{12}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{13}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The proof uses the hybrid argument. In particular, let $\mathcal{Q} = (q_1, \ldots, q_m)$ be the subsequence of oracle queries made to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$, where each query q_j has parameters (n_j, z_j) with $z_j \le i^*$; for every $z \in [0, |\mathcal{Q}|]$, let $\mathbf{H}_{13}^{i^*,j^*,z}(\lambda,b)$ be the experiment where we change the first z answers to the queries in $\mathcal{Q}$. Clearly, $\{\mathbf{H}_{13}^{i^*,j^*,0}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{12}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{13}^{i^*,j^*,|\mathcal{Q}|}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{13}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the lemma, it suffices to show that, for all $z \in [|\mathcal{Q}|]$, we have $\{\mathbf{H}_{13}^{i^*,j^*,z-1}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{13}^{i^*,j^*,z}(\lambda,b)\}_{\lambda\in\mathbb{N}}$. The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_{13}^{i^*,j^*,z-1}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{13}^{i^*,j^*,z}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_{z^*} with z^* being the party corrupted in the z-th query to $\mathcal{O}_{\mathsf{share}}(\cdot,\cdot)$. We let the z-th query be $q_x \in \mathcal{Q}$, and the query q_j be the first query such that $W^*_{z_j} \ge j^*$.

Let us argue that M_0 and M_1 are functionally equivalent.

Assuming that there is an input (k,t,q) for which M_0, M_1 produce different outputs, then $k \ge z_x$, $t > 0$, and $0 < q \le w_{z_x}$ must hold. Otherwise they both output $\perp$. We distinguish 2 major cases:

- $k = i^*$ and $t = t_{i^*}$:
  - If $x < j$, for any $0 < q \le w_{z_x}$, both M_0 and M_1 outputs $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||(W^*_{z_{x-1}}+q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||(W^*_{z_{x-1}}+q))$;
  - If $x = j$: if $q = w^*$, M_1 outputs (x_1, v_1) while M_0 outputs $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||(W^*_{z_{x-1}}+q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||(W^*_{z_{x-1}}+q))$, which are exactly the same values of x_1 and v_1; otherwise, if $q < w^*$, both M_0 and M_1 outputs $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||(W^*_{z_{x-1}}+q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||(W^*_{z_{x-1}}+q))$; for $w^* < q < w_{z_x}$ the outputs are the same described in the next case;
  - If $x > j$: M_0 and M_1 differs on how they compute the j^*-th evaluation of the polynomial. In particular, in M_1 we use x_2 and v_2, while in M_0, since we incremented l^*, we compute the j^*-th evaluation within the while loop ($j < l^*$). Notice that

$$v_2 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{x_2 - x_m}{x_j - x_m}$$

with

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*] \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

and

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||j) \text{ if } j \in [0, j^*] \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

Hence, in M_1 the j^*-th evaluation is basically v_2, computed as Lagrange interpolation of a polynomial where $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*), \mathsf{Eval}(K_f, i^*||t_{i^*}||j^*))$ is one of the points, while in M_0, the j^*-th evaluation is directly $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*), \mathsf{Eval}(K_f, i^*||t_{i^*}||j^*))$. It follows that M_0 and M_1 will output $(\mathsf{Eval}(K_{p,z_x}, i^*||t_{i^*}||(W_{z_x-1}+q)), S)$ (with exactly the same S).

- $k \neq i^*$ and $t \neq t_{i^*}$: both M_0 and M_1 will output $(\mathsf{Eval}(K_{p,z_x}, i^*||t_{i^*}||(W_{z_x-1}+q)), S)$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\mathbf{H}_{13}^{i^*,j^*,z-1}(\lambda, b)$ and $\mathbf{H}_{13}^{i^*,j^*,z}(\lambda, b)$ are indistinguishable. It follows that $\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)$ and $\mathbf{H}_{13}^{i^*,j^*}(\lambda, b)$ are indistinguishable.

Lemma 35. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\mathbf{H}_{13}^{i^*,t_{i^*}-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{14}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. Indeed all the parties $z \leq i^*$ now enter the if at line 7 in Figure 2, thus $\mathsf{Eval}(K_f, i^*||t_{i^*}||0)$ is never evaluated within the machines, while for all the parties $z > i^*$ the machines abort for an input $(k, t, \cdot) = (i^*, t_{i^*}, \cdot)$.

Lemma 36. *For every $i^* \in [\mathsf{poly}(\lambda)]$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{14}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{15}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by observing that the two hybrids differ only in the answer of $\mathcal{O}_{\mathsf{share}}$ on input (n_g, i^*); in the former hybrid $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu_b$ is a one-time-pad encryption under a uniform key K_{i^*}, while in the latter ct_{i^*} is sampled uniformly at random, which is identically distributed to a one-time-pad encryption.

Lemma 37. *For every $i^* \in [\mathsf{poly}(\lambda)]$, and for every $b \in \{0,1\}$, it holds that $\mathbf{H}_{15}^{i^*,t_{i^*}-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{16}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The proof is analogous to the one in Lemma 35.

Lemma 38. *For every $i^* \in [\mathsf{poly}(\lambda)]$, $j^* \in (t_{i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{28}^{i^*,j^*-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{17}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 34, with M_0 and M_1 inverted.

Lemma 39. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{17}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{18}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 33, with M_0 and M_1 inverted.

Lemma 40. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{18}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{19}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by pseudorandomness at the punctured point $i^*||t_{i^*}||j^*$. The proof is analogous to that in Lemma 32, with $K_{i^*} \xleftarrow{\$} \mathbb{Z}_p$ and $K_{i^*} = \mathsf{Eval}(K_f, i^*||t_{i^*}||0)$ inverted (if $j^* = 0$), and with $v_1 \xleftarrow{\$} \mathbb{Z}_p$ and $v_1 = \mathsf{Eval}(K_f, i^*||t_{i^*}||j^*)$ inverted (if $j^* > 0$).

Lemma 41. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{19}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{20}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by pseudorandomness at the punctured point $i^*||t_{i^*}||j^*$. The proof is analogous to that in Lemma 31, with $x_{i^*,0} \xleftarrow{\$} \mathbb{Z}_p$ and $x_{i^*,0} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||0)$ inverted (if $j^* = 0$), and with $x_1 \xleftarrow{\$} \mathbb{Z}_p$ and $x_1 = \mathsf{Eval}(K'_p, i^*||t_{i^*}||j^*)$ inverted (if $j^* > 0$).

Lemma 42. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{20}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{21}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||(W_{z_x-1} + w^*)$ (or $i^*||t_{i^*}||0$ if $j^* = 0$). The proof is analogous to that in Lemma 30 with $x_{i^*,0} = \mathsf{Eval}(K_p, i^*||t_{i^*}||0)$ and $x_{i^*,0} \xleftarrow{\$} \mathbb{Z}_p$ inverted (if $j^* = 0$), and with $x_1 = \mathsf{Eval}(K_{p,z_x}, i^*||t_{i^*}||(W_{z_x-1} + w^*))$ and $x_1 \xleftarrow{\$} \mathbb{Z}_p$ inverted (if $j^* > 0$).

Lemma 43. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{21}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{22}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 29, with M_0 and M_1 inverted.

Lemma 44. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{22}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 28, with M_0 and M_1 inverted.

Lemma 45. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{24}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by security of Lemma 1. The reduction for showing $\{\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{24}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ is analogous to the one used in Lemma 27. We omit it for brevity.

Lemma 46. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{24}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{25}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||(2^\lambda - j^*)$. The proof is analogous to that in Lemma 26 with s_{i^*,j^*} and $\mathsf{Eval}(K_p, i^*||t_{i^*}||(2^\lambda - j^*))$ inverted.

Lemma 47. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{25}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{26}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||(2^\lambda - j^*)$. The proof is analogous to that in Lemma 26 with r_{i^*,j^*} and $\mathsf{Eval}(K_f, i^*||t_{i^*}||(2^\lambda - j^*))$ inverted.

Lemma 48. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{26}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{27}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 24, with M_0 and M_1 inverted.

Lemma 49. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*,* $j^* \in (t_{i^*}, 0]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{27}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{28}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 23, with M_0 and M_1 inverted.

Lemma 50. *For every* $i^* \in [\mathsf{poly}(\lambda)]$*, and for every* $b \in \{0,1\}$*, it holds that* $\{\mathbf{H}_{28}^{i^*,0}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{29}^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$*.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 22, with M_0 and M_1 inverted.

Lemma 51. $\{\mathbf{H}_{29}^{n}(\lambda, 0)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{29}^{n}(\lambda, 1)\}_{\lambda\in\mathbb{N}}$*, where* $n \in \mathsf{poly}(\lambda)$ *is the number of parties chosen by the adversary.*

Proof. The lemma follows by simply observing that the distribution of the shares $(\sigma_u)_{u\in\mathcal{U}}$ where $\mathcal{U} = \bigcup_{q\in[q_{\mathsf{max}}]} \mathcal{U}_q$ is now independent of the message.

A.7 Proof of Theorem 10

Proof. The proof is similar to that of Theorem 7 (Appendix A.4), with the following differences:

- When constructing the set of PPRF inputs $\mathcal{Q}$ on which we measure the probability of presenting a collision, these points must include the SSB hash h_n.
- In addition, we need to measure only the probability of a collision for a single PPRF indexed by K_p (in contrast to the proof of Theorem 7, where we consider the collision probability of any pair of PPRFs indexed by K and K^*, with $(K, K^*) \in (\{K_p\} \cup \{K_{p,i}\}_{i\in\mathcal{I}})^2$.
- Since there is a single PPRF indexed by K_p, it is sufficient to consider only the case $K = K^* = K_p$ (and the related distinguisher $\mathsf{D}^=$) described in Appendix A.4.

A.8 Proof of Theorem 11

Proof. The proof of the theorem is essentially identical to that of Theorem 8 of Construction 5.1 except that when calculating the maximum input-size $\ell^{\mathsf{max}}_{M^{\mathsf{fw-share}}}$, worst-case running time $t^{\mathsf{max}}_{M^{\mathsf{dw-share}}}$, and description size $|M_p^{\mathsf{dw-share}}|$ we must consider the following bounds that are implied by the efficiency and succinctness of SSB (Definition 7):

- Since p is a prime of (λ^c)-bits for constant $c > 1$, each value in $\mathbb{F}_p$ can be represented using $|\mathbb{F}_p| = \log(p) = \log(\lambda^c) = O(\log \lambda)$;
- ℓ_{hash} and ℓ_{open} (hashes and openings produced by SSB) are bounded by $\mathsf{poly}(\lambda, |w|, \mathsf{suplog}(\lambda)) = \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$ since $|w| = O(\log \lambda)$;
- The hardcoded value hk is of size at most $|\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, 2^\lambda, 1)| = \mathsf{poly}(\lambda, |w| \mathsf{suplog}(\lambda)) = \mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$ since $|w| = O(\log \lambda)$;[16]
- The runtime of $\mathsf{Verify}(\mathsf{hk}, \cdot, \cdot, \cdot, \cdot)$ is bounded by $\mathsf{poly}(\lambda, \log(2^\lambda)) = \mathsf{poly}(\lambda)$;
- The description of the Turing machine code for $\mathsf{Verify}(\mathsf{hk}, h, i, w, \pi)$ has description size of $\mathsf{poly}(\lambda, \log(2^\lambda)) = \mathsf{poly}(\lambda)$.

A.9 Proof of Theorem 12

Restatement of Theorem 12. *If the iO obfuscator* Obf *is secure (Definition 9)* PPRF *is secure (Definition 2), and* SSB *is index hiding (Definition 5) and somewhere perfectly binding (Definition 6), then the scheme ESS scheme of Construction 6.1, for the evolving threshold access structure with dynamic weights* $\hat{\mathcal{A}}^{\mathsf{dw-ethr}}$ *with weights of size at most* $|w| = O(\log \lambda)$*, is a computationally private in the static setting (Definition 15).*

Proof. Let n, and $\mathcal{U} \notin \hat{\mathcal{A}}^{\mathsf{dw-ethr}}$ be, respectively, the number of parties, and the unqualified subset chosen by the adversary in the game defining the static computational privacy of $\boldsymbol{G}^{\mathsf{priv}}_{\mathsf{SS,A}}(\lambda, b)$. Consider the following hybrid experiments.

$\mathbf{H}_0(\lambda, b)$: This experiment is identical to $\boldsymbol{G}^{\mathsf{priv}}_{\mathsf{SS,A}}(\lambda, b)$.

We define:

- $W^*_{k^*,i^*} = \sum_{\substack{z \in \mathcal{U} \\ z \leq k^*}} \mathsf{weight}_{i^*}(z)$,
- $j^* \in [0, W^*_{i^*,i^*}]$,
- $z^* = \begin{cases} 0 \text{ if } j^* = 0 \\ \min(u \in \mathcal{U} \mid W^*_{u,i^*} > j^*) \text{ otherwise} \end{cases}$,
- $w^* = j^* - W^*_{z^*-1,i^*}$.

[16] Recall that, as described in the proof of Theorem 8, the values $\mathsf{suplog}(\lambda)$ are used to safely encode any value that depends polynomially on $n \in \mathsf{poly}(\lambda)$, for an *a priori* unknown value of n. This implies that $\mathsf{poly}(\lambda, \mathsf{suplog}(\lambda))$ can be rewritten as $\mathsf{poly}(\lambda, \log n)$.

$\mathbf{H}_1^{i^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}$: This is identical to $\mathbf{H}_{31}^{i^*-1}(\lambda, b)$, except that we change the machine used to generate $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In particular, the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1 where:

Case	**M**	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	0	z	0	0	0	0	0	$\perp$	0	0	0

Table 27. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

We will now run a series of hybrids: $\mathbf{H}_2^{i^*,j^*}(\lambda, b), \ldots, \mathbf{H}_{14}^{i^*,j^*}(\lambda, b)$ for $i^* \in \mathcal{U}$, and $j^* \in [0, W^*_{i^*,i^*}]$:

- $\mathbf{H}_2^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_{14}^{i^*,j^*-1}(\lambda, b)$ (with $\mathbf{H}_{14}^{i^*,-1}(\lambda, b) = \mathbf{H}_1^{i^*}(\lambda, b)$), except that, only if $j^* > 0$, we generate $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, 2^\lambda, z^*)$ binding to z^*.
- $\mathbf{H}_3^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_2^{i^*,j^*}(\lambda, b)$, except that, only if $j^* > 0$, we compute $h_{i^*} = \mathsf{Hash}(\mathsf{hk}, (\mathsf{weight}_{i^*}(i))_{i \in [i^*]})$, and we change the machine $M_p^{\mathsf{dw-share}}$ used to compute $\widetilde{M}_{z^*}$ so that $\widetilde{M}_{z^*}$ enters the if at line 5 in Figure 3, namely, on input $(k, t, h, \pi, w, q) = (i^*, t_{i^*}, h, \pi, w, w^*)$, (for some valid h, π, w), it outputs the explicitly hardwired value $\mathsf{sh}_{z^*,w^*}^{t_{i^*}}$, and otherwise behaves as before. Hence, we replace the machine $M_p^{\mathsf{dw-share}}$ used to compute $\widetilde{M}_{z^*}$ by M_0 instead of M_1 where:

Case	**M**	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z < z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
$z = z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	x_1	$\mathsf{sh}_{z^*,w^*}^{t_{i^*}}$	0	$\perp$	j^*	w^*	$w^* - 1$
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	$w^* - 1$
$z > z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	0

Table 28. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

With:

- $x_1 = \mathsf{Eval}(K_p, i^* || t_{i^*} || h_{i^*} || (2^{z^*\lambda} + w^*))$,

- $v_1 = \mathsf{sh}^{t_{i^*}}_{z^*,w^*}$ where:

$$\mathsf{sh}^{t_{i^*}}_{z^*,w^*} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda}+w^*)) - x_m}{x_j - x_m}$$

with

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

and

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

for all $j \in [0, t_{i^*})$,
- $q_1^* = w^*$.

If $j^* = 0$ the game is identical to $\mathbf{H}_2^{i^*,j^*}(\lambda, b)$.

– $\mathbf{H}_4^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_3^{i^*,j^*}(\lambda, b)$, except that we compute:
- $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)\})$,
- $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)\})$,
- $x_2 = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$,
- $v_2 = \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$,

and we change the machine used to generate $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In particular, the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1 where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z < z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
$z = z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	0	$\perp$	j^*	w^*	w^*-1
$z > z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	0

Table 29. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

– $\mathbf{H}_5^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_4^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the evaluation $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ is replaced by a random point $r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

Moreover, we set $v_2 = r_{i^*,j^*}$, and if $j^* = 0$ we compute $K_{i^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0))$ consistently as Lagrange interpolation of the resulting polynomial, while if $j^* > 0$ we compute $\mathsf{sh}^{t_{i^*}}_{z^*,w^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*))$ consistently as Lagrange interpolation of the resulting polynomial.

– $\mathbf{H}_6^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_5^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $x_{j^*} = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ is replaced by random point $s_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ s_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

Moreover, we set $x_2 = s_{i^*,j^*}$, and if $j^* = 0$ we compute $K_{i^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0))$ consistently as Lagrange interpolation of the resulting polynomial, while if $j^* > 0$ we compute $\mathsf{sh}^{t_{i^*}}_{z^*,w^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*))$ consistently as Lagrange interpolation of the resulting polynomial.

– $\mathbf{H}_7^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_6^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the challenger samples a random point $r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ that will be interpreted as follows:

- If $j^* = 0$: r_{i^*,j^*} is interpreted as the key K_{i^*} (instead of computing it as Lagrange interpolation in $(K_p, i^*||t_{i^*}||h_{i^*}||0)$);
- Otherwise, r_{i^*,j^*} is interpreted as $\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$ hardcoded into $\widetilde{M}_{z^*}$ (as v_1).

Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0) \text{ if } j = j^* = 0 \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*)) \text{ if } j = j^* \neq 0 \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

Moreover, we set

$$v_2 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_2 - x_m}{x_j - x_m}$$

with y_j and x_j (for all $j \in [0, t_{i^*})$) defined as above. Recall that in the previous hybrid we set $x_2 = s_{i^*,j^*}$.

– $\mathbf{H}_8^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_7^{i^*,j^*}(\lambda, b)$, except that the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1, where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z < z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
$z = z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*	w^*	w^*-1
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*	w^*	w^*-1
$z > z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0

Table 30. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

– $\mathbf{H}_9^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_8^{i^*,j^*}(\lambda, b)$, except that we compute:

- $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||h_{i^*}||j^*\})$,
- If $j^* = 0$: $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||h_{i^*}||0\})$, else $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*)\})$,
- $K'_{p,*} = \mathsf{Punc}(K'_p, \{i^*||t_{i^*}||h_{i^*}||j^*\})$

and the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1, where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z < z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
$z = z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*	w^*	w^*-1
$z > z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0

Table 31. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

– $\mathbf{H}_{10}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_9^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular,

the point

$$x_{j^*} = \begin{cases} \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0) \text{ if } j^* = 0 \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*)) \text{ otherwise} \end{cases}$$

is replaced by a random point $x_{j^*} \xleftarrow{\$} \mathbb{Z}_p$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ x_{j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

where r_{i^*,j^*} is the random point sampled in $\mathbf{H}_7^{i^*,j^*}(\lambda, b)$ and interpreted as K_{i^*} if $j^* = 0$ and $\mathsf{sh}_{z^*,w^*}^{t_{i^*}}$ otherwise. Moreover, we set $x_1 = x_{j^*}$, v_2 consistently as Lagrange interpolation of the resulting polynomial and, if $j^* = 0$, we modify the share σ_{i^*} to include $x_{i^*,0} = x_{j^*}$ rather than $\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0)$.

- $\mathbf{H}_{11}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_{10}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $x_{j^*} \xleftarrow{\$} \mathbb{Z}_p$ is replaced by $x_{j^*} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*)$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*) \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}.$$

where r_{i^*,j^*} is the random point sampled in $\mathbf{H}_7^{i^*,j^*}(\lambda, b)$ and interpreted as K_{i^*} if $j^* = 0$ and $\mathsf{sh}_{z^*,w^*}^{t_{i^*}}$ otherwise. Moreover, we set $x_1 = x_{j^*}$, v_2 consistently as Lagrange interpolation of the resulting polynomial and, if $j^* = 0$, we modify the share σ_{i^*} to include $x_{i^*,0} = x_{j^*}$ rather than $\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0)$.

- $\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_{11}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ is replaced by $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*)$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*) \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*] \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

Moreover, we set v_2 consistently as Lagrange interpolation of the resulting polynomial, and if $j^* > 0$ we set $v_1 = \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*)$.

– $\mathbf{H}_{13}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_{12}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes the obfuscation $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In particular, the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1, where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z < z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
$z = z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*	w^*	w^*-1
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*	w^*	w^*-1
$z > z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0

Table 32. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

– $\mathbf{H}_{14}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [0, W^*_{i^*,i^*}]$: This is identical to $\mathbf{H}_{13}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes the obfuscation $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In particular, the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1, where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z < z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
$z = z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	0	0	0	$\perp$	j^*+1	0	w^*
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	$W^*_{z^*-1,i^*}$	z^*	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*	w^*	w^*-1
$z > z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	j^*+1	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*	0	0

Table 33. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

$\mathbf{H}_{15}^{i^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}$: This is identical to $\mathbf{H}_{14}^{i^*,W^*_{i^*,i^*}}(\lambda, b)$, except that we change how the challenger computes the obfuscation $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In particular, we compute $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||h_{i^*}||0)\})$ and the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1, where:

Case	C	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z$	$\mathbf{C_0}$	$K_{f,*}$	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{C_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$

Table 34. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

$\mathbf{H}_{16}^{i^*}(\lambda, b)$: This is identical to $\mathbf{H}_{15}^{i^*}(\lambda, b)$, except that we change how the challenger computes the share for the party i^*. In particular, the challenger replaces $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu_b$ with $\mathsf{ct}_{i^*} \xleftarrow{\$} \mathbb{Z}_p$.

$\mathbf{H}_{17}^{i^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}$: This is identical to $\mathbf{H}_{16}^{i^*}(\lambda, b)$, except that we change how the challenger computes the obfuscation $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In particular, the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1, where:

Case	C	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z$	$\mathbf{C_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{C_1}$	$K_{f,*}$	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$

Table 35. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

We will now run a series of hybrids: $\mathbf{H}_{18}^{i^*,j^*}(\lambda, b), \ldots, \mathbf{H}_{30}^{i^*,j^*}(\lambda, b)$ for $i^* \in \mathcal{U}$ and $j^* \in [W^*_{i^*,i^*}, 0]$ where we undo all the moves from $\mathbf{H}_2^{i^*,j^*}(\lambda, b)$ to $\mathbf{H}_{14}^{i^*,j^*}(\lambda, b)$, except for removing the message from the ciphertext ct_{i^*}.

- $\mathbf{H}_{18}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{30}^{i^*,j^*-1}(\lambda, b)$, (with $\mathbf{H}_{30}^{i^*,-1}(\lambda, b) = \mathbf{H}_{17}^{i^*}(\lambda, b)$), except that, only if $j^* > 0$, we generate $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, 2^\lambda, z^*)$ binding to z^*.
- $\mathbf{H}_{19}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{18}^{i^*,j^*}(\lambda, b)$, except that the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all $z \in \mathcal{U}$ is replaced by M_0 instead of M_1 where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z > z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	0
$z = z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	j^*	0	w^*
$z < z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$

Table 36. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

With:

- $x_1 = \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*)$
- $\mathsf{sh}^{t_{i^*}}_{z^*,w^*} = \mathsf{Eval}(K_f, i^*, ||t_{i^*}||h_{i^*}||j^*)$,
- $x_2 \xleftarrow{\$} \mathbb{Z}_p$,
- $v_2 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m\neq j}}^{t_{i^*}-1} \frac{x_2 - x_m}{x_j - x_m}$, with

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*] \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

and

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*] \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

– $\mathbf{H}_{20}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{19}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes the obfuscation $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In particular, we compute:

- $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||h_{i^*}||j^*)\})$,
- If $j^* = 0$: $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||h_{i^*}||0\})$, else $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*)\})$,
- $K'_{p,*} = \mathsf{Punc}(K'_p, \{i^*||t_{i^*}||h_{i^*}||j^*\})$

and the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1 where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z > z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
$z = z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
$z < z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$

Table 37. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

– $\mathbf{H}_{21}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{20}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $y_{j^*} = \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*)$ is replaced by $y_{j^*} = r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*] \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

Moreover, we set v_2 consistently as Lagrange interpolation of the resulting polynomial, and if $j^* > 0$ we set $v_1 = r_{i^*,j^*}$, while if $j^* = 0$ we set $K_{i^*} = r_{i^*,j^*}$.

– $\mathbf{H}_{22}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{21}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $x_{j^*} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*)$ is replaced by $x_{j^*} = s_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{PPRF}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ s_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

Moreover, we set v_2 consistently as Lagrange interpolation of the resulting polynomial, and if $j^* > 0$ we set $x_1 = s_{i^*,j^*}$, while if $j^* = 0$ we modify the share σ_{i^*} to include $x_{i^*,0} = s_{i^*,j^*}$ rather than $\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||0)$.

– $\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{22}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $x_{j^*} = s_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ is replaced by:

$$x_{j^*} = \begin{cases} \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0) \text{ if } j^* = 0 \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*)) \text{ otherwise} \end{cases}$$

Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ x_{j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

Moreover, we set v_2 consistently as Lagrange interpolation of the resulting polynomial, and if $j^* > 0$ we set $x_1 = x_{j^*}$, while if $j^* = 0$ we modify the share σ_{i^*} to include $x_{i^*,0} = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0)$ rather than s_{i^*,j^*}.

– $\mathbf{H}_{24}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes the obfuscation $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In particular, the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1 where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z > z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
$z = z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
$z < z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	$K'_{p,*}$	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$

Table 38. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

– $\mathbf{H}_{25}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{24}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes the obfuscation $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In particular, we compute:

- $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)\})$,
- $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)\})$

and the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1 where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z > z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
$z = z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
$z < z^*$	$\mathbf{M_0}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$

Table 39. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

– $\mathbf{H}_{26}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{25}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the challenger samples $r_{i^*,j^*}, s_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ and the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ s_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

Moreover we set $v_2 = r_{i^*,j^*}$ and $x_2 = s_{i^*,j^*}$, and if $j^* > 0$ we set $v_1 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_1 - x_m}{x_j - x_m}$ (where we recall $x_1 = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*))$ from hybrid $\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)$, while if $j^* = 0$ we define $K_{i^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0))$ consistently as Lagrange interpolation of the resulting polynomial.

– $\mathbf{H}_{27}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{26}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the point $x_{j^*} = s_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ is replaced by $x_{j^*} = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0 j^*) \\ r_{i^*,j^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*+1, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

Moreover we set $x_2 = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$, and if $j^* > 0$ we set $v_1 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_1 - x_m}{x_j - x_m}$ (where we recall $x_1 = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*))$ from hybrid $\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)$, while if $j^* = 0$ we define $K_{i^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0))$ consistently as Lagrange interpolation of the resulting polynomial.

– $\mathbf{H}_{28}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{27}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes $f_{i^*}(x)$. In particular, the evaluation $y_{j^*} = r_{i^*,j^*} \xleftarrow{\$} \mathbb{Z}_p$ is replaced by $y_{j^*} = \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$. Specifically, the polynomial f_{i^*} is computed by interpolating the following t_{i^*} points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

over

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^*, t_{i^*}) \end{cases}$$

Moreover we set $v_2 = \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$, and if $j^* > 0$ we set $v_1 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_1 - x_m}{x_j - x_m}$ (where we recall $x_1 = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*))$ from hybrid $\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)$, while if $j^* = 0$ we define $K_{i^*} = f_{i^*}(\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0))$ consistently as Lagrange interpolation of the resulting polynomial.

– $\mathbf{H}_{29}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{28}^{i^*,j^*}(\lambda, b)$, except that we change how the challenger computes the obfuscation $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In particular, the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1 where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z > z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	j^*-1	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
$z = z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	0	$\perp$	j^*-1	w^*	w^*-1
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
$z < z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$

Table 40. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

– $\mathbf{H}_{30}^{i^*,j^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}, j^* \in [W^*_{i^*,i^*}, 0]$: This is identical to $\mathbf{H}_{29}^{i^*,j^*}(\lambda, b)$, except that the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_{z^*}$ is replaced by M_0 instead of M_1 where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$z > z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	j^*-1	0	0
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	x_2	v_2	j^*-1	0	0
$z = z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	j^*-1	0	w^*-1
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	x_1	$\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$	x_2	v_2	j^*-1	w^*	w^*-1
$z < z^*$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$
	$\mathbf{M_1}$	$K_{f,*}$	$K_{p,*}$	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	W^*_{z,i^*}	0	$\mathsf{weight}_{i^*}(z)$

Table 41. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

$\mathbf{H}_{31}^{i^*}(\lambda, b)$ **for** $i^* \in \mathcal{U}$: This is identical to $\mathbf{H}_{30}^{i^*,0}(\lambda, b)$, except that we change how the challenger computes the obfuscation $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$. In

particular, the machine $M_p^{\mathsf{dw-share}}$ we use to compute $\widetilde{M}_z$ for all the parties $z \in \mathcal{U}$ is replaced by M_0 instead of M_1 where:

Case	M	K_f	K_p	K'_p	hk	W^*	i	pos	t^*	x_1	v_1	x_2	v_2	l^*	q_1^*	q_2^*
$\forall z$	$\mathbf{M_0}$	K_f	K_p	K'_p	hk	0	z	0	0	0	0	0	$\perp$	0	0	0
	$\mathbf{M_1}$	K_f	K_p	K'_p	hk	W^*_{z-1,i^*}	z	i^*	t_{i^*}	0	0	0	$\perp$	0	0	0

Table 42. $M_p^{\mathsf{dw-share}}$ for all $z \in \mathcal{U}$

Lemma 52. *For every $i^* \in \mathcal{U}$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{31}^{i^*-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_1^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The proof uses the hybrid argument. In particular, for every $u \in [0, |\mathcal{U}|]$, let $\mathbf{H}_1^{i^*,u}(\lambda, b)$ be the experiment where we change how the challenger computes the shares σ_z for $z \in \mathcal{U}|_u$. Clearly, $\{\mathbf{H}_1^{i^*,0}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{31}^{i^*-1}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_1^{i^*,|\mathcal{U}|}(\lambda, b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_1^{i^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the lemma, it suffices to show that, for all $u \in [|\mathcal{U}|]$, we have $\{\mathbf{H}_1^{i^*,u-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_1^{i^*,u}(\lambda, b)\}_{\lambda\in\mathbb{N}}$. The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_1^{i^*,u-1}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_1^{i^*,u}(\lambda, b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_z with z being the u-th party in $\mathcal{U}$.

Let us argue that M_0 and M_1 are functionally equivalent.

Assuming that there is an input (k, t, h, π, w, q) for which M_0, M_1 produce different outputs, then $k \geq z$, $t > 0$, $\mathsf{Verify}(\mathsf{hk}, h, z, w, \pi) \neq 0$ and $0 < q \leq w$ must hold. Otherwise they both output $\perp$. We distinguish 2 cases:

- $k = i^*$ and $t = t_{i^*}$: In this case, for every $0 < q \leq w$ in M_0 we do not enter any different branch of code compared to M_1, since no condition is fully satisfied, i.e., $q \neq q_1^*$ and $q > q_2^*$ (recall that $q_1^* = q_2^* = 0$); moreover $j \geq l^*$ (recall that $l^* = 0$), and $v_2 = \perp$. So both M_0 and M_1 will output $(\mathsf{Eval}(K_p, k||t||h||(2^{z\lambda} + q)), S)$.
- $k \neq i^*$ or $t \neq t_{i^*}$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_p, k||t||h||(2^{z\lambda} + q)), S)$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\mathbf{H}_1^{i^*,u-1}(\lambda, b)$ and $\mathbf{H}_1^{i^*,u}(\lambda, b)$ are indistinguishable. It follows that $\mathbf{H}_1^{i^*}(\lambda, b)$ and $\mathbf{H}_{31}^{i^*-1}(\lambda, b)$ are indistinguishable.

Lemma 53. *For every $i^* \in \mathcal{U}$, $j^* \in [0, W^*_{i^*,i^*}]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{14}^{i^*,j^*-1}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_2^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. First of all, we note that for $j^* = 0$, $\mathbf{H}_{14}^{i^*,j^*-1}(\lambda, b) = \mathbf{H}_2^{i^*,j^*}(\lambda, b)$. For $j^* > 0$, the lemma follows by SSB being index hiding (Definition 5). We can show how to turn any distinguisher D between the two hybrids into an adversary attacking the index hiding of SSB as follows:

– The reduction receives two messages μ_0, μ_1, an integer n and an unauthorized subset $\mathcal{U} \notin \hat{\mathcal{A}}_n^{\mathsf{dw-ethr}}$ from the $\boldsymbol{G}_{\mathsf{SS,A}}^{\mathsf{priv}}(\lambda, b)$ adversary D;
– The reduction sends to the SSB challenger the integer 2^λ and the indices $i_0 = z^* - 1$ and $i_1 = z^*$;
– The SSB challenger samples a random bit b, runs $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, 2^\lambda, i_b)$, and returns hk to the reduction;
– The reduction proceeds computing:
  • $K_f \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$,
  • $K_p \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$,
  • $K'_p \xleftarrow{\$} \mathsf{KeyGen}(1^\lambda)$.
– Then, the reduction proceeds computing the shares $(\sigma_i)_{i\in[n]}$ as follows:
  • If $i = i^*$: the dealer computes $h_{i^*} = \mathsf{Hash}(\mathsf{hk},(\mathsf{weight}_{i^*}(i))_{i\in[i^*]})$ and $K_{i^*} = \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||0)$ Then the dealer sets $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu$. Then, if $i^* = z^*$, the dealer sets:

    $$M_{i^*} = M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, W^*_{i^*-1,i^*}, i^*, i^*, t_{i^*}, 0, 0, 0, \perp, j^*, 0, w^*-1],$$

    otherwise it sets:

    $$M_{i^*} = M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, W^*_{i^*-1,i^*}, i^*, i^*, t_{i^*}, 0, 0, 0, \perp, j^*, 0, 0].$$

    Finally, $\sigma_{i^*} = (\mathsf{hk}, \mathsf{ct}_{i^*}, \widetilde{M}_{i^*}, x_{i^*,0})$, where $x_{i^*,0} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||0)$.
  • If $i < i^*$: the dealer sets $\mathsf{ct}_i \xleftarrow{\$} \mathbb{Z}_p$. Then, if $i = z^*$, the dealer sets:

    $$M_i = M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, 0, 0, 0, \perp, j^*, 0, w^* - 1],$$

    otherwise if $i < z^*$ it sets:

    $$M_i = M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, 0, 0, 0, \perp, W^*_{i,i^*}, 0, \mathsf{weight}_{i^*}(i)],$$

    otherwise if $i > z^*$ it sets:

    $$M_i = M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, 0, 0, 0, \perp, j^*, 0, 0],$$

    and finally $\sigma_i = (\mathsf{hk}, \mathsf{ct}_i, \widetilde{M}_i, x_{i,0})$ where $x_{i,0} = \mathsf{Eval}(K_p, i||t_i||h_i||0)$.
  • If $i > i^*$: the dealer computes $h_i = \mathsf{Hash}(\mathsf{hk},(\mathsf{weight}_i(x))_{x\in[i]})$, and $K_i = f_i(\mathsf{Eval}(K_p, i||t_i||h_i||0))$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_f, i||t_i||h_i||(2^\lambda - j))$ at the points $\mathsf{Eval}(K_p, i||t_i||h_i||(2^\lambda - j))$ for all $j \in [0, t_i)$ and sets $\mathsf{ct}_i = K_i \cdot \mu$. Then, the dealer sets:

    $$M_i = M_p^{\mathsf{dw-share}}[K_f, K_p, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, 0, 0, 0, \perp, j^*, 0, 0],$$

    and finally $\sigma_i = (\mathsf{hk}, \mathsf{ct}_i, \widetilde{M}_i, x_{i,0})$, where $x_{i,0} = \mathsf{Eval}(K_p, i||t_i||h_i||0)$.
– The reduction forwards $\{\sigma_u\}_{u\in\mathcal{U}}$ to D.
– Finally, the reduction outputs whatever D outputs.

It follows that $\mathbf{H}_{14}^{i^*,j^*-1}(\lambda,b)$ and $\mathbf{H}_{2}^{i^*,j^*}(\lambda,b)$ are indistinguishable.

Lemma 54. *For every $i^* \in \mathcal{U}$, $j^* \in [0, W^*_{i^*,i^*}]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{2}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{3}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. First of all, we note that for $j^* = 0$, $\mathbf{H}_{2}^{i^*,j^*}(\lambda,b) = \mathbf{H}_{3}^{i^*,j^*}(\lambda,b)$. For $j^* > 0$, the lemma follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_{3}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{2}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_{z^*} with $z^* = \min(u \in \mathcal{U} \mid W^*_{u,i^*} > j^*)$.

Let us argue that M_0 and M_1 are functionally equivalent.

Assuming that there is an input (k,t,h,π,w,q) for which M_0, M_1 produce different outputs, then $k \geq z^*$, $t > 0$, $\mathsf{Verify}(\mathsf{hk},h,z^*,w,\pi) \neq 0$ and $0 < q \leq w$ must hold. Otherwise they both output $\perp$. We distinguish 3 cases:

- $k = i^*$, $t = t_{i^*}$, and $q = w^*$: In this case, in M_0 we output (x_1, v_1), which corresponds to $(\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*)), \mathsf{sh}^{t_{i^*}}_{z^*,w^*})$, while in M_1 we output $(\mathsf{Eval}(K_p, k||t||h||(2^{z^*\lambda}+q)), S)$. We note that $\mathsf{hk} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{block}}}, 2^\lambda, z^*)$ and $h_{i^*} = \mathsf{Hash}(\mathsf{hk}, (\mathsf{weight}_{i^*}(i))_{i\in[i^*]})$ are honestly generated by the reduction and so since SSB is statistically binding, we know that $w = w_{z^*}$. This means that $\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$ and S are computed exactly at the same way, i.e., as the Lagrange interpolation in $\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda}+w^*))$ of a degree $t_{i^*}-1$ polynomial evaluated at the t_{i^*} distinct points:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j < j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \geq j^* \end{cases}$$

and

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j < j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \geq j^* \end{cases}$$

for $j \in [0, t_{i^*})$.
- $k = i^*$, $t = t_{i^*}$, and $q \leq w^* - 1$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K'_p, k||t||h||(W^*_{z^*-1,i^*} + q)), \mathsf{Eval}(K_f, k||t||h||(W^*_{z^*-1,i^*} + q)))$.
- $k \neq i^*$ or $t \neq t_{i^*}$ or $q > w^*$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_p, k||t||h||(2^{z^*\lambda} + q)), S)$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\{\mathbf{H}_{3}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{2}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ are indistinguishable.

Lemma 55. *For every $i^* \in \mathcal{U}$, $j^* \in [0, W^*_{i^*,i^*}]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{3}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{4}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The proof uses the hybrid argument. In particular, for every $u \in [0, |\mathcal{U}|]$, let $\mathbf{H}_{4}^{i^*,j^*,u}(\lambda,b)$ be the experiment where we change how the challenger computes the shares σ_z for $z \in \mathcal{U}|_u$. Clearly, $\{\mathbf{H}_{4}^{i^*,j^*,0}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{3}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{4}^{i^*,j^*,|\mathcal{U}|}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{4}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the

lemma, it suffices to show that, for all $u \in [|\mathcal{U}|]$, we have $\{\mathbf{H}_4^{i^*,j^*,u-1}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_4^{i^*,j^*,u}(\lambda,b)\}_{\lambda\in\mathbb{N}}$. The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_4^{i^*,j^*,u-1}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_4^{i^*,j^*,u}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_z with z being the u-th party in $\mathcal{U}$.

Let us argue that M_0 and M_1 are functionally equivalent.

Assuming that there is an input (k,t,h,π,w,q) for which M_0, M_1 produce different outputs, then $k \geq z$, $t > 0$, $\mathsf{Verify}(\mathsf{hk},h,z,w,\pi) \neq 0$ and $0 < q \leq w$ must hold. Otherwise they both output $\perp$. We distinguish 2 major cases:

- $k = i^*$ and $t = t_{i^*}$:
  - If $z < z^*$: In this case, for every $0 < q \leq w$ neither in M_0 or in M_1 $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ and $\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ are never called, as both machines output $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*} + q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*}+q)))$. Notice that $2^\lambda - j^*$ and $W^*_{z-1,i^*} + q$ never intersect, since by assumption $W_n = \mathsf{poly}(\lambda)$.
  - If $z = z^*$: In this case, neither in M_0 or in M_1 $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ and $\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ are never called, as both machines output:
    - (x_1, v_1) for $q = w^*$,
    - $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*} + q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*} + q)))$ for $0 < q < w^*$,
    - $(\mathsf{Eval}(K_p, k||t||h||(2^{z\lambda}+q)), S)$ for $q > w^*$, where $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ and $\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ are already set within x_2 and v_2;
  - If $z > z^*$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_p, k||t||h||(2^{z\lambda}+q)), S)$, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ or $\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$, whose values are already set within x_2 and v_2;
- $k \neq i^*$ or $t \neq t_{i^*}$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_p, k||t||h||(2^{z\lambda}+q)), S)$, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ or $\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\mathbf{H}_4^{i^*,j^*,u-1}(\lambda,b)$ and $\mathbf{H}_4^{i^*,j^*,u}(\lambda,b)$ are indistinguishable. It follows that $\mathbf{H}_4^{i^*,j^*}(\lambda,b)$ and $\mathbf{H}_3^{i^*,j^*}(\lambda,b)$ are indistinguishable.

Lemma 56. *For every $i^* \in \mathcal{U}$, $j^* \in [0, W^*_{i^*,i^*}]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_4^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_5^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)$. We can show how to turn any distinguisher D between the two hybrids into an adversary attacking the pseudorandomness at punctured points of PPRF (Definition 2) as follows:

- The reduction receives the messages $\mu_0, \mu_1 \in \mathcal{M}$, the integer n and the unqualified subset $\mathcal{U} \notin \hat{\mathcal{A}}_n^{\mathsf{dw-ethr}}$ from the $\boldsymbol{G}^{\mathsf{priv}}_{\mathsf{SS},\mathsf{D}}$ adversary D.

– The reduction sends to the PPRF security game the point $i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)$ and receives $K^* = \mathsf{Punc}(K, \{i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)\})$ and either a uniform value $\bar{u} \xleftarrow{\$} \mathbb{Z}_p$ or the value $\bar{u} = \mathsf{Eval}(K, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$, where K is the PPRF key generated by the PPRF challenger.
– The reduction proceeds computing:
  - $x_1 = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*))$,
  - $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)\})$,
  - $x_2 = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$.

  and then the shares for parties $i \in [n]$:
  - When computing σ_{i^*}: the dealer computes $f_{i^*}(x)$ as the unique polynomial of degree $t_{i^*} - 1$ obtained by interpolating the values:

$$y_j = \begin{cases} \mathsf{Eval}(K^*, i^*||t_{i^*}||h_{i^*}||j), \text{ if } j < j^* \\ \bar{u} \text{ if } j = j^* \\ \mathsf{Eval}(K^*, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j > j^* \end{cases}$$

    at the points

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j < j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \geq j^* \end{cases}$$

    and

$$x_{i^*,0} = \begin{cases} \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0) \text{ if } j^* = 0 \\ \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||0) \text{ otherwise} \end{cases}$$

    and

$$K_{i^*} = \begin{cases} \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_{i^*,0} - x_m}{x_j - x_m} \text{ if } j^* = 0 \\ \mathsf{Eval}(K^*, i^*||t_{i^*}||h_{i^*}||0) \text{ otherwise} \end{cases},$$

    and sets $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu_b$. Then, if $j^* > 0$, the challenger computes:

$$\mathsf{sh}^{t_{i^*}}_{z^*,w^*} = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*)) - x_m}{x_j - x_m},$$

    and if $i^* = z^*$ sets:

$$M_{i^*} = M_p^{\mathsf{dw-share}}[K^*, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i^*-1,i^*}, i^*, i^*, t_{i^*}, x_1, \mathsf{sh}^{t_{i^*}}_{z^*,w^*}, x_2, \bar{u}, j^*, w^*, w^* - 1],$$

    otherwise, if $i^* \neq z^*$, it sets:

$$M_{i^*} = M_p^{\mathsf{dw-share}}[K^*, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i^*-1,i^*}, i^*, i^*, t_{i^*}, 0, 0, x_2, \bar{u}, j^*, 0, 0].$$

    Finally, $\sigma_{i^*} = (\mathsf{hk}, \mathsf{ct}_{i^*}, \widetilde{M}_{i^*}, x_{i^*,0})$.

- When computing $(\sigma_i)_{i<i^*}$: the challenger sets $\mathsf{ct}_i \xleftarrow{\$} \mathbb{Z}_p$. Then, if $i = z^*$, the dealer sets:

$$M_i = M_p^{\mathsf{dw-share}}[K^*, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, x_1, \mathsf{sh}^{t_{i^*}}_{z^*,w^*}, x_2, \bar{u}, j^*, w^*, w^*-1],$$

otherwise if $i < z^*$ it sets:

$$M_i = M_p^{\mathsf{dw-share}}[K^*, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, 0, 0, 0, \perp, W^*_{i,i^*}, 0, \mathsf{weight}_{i^*}(i)],$$

otherwise if $i > z^*$ it sets:

$$M_i = M_p^{\mathsf{dw-share}}[K^*, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, 0, 0, x_2, \bar{u}, j^*, 0, 0],$$

and finally $\sigma_i = (\mathsf{hk}, \mathsf{ct}_i, \widetilde{M}_i, x_{i,0})$ with $x_{i,0} = \mathsf{Eval}(K'_p, i||t_i||h_i||0)$.
- When computing $(\sigma_i)_{i>i^*}$: the dealer computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K^*, i||t_i||h_i||(2^\lambda - j))$ at the points $\mathsf{Eval}(K_p, i||t_i||h_i||(2^\lambda - j))$ for all $j \in [0, t_i)$ and sets $\mathsf{ct}_i = K_i \cdot \mu_b$. Then, the dealer sets:

$$M_i = M_p^{\mathsf{dw-share}}[K^*, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, 0, 0, x_2, \bar{u}, j^*, 0, 0],$$

and finally $\sigma_i = (\mathsf{hk}, \mathsf{ct}_i, \widetilde{M}_i, x_{i,0})$, with $x_{i,0} = \mathsf{Eval}(K_p, i||t_i||h_i||0)$.

– The reduction forwards $\{\sigma_u\}_{u \in \mathcal{U}}$ to D.
– Finally, the reduction outputs whatever D outputs.

It follows that $\mathbf{H}_4^{i^*,j^*}(\lambda, b)$ and $\mathbf{H}_5^{i^*,j^*}(\lambda, b)$ are indistinguishable.

Lemma 57. *For every $i^* \in \mathcal{U}$, $j^* \in [0, W^*_{i^*,i^*}]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_5^{i^*,j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}} \approx_c \{\mathbf{H}_6^{i^*,j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)$. The reduction for showing $\{\mathbf{H}_5^{i^*,j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}} \approx_c \{\mathbf{H}_6^{i^*,j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}}$ is similar to the one used in Lemma 56 where K^* and $\bar{u}$ are used as $K_{p,*}$ and x_2 (and hence x_{j^*}) respectively (instead of $K_{f,*}$ and v_2), while the point v_2 (and hence y_{j^*} is randomly sampled). We omit it for brevity.

Lemma 58. *For every $i^* \in \mathcal{U}$, $j^* \in [0, W^*_{i^*,i^*}]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_6^{i^*,j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}} \equiv \{\mathbf{H}_7^{i^*,j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}}$.*

Proof. The lemma follows by Lemma 1. We can show how to turn any distinguisher D between the two hybrids into an adversary attacking the security of Lemma 1 as follows:

– The reduction receives the messages $\mu_0, \mu_1 \in \mathcal{M}$, the integer n and the unqualified subset $\mathcal{U} \notin \hat{\mathcal{A}}_n^{\mathsf{dw-ethr}}$ from the $\boldsymbol{G}^{\mathsf{priv}}_{\mathsf{SS},\mathsf{D}}$ adversary D.

– The reduction sends to the security game of Lemma 1:
  - $n = n$;
  - $t = t_{i^*}$;
  - $\mathcal{I} = \{x_j \mid x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$ with $|\mathcal{I}| = t_{i^*} - 1$;
  - $i_{t-1} = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$;
  - $i^* = \begin{cases} \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0) \text{ if } j^* = 0 \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*)) \text{ otherwise} \end{cases}$
  - $\mathcal{Y} = \{y_j \mid y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$ with $|\mathcal{Y}| = t_{i^*} - 1$;

– The Lemma 1 challenger randomly samples y_{t-1} and computes r_b either as a random value or as $f(i^*) = \sum_{j=0}^{t-1} y_j \ell_j(i^*)$, where $\ell_j(x) = \prod_{\substack{0 \le m \le t-1 \\ m \ne j}} \frac{x - x_m}{x_j - x_m}$. Then, it forwards r_b to the reduction.

– The reduction proceeds computing:
  - $K_{p,*} = \mathsf{Punc}(K_p, \{i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)\})$,
  - $K_{f,*} = \mathsf{Punc}(K_f, \{i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)\})$,
  - $x_1 = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*))$

  and then the shares for parties $i \in [n]$:
  - When computing σ_{i^*}: the dealer computes $f_{i^*}(x)$ as the unique polynomial of degree $t_{i^*} - 1$ obtained by interpolating the values:

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ r_b \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

    at the points

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0) \text{ if } j = j^* = 0 \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*)) \text{ if } j = j^* \ne 0 \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

    Then the challenger sets:

$$K_{i^*} = \begin{cases} r_b \text{ if } j^* = 0 \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||0) \text{ otherwise} \end{cases}$$

    and, if $j^* > 0$, $\mathsf{sh}^{t_{i^*}}_{z^*, w^*} = r_b$. Then, the challenger sets

$$v_2 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \ne j}}^{t_{i^*}-1} \frac{x_2 - x_m}{x_j - x_m},$$

where we recall $x_2 \xleftarrow{\$} \mathbb{Z}_p$ from previous hybrid.
Then, the challenger sets $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu_b$. Finally, if $i^* = z^*$, the challenger sets:

$$M_{i^*} = M_p^{\mathsf{dw-share}}[K_{f,*}, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i^*-1,i^*}, i^*, i^*, t_{i^*}, x_1, \mathsf{sh}^{t_{i^*}}_{z^*,w^*}, x_2, v_2, j^*, w^*, w^*-1],$$

otherwise it sets:

$$M_{i^*} = M_p^{\mathsf{dw-share}}[K_{f,*}, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i^*-1,i^*}, i^*, i^*, t_{i^*}, 0, 0, x_2, v_2, j^*, 0, 0].$$

Finally, $\sigma_{i^*} = (\mathsf{hk}, \mathsf{ct}_{i^*}, \widetilde{M}_{i^*}, x_{i^*,0})$, where $x_{i^*,0} = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0)$ if $j^* = 0$, while $x_{i^*,0} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||0)$ if $j^* > 0$.

- When computing $(\sigma_i)_{i<i^*}$: the challenger sets $\mathsf{ct}_i \xleftarrow{\$} \mathbb{Z}_p$. Then, if $i = z^*$, the dealer sets:

$$M_i = M_p^{\mathsf{dw-share}}[K_{f,*}, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, x_1, r_b, x_2, v_2, j^*, w^*, w^*-1],$$

otherwise if $i > z^*$ it sets:

$$M_i = M_p^{\mathsf{dw-share}}[K_{f,*}, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, 0, 0, x_2, v_2, j^*, 0, 0],$$

otherwise if $i < z^*$, it sets:

$$M_i = M_p^{\mathsf{dw-share}}[K_{f,*}, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, 0, 0, 0, \perp, W^*_{i,i^*}, 0, \mathsf{weight}_{i^*}(i)],$$

and finally $\sigma_i = (\mathsf{hk}, \mathsf{ct}_i, \widetilde{M}_i, x_{i,0})$, where $x_{i,0} = \mathsf{Eval}(K_p, i||t_i||h_i||0)$.
- When computing $(\sigma_i)_{i>i^*}$: the dealer computes $K_i = f_i(0)$ where $f_i(x)$ is the unique polynomial of degree $t_i - 1$ obtained by interpolating the evaluations $\mathsf{Eval}(K_f, i||t_i||h_i||(2^\lambda - j))$ at the points $\mathsf{Eval}(K_p, i||t_i||h_i||(2^\lambda - j))$ for all $j \in [0, t_i)$ and sets:
  * $\mathsf{ct}_i = K_i \cdot \mu_b$,
  * $M_i = M_p^{\mathsf{dw-share}}[K_{f,*}, K_{p,*}, K'_p, \mathsf{hk}, W^*_{i-1,i^*}, i, i^*, t_{i^*}, 0, 0, x_2, v_2, j^*, 0, 0]$

  and finally $\sigma_i = (\mathsf{hk}, \mathsf{ct}_i, \widetilde{M}_i, x_{i,0})$, where $x_{i,0} = \mathsf{Eval}(K_p, i||t_i||h_i||0)$.

– The reduction forwards $\{\sigma_u\}_{u \in \mathcal{U}}$ to D.
– Finally the reduction outputs whatever D outputs.

It follows that $\mathbf{H}_6^{i^*,j^*}(\lambda, b) \equiv \mathbf{H}_7^{i^*,j^*}(\lambda, b)$.

Lemma 59. *For every $i^* \in \mathcal{U}$, $j^* \in [0, W^*_{i^*,i^*}]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_7^{i^*,j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}} \approx_c \{\mathbf{H}_8^{i^*,j^*}(\lambda, b)\}_{\lambda \in \mathbb{N}}$.*

Proof. The lemma follows by security of Obf, since, as shown in Lemma 55, $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ and $\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ are never called. We omit the proof here for brevity.

Lemma 60. *For every $i^* \in \mathcal{U}$, $j^* \in [0, W^*_{i^*,i^*}]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_8^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_9^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The proof uses the hybrid argument. In particular, for every $u \in [0, |\mathcal{U}|]$, let $\mathbf{H}_9^{i^*,j^*,u}(\lambda,b)$ be the experiment where we change how the challenger computes the shares σ_z for $z \in \mathcal{U}|_u$. Clearly, $\{\mathbf{H}_9^{i^*,j^*,0}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_8^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_9^{i^*,j^*,|\mathcal{U}|}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_9^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the lemma, it suffices to show that, for all $u \in [|\mathcal{U}|]$, we have $\{\mathbf{H}_9^{i^*,j^*,u-1}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_9^{i^*,j^*,u}(\lambda,b)\}_{\lambda\in\mathbb{N}}$. The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_9^{i^*,j^*,u-1}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_9^{i^*,j^*,u}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_z with z being the u-th party in $\mathcal{U}$.

Let us argue that M_0 and M_1 are functionally equivalent.

Assuming that there is an input (k,t,h,π,w,q) for which M_0, M_1 produce different outputs, then $k \geq z$, $t > 0$, $\mathsf{Verify}(\mathsf{hk},h,z,w,\pi) \neq 0$, and $0 < q \leq w$ must hold. Otherwise they both output $\perp$. We distinguish 2 major cases:

- If $k = i^*$ and $t = t_{i^*}$:
  - If $z < z^*$: In this case, for every $0 < q \leq w$ neither in M_0 or in M_1 $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*)$ and $\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*)$ are never called, as both machines output $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*}+q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*}+q)))$, and for every $q \in [w]$ we have that $W^*_{z-1,i^*} + q \leq W^*_{z,i^*} \leq W^*_{z^*-1,i^*}$ (since $z < z^*$), and $W^*_{z^*-1,i^*} < j^*$ by definition, hence $W^*_{z-1,i^*} + q < j^*$ for every $q \in [w]$.
  - If $z = z^*$: In this case, neither in M_0 or in M_1 $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*)$ and $\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*)$ are never called, as both machines output:
    - (x_1, v_1) for $q = w^*$;
    - $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*}+q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*}+q)))$ for $0 < q < w^*$;
    - $(\mathsf{Eval}(K_p, k||t||h||(2^{z\lambda}+q)), S)$ for $w^* < q \leq w$, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*)$ or $\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*)$ because l^* is equal to j^* (hence we exit the while-loop);
  - If $z > z^*$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_p, k||t||h||(2^{z\lambda}+q)), S)$, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*)$ or $\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*)$ because l^* is equal to j^*.
- $k \neq i^*$ or $t \neq t_{i^*}$: In this case both M_0 and M_1 will output $(\mathsf{Eval}(K_p, k||t||h||(2^{z\lambda}+q)), S)$, without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*)$ or $\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*)$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\mathbf{H}_9^{i^*,j^*,u-1}(\lambda,b)$ and $\mathbf{H}_9^{i^*,j^*,u}(\lambda,b)$ are indistinguishable. It follows that $\mathbf{H}_9^{i^*,j^*}(\lambda,b)$ and $\mathbf{H}_8^{i^*,j^*}(\lambda,b)$ are indistinguishable.

Lemma 61. *For every $i^* \in \mathcal{U}$, $j^* \in [0, W^*_{i^*,i^*}]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_9^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{10}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||h_{i^*}||0$, if $j^* = 0$, and $i^*||t_{i^*}||h_{i^*}||(W_{z^*-1,i^*} + w^*)$ otherwise. The reduction

for showing $\{\mathbf{H}_9^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{10}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ is similar to the one used in Lemma 56, where K^* and $\bar{u}$ are used as $K_{p,*}$ and $x_{i^*,0}$ (if $j^*=0$) and as $K_{p,*}$ and x_1 (if $j^*>0$) (rather than $K_{f,*}$ and v_2). Moreover x_2 is sampled randomly and v_2 is computed as Lagrange interpolation. We omit it for brevity.

Lemma 62. *For every $i^*\in\mathcal{U}$, $j^*\in[0,W^*_{i^*,i^*}]$, and for every $b\in\{0,1\}$, it holds that $\{\mathbf{H}_{10}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{11}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||h_{i^*}||j^*$. The reduction for showing $\{\mathbf{H}_{10}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{11}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ is similar to the one used in Lemma 56, where K^* and $\bar{u}$ are used as $K'_{p,*}$ and $x_{i^*,0}$ (if $j^*=0$) and as $K'_{p,*}$ and x_1 (if $j^*>0$) (rather than $K_{f,*}$ and v_2). Moreover x_2 is sampled randomly and v_2 is computed as Lagrange interpolation. We omit it for brevity.

Lemma 63. *For every $i^*\in\mathcal{U}$, $j^*\in[0,W^*_{i^*,i^*}]$, and for every $b\in\{0,1\}$, it holds that $\{\mathbf{H}_{11}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{12}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||h_{i^*}||j^*$. The reduction for showing $\{\mathbf{H}_{11}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{12}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ is similar to the one used in Lemma 56 where K^* and $\bar{u}$ are used as $K_{f,*}$ and K_{i^*} (if $j^*=0$) and as $K_{f,*}$ and v_1 (if $j^*>0$) (rather than $K_{f,*}$ and v_2). Moreover x_2 is sampled randomly and v_2 is computed as Lagrange interpolation. We omit it for brevity.

Lemma 64. *For every $i^*\in\mathcal{U}$, $j^*\in[0,W^*_{i^*,i^*}]$, and for every $b\in\{0,1\}$, it holds that $\{\mathbf{H}_{12}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{13}^{i^*,j^*}(\lambda,b))\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 60 with M_0 and M_1 inverted.

Lemma 65. *For every $i^*\in\mathcal{U}$, $j^*\in[0,W^*_{i^*,i^*}]$, and for every $b\in\{0,1\}$, it holds that $\mathbf{H}_{13}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{14}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The proof uses the hybrid argument. In particular, for every $u\in[0,|\mathcal{U}|]$, let $\mathbf{H}_{14}^{i^*,j^*,u}(\lambda,b)$ be the experiment where we change how the challenger computes the shares σ_z for $z\in\mathcal{U}|_u$. Clearly, $\{\mathbf{H}_{14}^{i^*,j^*,0}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{13}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{14}^{i^*,j^*,|\mathcal{U}|}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{14}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the lemma, it suffices to show that, for all $u\in[|\mathcal{U}|]$, we have $\{\mathbf{H}_{14}^{i^*,j^*,u-1}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{14}^{i^*,j^*,u}(\lambda,b)\}_{\lambda\in\mathbb{N}}$. The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_{14}^{i^*,j^*,u-1}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{14}^{i^*,j^*,u}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_z with z being the u-th party in $\mathcal{U}$.

Let us argue that M_0 and M_1 are functionally equivalent.

Assuming that there is an input (k,t,h,π,w,q) for which M_0, M_1 produce different outputs, then $k\geq z$, $t>0$, $\mathsf{Verify}(\mathsf{hk},,h,z,w,\pi)\neq 0$, and $0<q\leq w$ must hold. Otherwise they both output $\perp$. We distinguish 2 major cases:

– $k = i^*$ and $t = t_{i^*}$:
  - If $z < z^*$, for any $0 < q \leq w$, both M_0 and M_1 outputs $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*} + q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*} + q)))$;
  - If $z = z^*$: if $q = w^*$, M_1 outputs (x_1, v_1) while M_0 outputs $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*} + q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*} + q)$, which are exactly the same values of x_1 and v_1; otherwise, if $q < w^*$, both M_0 and M_1 outputs $(\mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*}+q)), \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(W^*_{z-1,i^*}+ q)$; for $w^* < q \leq w$ the outputs are the same described in the next case;
  - If $z > z^*$: M_0 and M_1 differs on how they compute the j^*-th evaluation of the polynomial. In particular, in M_1 we use x_2 and v_2, while in M_0, since we incremented l^*, we compute the j^*-th evaluation within the while loop ($j < l^*$). Notice that

$$v_2 = \sum_{j=0}^{t_{i^*}-1} y_j \cdot \prod_{\substack{m=0 \\ m \neq j}}^{t_{i^*}-1} \frac{x_2 - x_m}{x_j - x_m}$$

    with

$$y_j = \begin{cases} \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ \mathsf{sh}^{t_{i^*}}_{z^*,w^*} \text{ if } j = j^* \\ \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

    and

$$x_j = \begin{cases} \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j) \text{ if } j \in [0, j^*) \\ x_1 \text{ if } j = j^* \\ \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j)) \text{ if } j \in [j^* + 1, t_{i^*}) \end{cases}$$

    where:
    * $x_1 = \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*)$,
    * $\mathsf{sh}^{t_{i^*}}_{z^*,w^*} = \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*)$.

    Hence, in M_1 the j^*-th evaluation is basically v_2, computed as Lagrange interpolation of a polynomial where $\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$ is one of the points, while in M_0, the j^*-th evaluation is directly $\mathsf{sh}^{t_{i^*}}_{z^*,w^*}$. It follows that M_0 and M_1 will output $(\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z\lambda} + q)), S)$ (with exactly the same S).
– $k \neq i^*$ and $t \neq t_{i^*}$: both M_0 and M_1 will output $(\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z\lambda} + q)), S)$.

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\mathbf{H}^{i^*,j^*,u-1}_{14}(\lambda, b)$ and $\mathbf{H}^{i^*,j^*,u}_{14}(\lambda, b)$ are indistinguishable. It follows that $\mathbf{H}^{i^*,j^*}_{13}(\lambda, b)$ and $\mathbf{H}^{i^*,j^*}_{14}(\lambda, b)$ are indistinguishable.

Lemma 66. *For every $i^* \in \mathcal{U}$ and for every $b \in \{0, 1\}$, it holds that $\{\mathbf{H}^{i^*,W^*_{i^*,i^*}}_{14}(\lambda, b)\}_{\lambda \in \mathbb{N}} \approx_c \{\mathbf{H}^{i^*}_{15}(\lambda, b)\}_{\lambda \in \mathbb{N}}$.*

Proof. The proof uses the hybrid argument. In particular, for every $u \in [0, |\mathcal{U}|]$, let $\mathbf{H}_{15}^{i^*,u}(\lambda, b)$ be the experiment where we change how the challenger computes the shares σ_z for $z \in \mathcal{U}|_u$. Clearly, $\{\mathbf{H}_{15}^{i^*,0}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{14}^{i^*,W^*_{i^*,i^*}-1}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{15}^{i^*,|\mathcal{U}|}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{15}^{i^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$, and thus in order to prove the lemma, it suffices to show that, for all $u \in [|\mathcal{U}|]$, we have $\{\mathbf{H}_{15}^{i^*,u-1}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{15}^{i^*,u}(\lambda,b)\}_{\lambda\in\mathbb{N}}$. The latter statement follows by security of Obf. Indeed, the only difference between $\{\mathbf{H}_{15}^{i^*,u-1}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ and $\{\mathbf{H}_{15}^{i^*,u}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ relies in the computation of the share σ_z with z being the u-th party in $\mathcal{U}$.

Let us argue that M_0 and M_1 are functionally equivalent.

Assuming that there is an input (k, t, h, π, w, q) for which M_0, M_1 produce different outputs, then $k \geq z$, $t > 0$, $\mathsf{Verify}(\mathsf{hk}, h, z, w, \pi) \neq 0$, and $0 < q \leq w$ must hold. Otherwise they both output $\perp$. We distinguish 2 major cases:

- If $k = i^*$ and $t = t_{i^*}$:
  - If $z \leq i^*$: In this case, for every $0 < q \leq w$, both M_0 and M_1 will output $(\mathsf{Eval}(K'_p, k||t||h||(W^*_{z-1,i^*}+q)), \mathsf{Eval}(K_f, k||t||h||(W^*_{z-1,i^*}+q)))$ without ever calling $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||0)$;
  - If $z > i^*$: Both M_0 and M_1 will abort;
- $k \neq i^*$ or $t \neq t_{i^*}$: In this case both M_0 and M_1 will never call $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||0)$;

It follows that there does not exist an input that makes M_0 and M_1 have a different output, thus $\mathbf{H}_{15}^{i^*,u-1}(\lambda,b)$ and $\mathbf{H}_{15}^{i^*,u}(\lambda,b)$ are indistinguishable. It follows that $\mathbf{H}_{15}^{i^*}(\lambda,b)$ and $\mathbf{H}_{14}^{i^*,W^*_{i^*,i^*}}(\lambda,b)$ are indistinguishable.

Lemma 67. *For every $i^* \in \mathcal{U}$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{15}^{i^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{16}^{i^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by observing that the two hybrids differs only in the share σ_{i^*}: in the former hybrid $\mathsf{ct}_{i^*} = K_{i^*} \cdot \mu_b$ is a one-time-pad encryption under a uniform key K_{i^*}, while in the latter ct_{i^*} is sampled uniformly at random, which is identically distributed to a one-time-pad encryption.

Lemma 68. *For every $i^* \in \mathcal{U}$ and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{16}^{i^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{17}^{i^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 66.

Lemma 69. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{30}^{i^*,j^*-1}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{18}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by SSB being index hiding (Definition 5). The reduction for showing $\{\mathbf{H}_{30}^{i^*,j^*-1}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{18}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ is analogous to the one used in Lemma 53. We omit it for brevity.

Lemma 70. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{18}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{19}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 65, with M_0 and M_1 inverted.

Lemma 71. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{19}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{20}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 64, with M_0 and M_1 inverted.

Lemma 72. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{20}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{21}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness at the punctured point $i^*||t_{i^*}||h_{i^*}||j^*$. The proof is analogous to that in Lemma 63, with $K_{i^*} \xleftarrow{\$} \mathbb{Z}_p$ and $K_{i^*} = \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||0)$ inverted (if $j^* = 0$), and with $v_1 \xleftarrow{\$} \mathbb{Z}_p$ and $v_1 = \mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||j^*)$ inverted (if $j^* > 0$).

Lemma 73. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{21}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{22}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness at the punctured point $i^*||t_{i^*}||h_{i^*}||j^*$. The proof is analogous to that in Lemma 62, with $x_{i^*,0} \xleftarrow{\$} \mathbb{Z}_p$ and $x_{i^*,0} = \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||0)$ inverted (if $j^* = 0$), and with $x_1 \xleftarrow{\$} \mathbb{Z}_p$ and $x_1 = \mathsf{Eval}(K'_p, i^*||t_{i^*}||h_{i^*}||j^*)$ inverted (if $j^* > 0$).

Lemma 74. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{22}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda}+w^*)$ (or $i^*||t_{i^*}||h_{i^*}||0$ if $j^* = 0$). The proof is analogous to that in Lemma 61 with $x_{i^*,0} = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||0)$ and $x_{i^*,0} \xleftarrow{\$} \mathbb{Z}_p$ inverted (if $j^* = 0$), and with $x_1 = \mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^{z^*\lambda} + w^*))$ and $x_1 \xleftarrow{\$} \mathbb{Z}_p$ inverted (if $j^* > 0$).

Lemma 75. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{23}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{24}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 60, with M_0 and M_1 inverted.

Lemma 76. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{24}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{25}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 59, with M_0 and M_1 inverted.

Lemma 77. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{25}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{26}^{i^*,j^*}(\lambda, b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Lemma 1. The reduction for showing $\{\mathbf{H}_{25}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{26}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$ is analogous to the one used in Lemma 58. We omit it for brevity.

Lemma 78. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{26}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{27}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)$. The proof is analogous to that in Lemma 57 with s_{i^*,j^*} and $\mathsf{Eval}(K_p, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ inverted.

Lemma 79. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{27}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{28}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by pseudorandomness of PPRF at the punctured point $i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*)$. The proof is analogous to that in Lemma 56 with r_{i^*,j^*} and $\mathsf{Eval}(K_f, i^*||t_{i^*}||h_{i^*}||(2^\lambda - j^*))$ inverted.

Lemma 80. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{28}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{29}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 55, with M_0 and M_1 inverted.

Lemma 81. *For every $i^* \in \mathcal{U}$, $j^* \in [W^*_{i^*,i^*}, 0]$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{29}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{30}^{i^*,j^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 54, with M_0 and M_1 inverted.

Lemma 82. *For every $i^* \in \mathcal{U}$, and for every $b \in \{0,1\}$, it holds that $\{\mathbf{H}_{30}^{i^*,0}(\lambda,b)\}_{\lambda\in\mathbb{N}} \approx_c \{\mathbf{H}_{31}^{i^*}(\lambda,b)\}_{\lambda\in\mathbb{N}}$.*

Proof. The lemma follows by security of Obf. The proof is analogous to that in Lemma 52, with M_0 and M_1 inverted.

Lemma 83. $\{\mathbf{H}_{31}^{|\mathcal{U}|}(\lambda,0)\}_{\lambda\in\mathbb{N}} \equiv \{\mathbf{H}_{31}^{|\mathcal{U}|}(\lambda,1)\}_{\lambda\in\mathbb{N}}$.

Proof. The lemma follows by simply observing that the distributions of the shares $(\sigma_u)_{u\in\mathcal{U}}$ is now independent of the message.